TRABAJO PRÁCTICO: DESARROLLO DE ALGORITMOS

1. Datos informativos

Asignatura:	Teoría de la programación
Docente:	José Oswaldo Guamán Quinche
Estudiante(s):	Brigith de los Angeles Toledo Jumbo
Curso / Paralelo:	1 A
Fecha de entrega:	03/11/2026
Tema:	Desarrollo de algoritmos: representación gráfica, pseudocódigo, prueba de escritorio y codificación en C

2. Objetivo

Desarrollar los algoritmos propuestos por el docente aplicando las distintas formas de representación y construcción de soluciones: diagrama de flujo, pseudocódigo, prueba de escritorio y su implementación en el lenguaje de programación C.

3. Estructura del trabajo

1. **Enunciado del algoritmo**

5 ejercicios avanzados sin repetir

1) EJ-366

En una empresa de telecomunicaciones, diseñe un algoritmo que calcule el salario neto de un trabajador. Lea el salario bruto (real), el número de cargas familiares (entero) y los años de antigüedad (entero). Use constantes simbólicas BONO_CARGA = 50.0, BONO_ANTIGUEDAD = 30.0, DESCUENTO_ISSS = 0.03, DESCUENTO_AFP = 0.0725, LIMITE_ANTIGUEDAD = 10. Calcule: bonos = cargas * BONO_CARGA + min (antigüedad, LIMITE_ANTIGUEDAD) * BONO_ANTIGUEDAD; descuentos = salario_bruto * (DESCUENTO_ISSS + DESCUENTO_AFP); salario_neto = salario_bruto + bonos - descuentos. Evalúe la expresión lógica (salario_neto > salario_bruto). Muestre por pantalla el desglose. Adicionalmente, calcule la mitad del resultado obtenido. Incluya una sección de conclusiones al final del algoritmo.

1.1. Análisis del problema

Datos de entrada:

- Bonos de carga y antigüedad
- Descuentos de ISSS y AFP
- Límite de antigüedad
- Salario bruto
- Número de cargas familiares
- Años de antigüedad

Proceso

- bonos = cargas * BONO_CARGA + anios_validos * BONO_ANTIGUEDAD
- descuentos = salario_bruto * (DESCUENTO_ISSS + DESCUENTO_AFP)
- salario_neto = salario_bruto + bonos - descuentos.
- mitad = salario_neto / 2
- neto = (salario_neto > salario_bruto)

Datos de salida:

- Total de los bonos
- Total de los descuentos
- Salario neto
- La mitad del salario neto
- Resultado de la expresión lógica
- Conclusiones

1.2. Diagrama de flujo

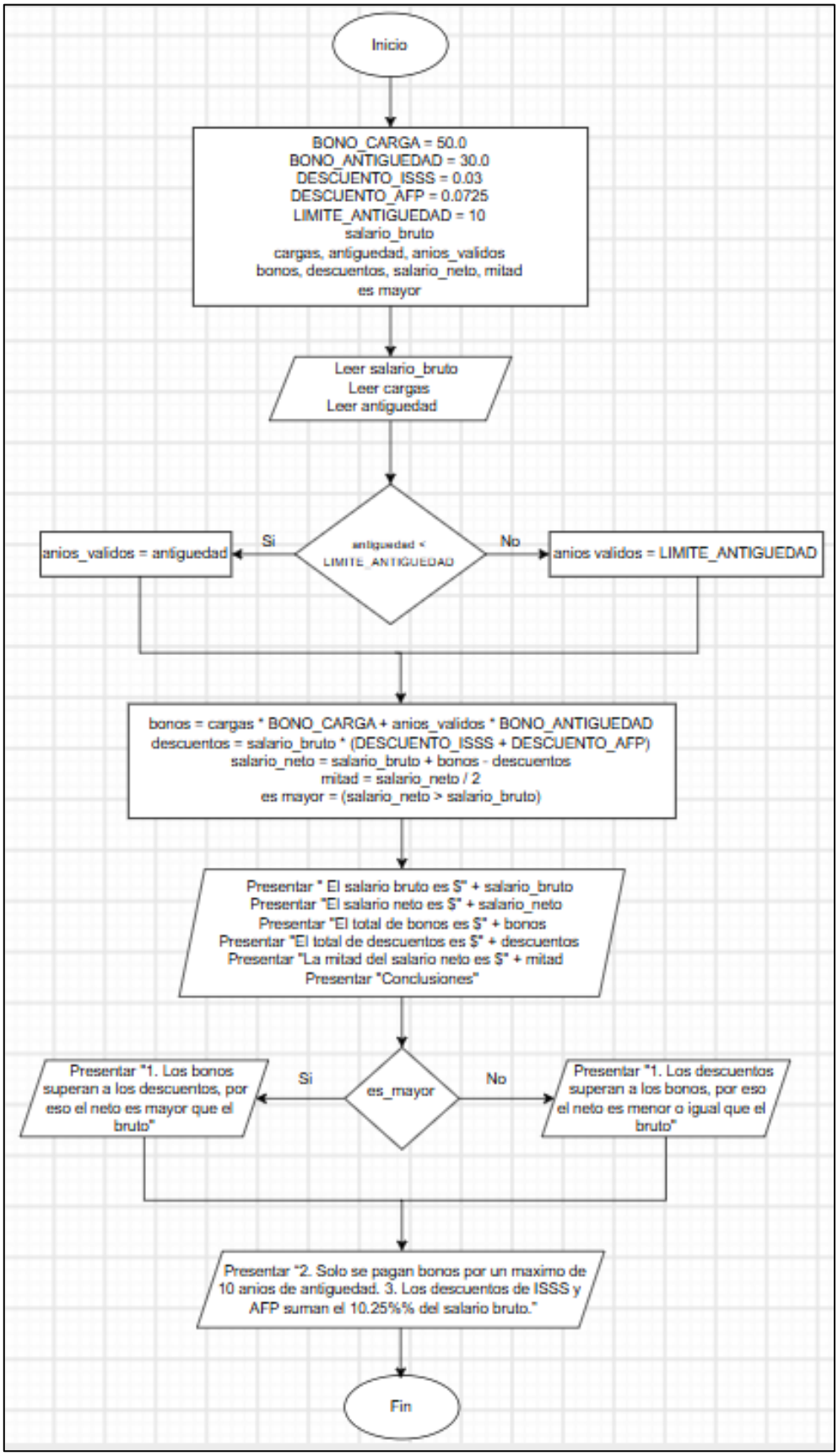


Figura 1: diagrama de flujo ejercicio 366 en Draw.io

1.3. Pseudocódigo

```
Inicio

BONO_CARGA = 50.0;
BONO_ANTIGUEDAD = 30.0;
DESCUENTO_ISSS = 0.03;
DESCUENTO_AFP = 0.0725;
LIMITE_ANTIGUEDAD = 10;
salario_bruto;
cargas, antiguedad, anios_validos;
bonos, descuentos, salario_neto, mitad;
es mayor;

Leer salario_bruto;
Leer cargas;
Leer antiguedad;

Si (antiguedad < LIMITE_ANTIGUEDAD) entonces
anios_validos = antigüedad;
Sino
anios validos = LIMITE_ANTIGUEDAD;
Fin Si

bonos = cargas * BONO_CARGA + anios_validos * BONO_ANTIGUEDAD;
descuentos = salario_bruto * (DESCUENTO_ISSS + DESCUENTO_AFP);
salario_neto = salario_bruto + bonos - descuentos;
mitad = salario_neto / 2;
es mayor = (salario_neto > salario_bruto);

Presentar " El salario bruto es $" + salario_bruto;
Presentar "El salario neto es $" + salario_neto;
Presentar "El total de bonos es $" + bonos;
Presentar "El total de descuentos es $" + descuentos;
Presentar "La mitad del salario neto es $" + mitad;
Presentar "Conclusiones";

Si (es_mayor) entonces
Presentar "1. Los bonos superan a los descuentos, por eso el neto es mayor que el bruto.";
Sino
Presentar "1. Los descuentos superan a los bonos, por eso el neto es menor o igual que el
bruto.";
Fin Si

Presentar "2. Solo se pagan bonos por un maximo de 10 anios de antiguedad. 3. Los
descuentos de ISSS y AFP suman el 10.25%% del salario bruto.";

Fin
```

Figura 2: pseudocódigo del ejercicio 366

1.4. Prueba de escritorio

Prueba 1: Antigüedad menor al límite y salario neto mayor que el bruto

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	salario_bruto cargas antiguedad		Ingrese el salario bruto: 600 Ingrese el numero de cargas familiares: 2 Ingrese los anios de antiguedad: 6
2	anios_validos = 6	(antiguedad < LIMITE_ANTIGUEDAD) (6 < 10) = TRUE	
3	bonos	(2*50 + 6*30) = 280	
4	descuentos	600 * (0.03 + 0.0725) = 61.5	
5	salario_neto	600 + 280 − 61.5 = 818.5	
6	mitad	818.5 / 2 = 409.25	
7	neto_mayor	(818.5 > 600) = TRUE	
8	Presentar	neto_mayor = TRUE	El salario bruto es: $ 600.00 El salario neto: $ 818.50 El total de bonos es: $ 280.00 El total de descuentos: $61.50 La mitad del salario neto es: $ 409.25 Conclusiones 1. Neto > Bruto: Verdadero 2. Los bonos superan a los descuentos, por eso el neto es mayor que el bruto. 3. Solo se pagan bonos por un maximo de 10 anios de antiguedad. 4. Los descuentos de ISSS y AFP suman el 10.25%% del salario bruto.

Prueba 2: Antigüedad mayor al límite y salario neto menor que el bruto

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	salario_bruto cargas antiguedad		Ingrese el salario bruto: 5000 Ingrese el numero de cargas familiares: 0 Ingrese los anios de antiguedad: 15

2	anios_validos = 15	(antiguedad < LIMITE_ANTIGUEDAD) (15 < 10) = FALSE anios_validos= LIMITE_ANTIGUEDAD	
3	bonos	(0*50 + 10*30) = 300	
4	descuentos	5000 * (0.03 + 0.0725) = 512.5	
5	salario_neto	5000 + 300 − 512.5 = 4787.5	
6	mitad	4787.5 / 2 = 2393.75	
7	neto_mayor	(4787.5 > 5000) = FALSE	
8	Presentar	neto_mayor = FALSE	El salario bruto es: $ 5000.00 El salario neto: $ 4787.50 El total de bonos es: $ 300.00 El total de descuentos: $ 512.50 La mitad del salario neto es: $ 2393.75 Conclusiones 1. Neto > Bruto: Falso 2. Los descuentos superan a los bonos, por eso el neto es menor o igual que el bruto. 3. Solo se pagan bonos por un maximo de 10 anios de antiguedad. 4. Los descuentos de ISSS y AFP suman el 10.25%% del

1.5. Codificación en lenguaje C

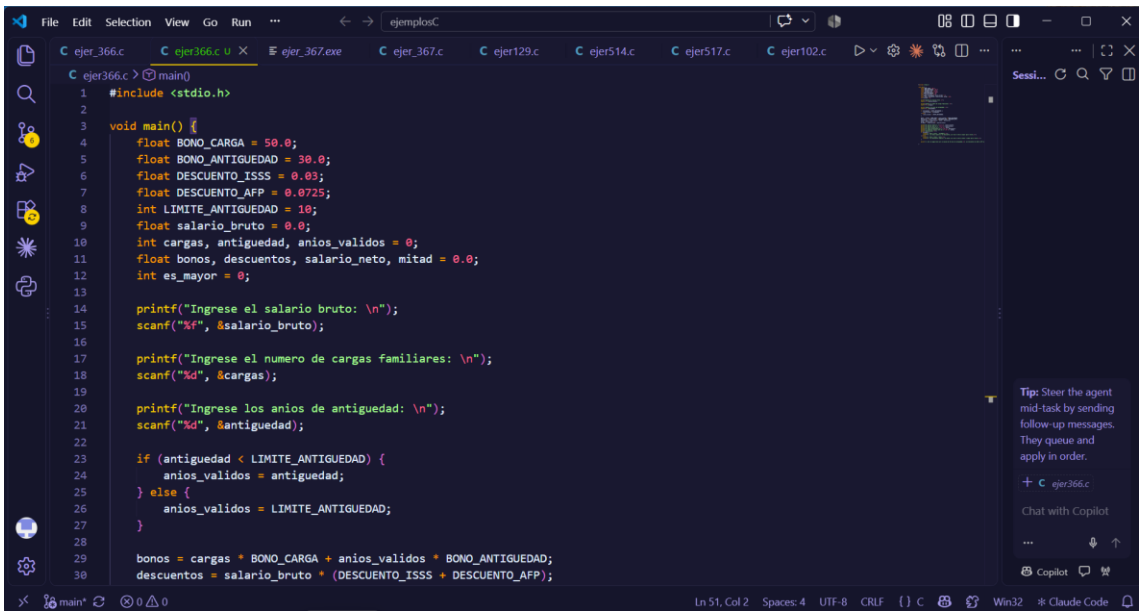


Figura 3: código del ejercicio 366, variables y condición.

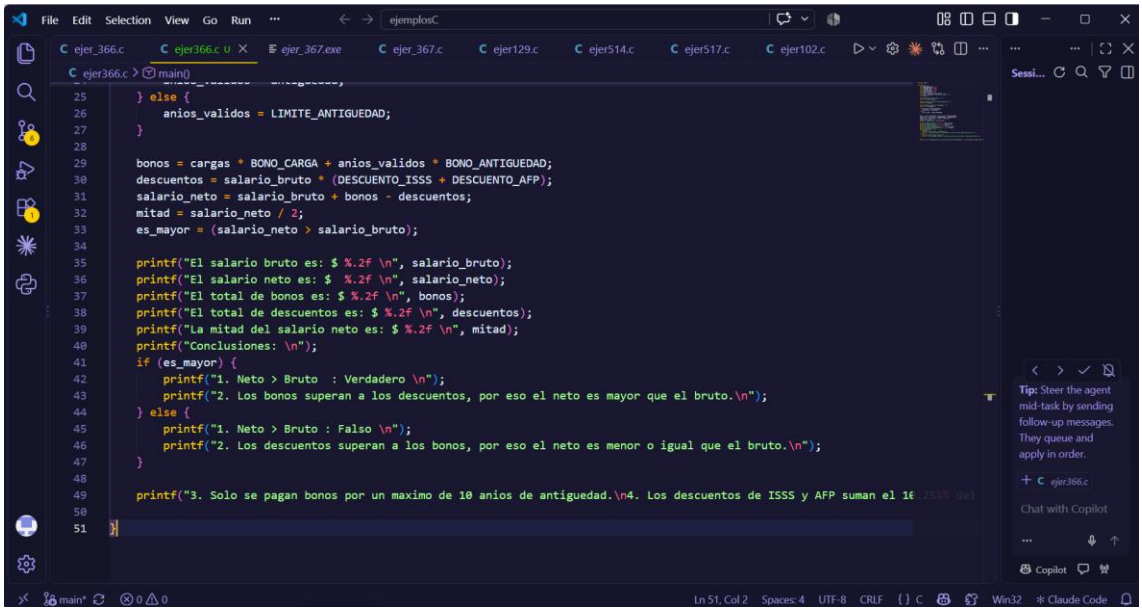


Figura 4: código del ejercicio 366, proceso y salida

1.6. Salida del programa

```
Problems  Output  Debug Console  Terminal  Ports

Microsoft Windows [Versión 10.0.26300.9711]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>gcc ejer366.c -o ejer366

C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>ejer366
Ingrese el salario bruto:
600
Ingrese el numero de cargas familiares:
2
Ingrese los anios de antiguedad:
6
El salario bruto es: $ 600.00
El salario neto es: $  818.50
El total de bonos es: $ 280.00
El total de descuentos es: $ 61.50
La mitad del salario neto es: $ 409.25
Conclusiones:
1. Neto > Bruto  : Verdadero
2. Los bonos superan a los descuentos, por eso el neto es mayor que el bruto.
3. Solo se pagan bonos por un maximo de 10 anios de antiguedad.
4. Los descuentos de ISSS y AFP suman el 10.25% del salario bruto.
```

Figura 5: comprobación de la prueba 1

```
C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>ejer366
Ingrese el salario bruto:
5000
Ingrese el numero de cargas familiares:
0
Ingrese los anios de antiguedad:
15
El salario bruto es: $ 5000.00
El salario neto es: $  4787.50
El total de bonos es: $ 300.00
El total de descuentos es: $ 512.50
La mitad del salario neto es: $ 2393.75
Conclusiones:
1. Neto > Bruto : Falso
2. Los descuentos superan a los bonos, por eso el neto es menor o igual que el bruto.
3. Solo se pagan bonos por un maximo de 10 anios de antiguedad.
4. Los descuentos de ISSS y AFP suman el 10.25% del salario bruto.
```

Figura 6: comprobación de la prueba 2

2. Enunciado del algoritmo

2) EJ-367

En un banco, construya un algoritmo que simule un sistema de validación de PIN. Use una constante simbólica PIN_CORRECTO = 1234 (entero de 4 dígitos) y MAX_INTENTOS = 3. Lea un PIN ingresado (entero). Evalúe la expresión relacional (pin_ingresado == PIN_CORRECTO). Asigne el resultado a una variable booleana acceso_permitido. Use la expresión aritmética (intentos + 1) para llevar un contador (inicializado con literal 0). Evalúe la expresión lógica (intentos < MAX_INTENTOS AND NOT acceso_permitido). Muestre por pantalla el estado de la validación. Adicionalmente, calcule la mitad del resultado obtenido. Verifique que el algoritmo funcione correctamente con valores limite.

2.1. Análisis del problema

Datos de entrada:

- Pin correcto (1234)
- Máximo de intentos (3)
- Pin ingresado
- Intentos

Proceso

- acceso_permitido = (pin_ingresado == PIN_CORRECTO)
- intentos = (intentos + 1)
- mitad = intentos / 2.0

Datos de salida:

- El pin ingresado
- Los intentos restantes
- Estado de validación

2.2. Diagrama de flujo

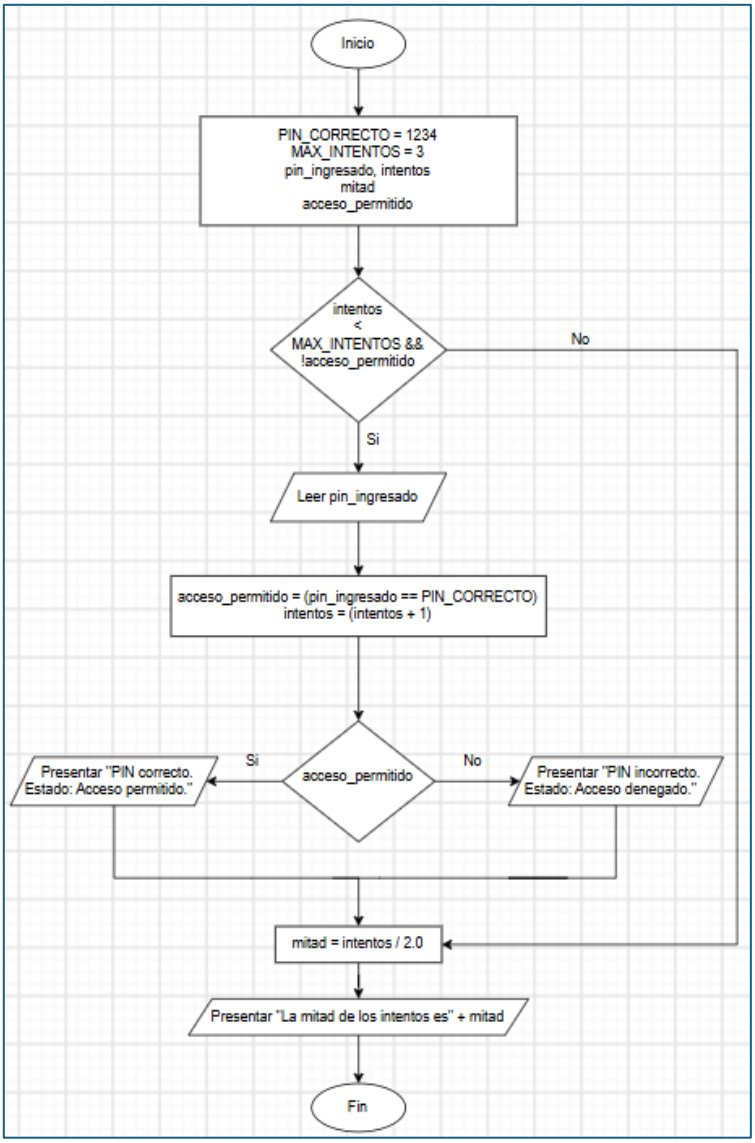


Figura 7: diagrama de flujo ejercicio 367 en Draw.io

2.3. Pseudocódigo

```
Inicio
PIN_CORRECTO = 1234
MAX_INTENTOS = 3
pin_ingresado, intentos
mitad
acceso_permitido

Mientras (intentos < MAX_INTENTOS && !acceso_permitido)
Leer pin_ingresado
acceso_permitido = (pin_ingresado == PIN_CORRECTO)
intentos = (intentos + 1)

Si (acceso_permitido) entonces
Presentar "PIN correcto. Estado: Acceso permitido."
Sino
Presentar "PIN incorrecto. Estado: Acceso denegado."
Fin Si

mitad = intentos / 2.0
Presentar "La mitad de los intentos es" + mitad

Fin
```

Figura 8: pseudocódigo del ejercicio 367

2.4. Prueba de escritorio

Prueba 1: Acceso permitido en el primer intento

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	intentos = 0	(intentos < MAX_INTENTOS && !acceso_permitido) (0 < 3) = TRUE	
2	pin_ingresado acceso_permitido	(pin_ingresado == PIN_CORRECTO) (1234 == 1234) = TRUE	Ingrese el PIN: 1234
3	intentos = 1	0 + 1 = 1 (1 < 3) = FALSE (sale del ciclo)	
4	Presentar		PIN correcto. Estado: Acceso permitido
5	mitad	1 / 2 = 0.5	
6	Presentar		La mitad de los intentos es: 0.5

Prueba 2: Acceso permitido en el último intento

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	intentos = 0	(intentos < MAX_INTENTOS && !acceso_permitido) (0 < 3) = TRUE	
2	pin_ingresado acceso_permitido	(pin_ingresado == PIN_CORRECTO) (1598 == 1234) = FALSE	Ingrese el PIN: 1598
3	intentos = 1	0 + 1 = 1 (1 < 3) = TRUE	
4	Presentar		PIN incorrecto. Estado: Acceso denegado Intentos restantes: 2
5	intentos = 1	(intentos < MAX_INTENTOS && !acceso_permitido) (1 < 3) = TRUE	
6	acceso_permitido	(94567 == 1234) = FALSE	Ingrese el PIN: 94567
7	intentos = 2	1 + 1 = 2 (2 < 3) = TRUE	
8	Presentar		PIN incorrecto. Estado: Acceso denegado Intentos restantes: 1
9	intentos = 2	(intentos < MAX_INTENTOS && !acceso_permitido) (2 < 3) = TRUE	
10	acceso_permitido	(1234 == 1234) = TRUE	Ingrese el PIN: 1234
11	intentos = 3	2 + 1 = 3 (3 < 3) = FALSE (sale del ciclo)	
12	Presentar		PIN correcto. Estado: Acceso permitido
13	mitad	1 / 2 = 1.5	
14	Presentar		La mitad de los intentos es: 1.5

Prueba 3: Acceso denegado en los tres intentos

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	intentos = 0	(intentos < MAX_INTENTOS && !acceso_permitido) (0 < 3) = TRUE	
2	pin_ingresado acceso_permitido	(pin_ingresado == PIN_CORRECTO) (12615 == 1234) = FALSE	Ingrese el PIN: 12615
3	intentos = 1	0 + 1 = 1 (1 < 3) = TRUE	
4	Presentar		PIN incorrecto. Estado: Acceso denegado Intentos restantes: 2

5	intentos = 1	(intentos < MAX_INTENTOS && !acceso_permitido) (1 < 3) = TRUE	
6	acceso_permitido	(7456 == 1234) = FALSE	Ingrese el PIN: 7456
7	intentos = 2	1 + 1 = 2 (2 < 3) = TRUE	
8	Presentar		PIN incorrecto. Estado: Acceso denegado Intentos restantes: 1
9	intentos = 2	(intentos < MAX_INTENTOS && !acceso_permitido) (2 < 3) = TRUE	
10	acceso_permitido	(1459 == 1234) = FALSE	Ingrese el PIN: 1459
11	intentos = 3	2 + 1 = 3 (3 < 3) = FALSE (sale del ciclo)	
12	Presentar		PIN incorrecto. Estado: Acceso denegado Intentos restantes: 0
13	mitad	1 / 2 = 1.5	
14	Presentar		La mitad de los intentos es: 1.5

2.5. Codificación en lenguaje C

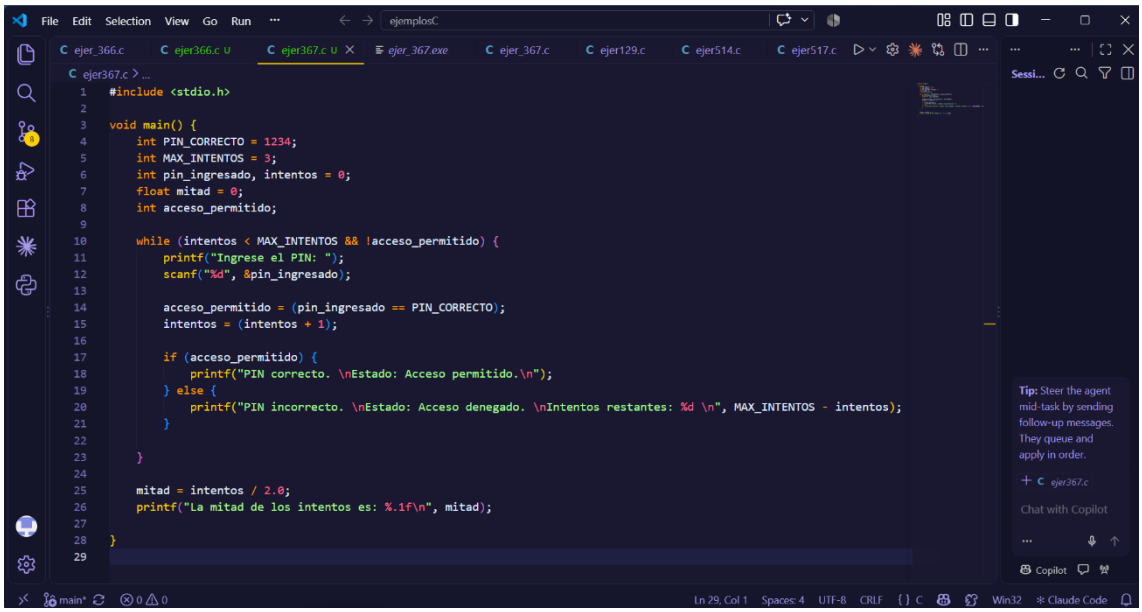


Figura 9: código del ejercicio 367, variables, proceso y salida

2.6. Salida del programa

```
Problems  Output  Debug Console  Terminal  Ports

Microsoft Windows [Versión 10.0.26300.9711]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>gcc ejer367.c -o ejer367

C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>ejer367
Ingrese el PIN: 1234
PIN correcto.
Estado: Acceso permitido.
La mitad de los intentos es: 0.5
```

Figura 10: comprobación de la prueba 1

```
C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>ejer367
Ingrese el PIN: 1598
PIN incorrecto.
Estado: Acceso denegado.
Intentos restantes: 2
Ingrese el PIN: 94567
PIN incorrecto.
Estado: Acceso denegado.
Intentos restantes: 1
Ingrese el PIN: 1234
PIN correcto.
Estado: Acceso permitido.
La mitad de los intentos es: 1.5
```

Figura 11: comprobación de la prueba 2

```
C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>ejer367
Ingrese el PIN: 12615
PIN incorrecto.
Estado: Acceso denegado.
Intentos restantes: 2
Ingrese el PIN: 7456
PIN incorrecto.
Estado: Acceso denegado.
Intentos restantes: 1
Ingrese el PIN: 1459
PIN incorrecto.
Estado: Acceso denegado.
Intentos restantes: 0
La mitad de los intentos es: 1.5
```

Figura 12: comprobación de la prueba 3

3. Enunciado del algoritmo

3) EJ-368

En un supermercado, diseñe un algoritmo que lea una fecha (dia, mes, año como enteros) y calcule el día juliano (numero de día del año, 1-366). Use constantes simbólicas para los días por mes: DIA_ENE = 31, DIA_FEB = 28, DIA_FEB_BISIESTO = 29, DIA_MAR = 31, DIA_ABR = 30, etc. Evalúe la expresión lógica para año bisiesto: (año % 4 == 0 AND (año % 100 != 0 OR año % 400 == 0)). Calcule el día juliano sumando los días de los meses anteriores más el día actual. Asigne el resultado a una variable entera. Muestre por pantalla la fecha y el día juliano. Adicionalmente, calcule la mitad del resultado obtenido. Agregue al inicio una descripción breve de los pasos a seguir.

3.1. Análisis del problema

Datos de entrada:

- Días por mes en juliano
- Fecha
- Dia juliano

Proceso

- bisiesto = (anio % 4 == 0 && (anio % 100 != 0 || anio % 400 == 0))
- mitad = dia_juliano / 2

Datos de salida:

- Fecha (día, mes y año)
- Dia juliano
- Mitad

3.2. Diagrama de flujo

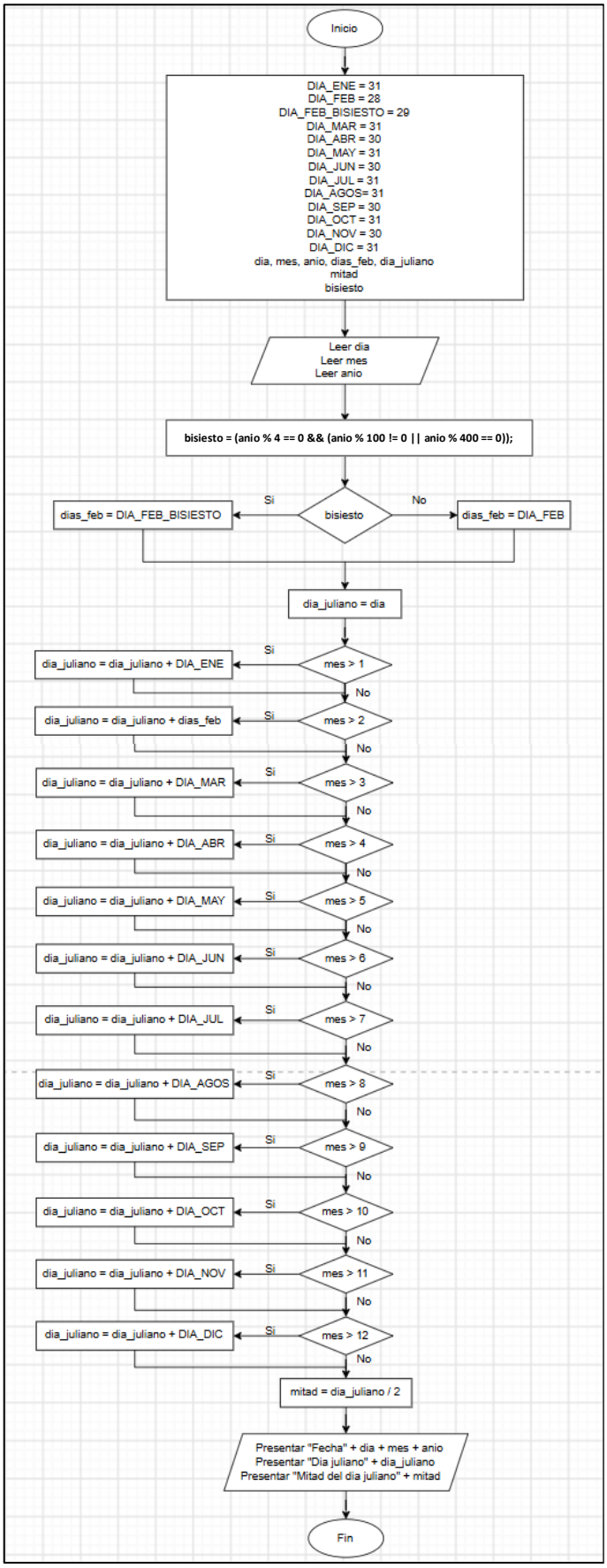


Figura 13: diagrama de flujo ejercicio 368 en Draw.io

3.3. Pseudocódigo

```
Inicio

DIA_ENE = 31;
DIA_FEB = 28;
DIA_FEB_BISIESTO = 29;
DIA_MAR = 31;
DIA_ABR = 30;
DIA_MAY = 31;
DIA_JUN = 30;
DIA_JUL = 31;
DIA_AGOS= 31;
DIA_SEP = 30;
DIA_OCT = 31;
DIA_NOV = 30;
DIA_DIC = 31;
dia, mes, anio, dias_feb, dia_juliano;
mitad;
bisiesto;

Leer dia;
Leer mes;
Leer anio;

bisiesto = (anio % 4 == 0 && (anio % 100 != 0 || anio % 400 == 0));

Si (bisiesto) entonces
dias_feb = DIA_FEB_BISIESTO;
Sino
dias_feb = DIA_FEB;
Fin Si

dia_juliano = dia;

Si (mes > 1) entonces
dia_juliano = dia_juliano + DIA_ENE;
Fin Si
Si (mes > 2) entonces
dia_juliano = dia_juliano + dias_feb;
Fin Si
Si (mes > 3) entonces
dia_juliano = dia_juliano + DIA_MAR;
Fin Si
Si (mes > 4) entonces
dia_juliano = dia_juliano + DIA_ABR;
Fin Si
Si (mes > 5) entonces
dia_juliano = dia_juliano + DIA_MAY;
Fin Si
Si (mes > 6) entonces
dia_juliano = dia_juliano + DIA_JUN;
Fin Si
Si (mes > 7) entonces
dia_juliano = dia_juliano + DIA_JUL;
Fin Si
Si (mes > 8) entonces
dia_juliano = dia_juliano + DIA_AGOS;
Fin Si
Si (mes > 9) entonces
dia_juliano = dia_juliano + DIA_SEP;
Fin Si
Si (mes > 10) entonces
dia_juliano = dia_juliano + DIA_OCT;
Fin Si
Si (mes > 11) entonces
dia_juliano = dia_juliano + DIA_NOV;
Fin Si
Si (mes > 12) entonces
dia_juliano = dia_juliano + DIA_DIC;
Fin Si

mitad = dia_juliano / 2;

Presentar "Fecha" + dia + mes + anio;
Presentar "Dia juliano" + dia_juliano;
Presentar "Mitad del dia juliano" + mitad;

Fin
```

Figura 14: pseudocódigo del ejercicio 368

3.4. Prueba de escritorio

Prueba 1: Año no bisiesto

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	dia mes anio		Dia: 25 Mes: 10 Anio: 2026
2	bisiesto	(anio % 4 == 0 && (anio % 100 != 0 || anio % 400 == 0)) (2026 % 4 == 0) = (2 == 0)) = FALSE	
3	dias_feb	bisiesto = FALSE dias_feb = DIA_FEB = 28	
4	dia_juliano	dia_juliano = dia = 25	
5	dia_juliano	(10 > 1) = TRUE 25 + DIA_ENE (31) = 56	
6	dia_juliano	(10 > 2) = TRUE 56 + dias_feb (28) = 84	
7	dia_juliano	(10 > 3) = TRUE 84 + DIA_MAR (31) = 115	
8	dia_juliano	(10 > 4) = TRUE 115 + DIA_ABR (30) = 145	
9	dia_juliano	(10 > 5) = TRUE 145 + DIA_MAY (31) = 176	
10	dia_juliano	(10 > 6) = TRUE 176 + DIA_JUN (30) = 206	
11	dia_juliano	(10 > 7) = TRUE 206 + DIA_JUL (31) = 237	
12	dia_juliano	(10 > 8) = TRUE 237 + DIA_AGO (31) = 268	
13	dia_juliano	(10 > 9) = TRUE 268 + DIA_SEP (30) = 298	
14	dia_juliano	(10 > 10) = FALSE (10 > 11) = FALSE (10 > 12) = FALSE	
15	mitad	298 / 2.0 = 149.0	
16	Presentar		Fecha: 25/10/2026 Dia juliano: 298 Mitad del dia juliano: 149.00

Prueba 2: Año bisiesto

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	dia mes anio		Dia: 29 Mes: 02 Anio: 2024
2	bisiesto	(2024 % 4 == 0) = TRUE (2024 % 100 != 0) = (24 != 0) = TRUE	
3	dias_feb	bisiesto = TRUE dias_feb = DIA_FEB_BISIESTO = 29	
4	dia_juliano	dia_juliano = dia = 29	
5	dia_juliano	(2 > 1) = TRUE 29 + DIA_ENE (31) = 60	
6	dia_juliano	(2 > 2) = FALSE (2 > n) = FALSE	
7	mitad	60 / 2.0 = 30.0	
8	Presentar		Fecha: 29/02/2024 Dia juliano: 60 Mitad del dia juliano: 30.0

3.5. Codificación en lenguaje C

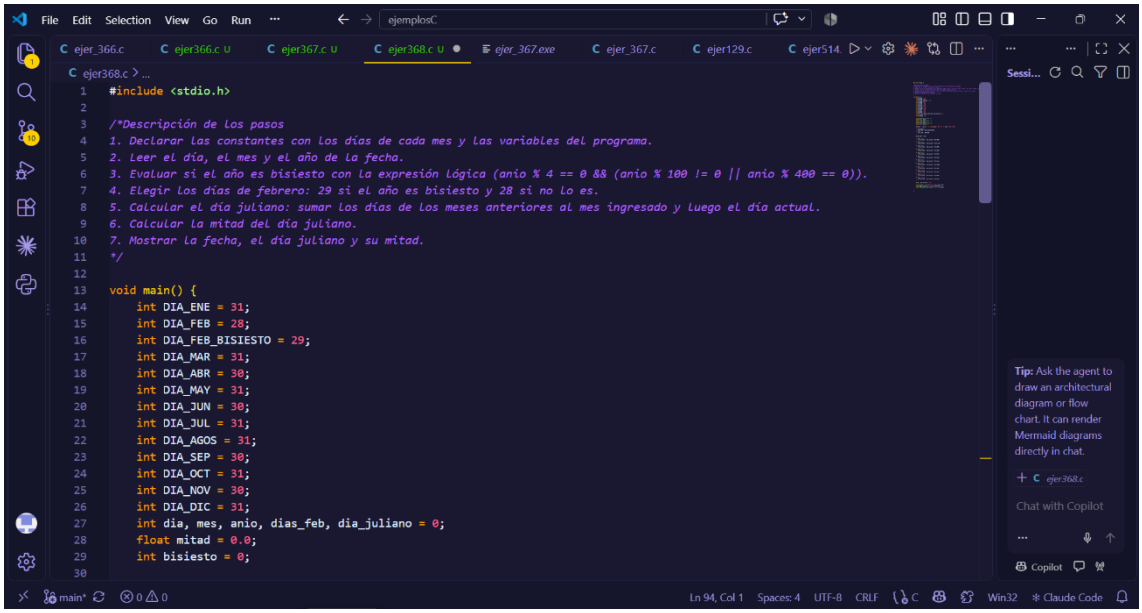
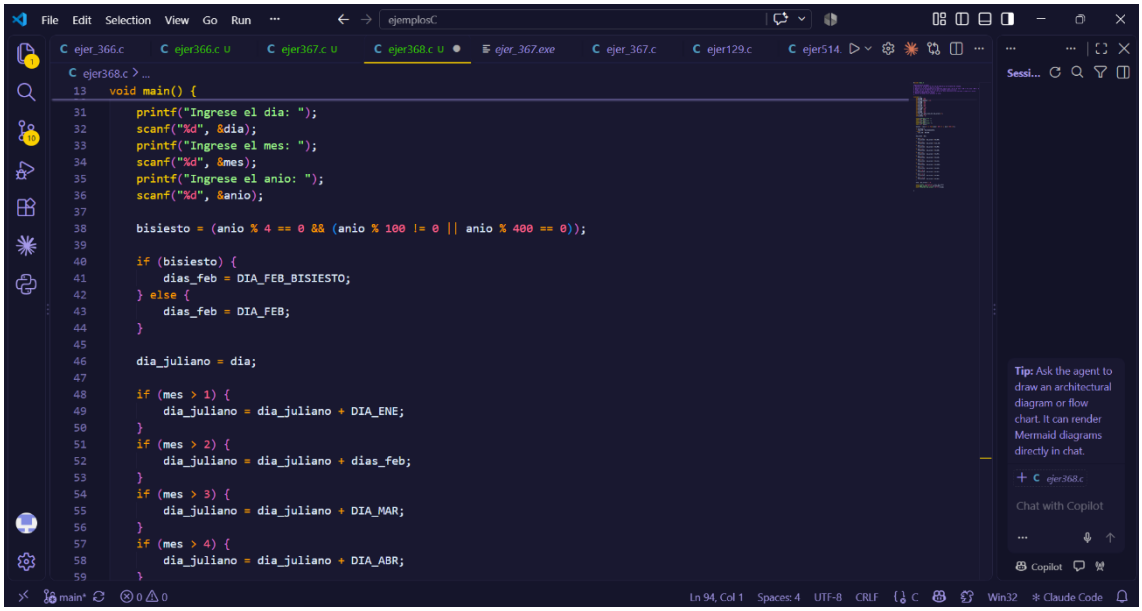


Figura 15: código del ejercicio 368, descripción de los pasos a seguir y variables



```c
void main() {
    printf("Ingrese el dia: ");
    scanf("%d", &dia);
    printf("Ingrese el mes: ");
    scanf("%d", &mes);
    printf("Ingrese el anio: ");
    scanf("%d", &anio);

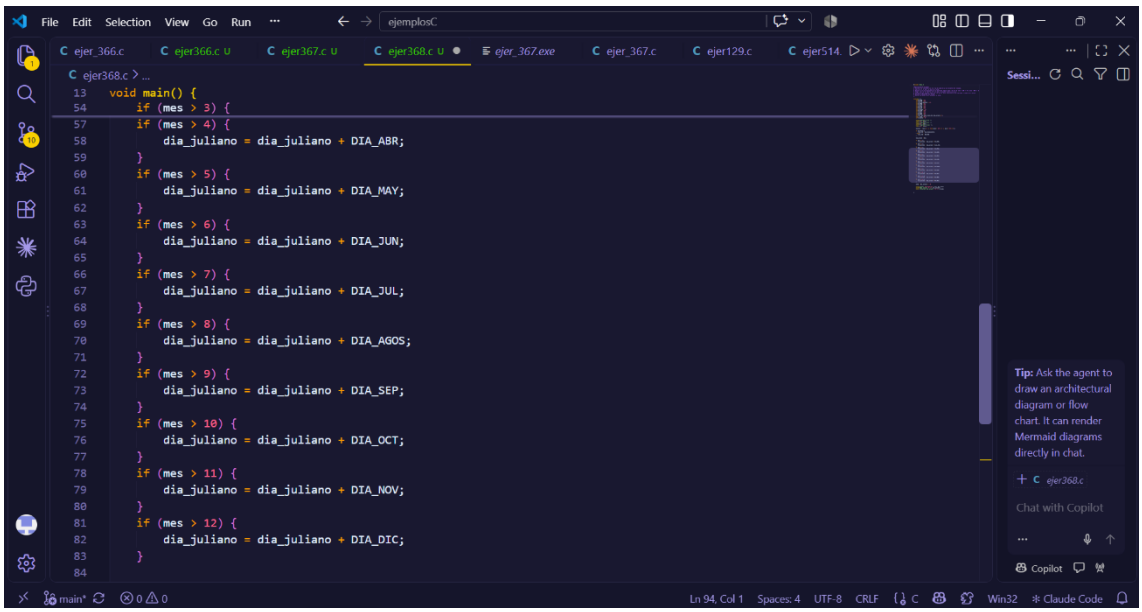
    bisiesto = (anio % 4 == 0 && (anio % 100 != 0 || anio % 400 == 0));

    if (bisiesto) {
        dias_feb = DIA_FEB_BISIESTO;
    } else {
        dias_feb = DIA_FEB;
    }

    dia_juliano = dia;

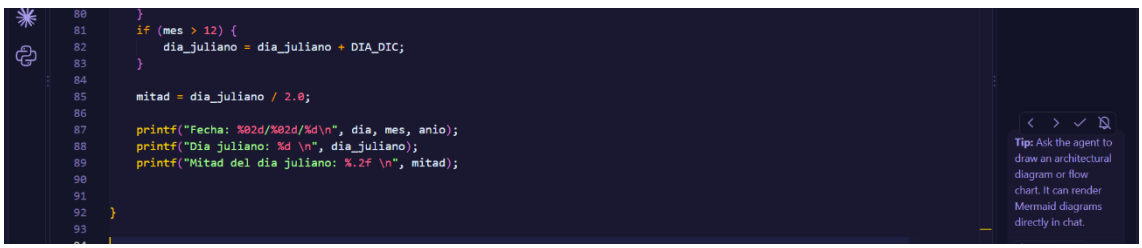
    if (mes > 1) {
        dia_juliano = dia_juliano + DIA_ENE;
    }
    if (mes > 2) {
        dia_juliano = dia_juliano + dias_feb;
    }
    if (mes > 3) {
        dia_juliano = dia_juliano + DIA_MAR;
    }
    if (mes > 4) {
        dia_juliano = dia_juliano + DIA_ABR;
    }
```

Figura 16: código del ejercicio 368, proceso



```c
void main() {
    if (mes > 3) {
    if (mes > 4) {
        dia_juliano = dia_juliano + DIA_ABR;
    }
    if (mes > 5) {
        dia_juliano = dia_juliano + DIA_MAY;
    }
    if (mes > 6) {
        dia_juliano = dia_juliano + DIA_JUN;
    }
    if (mes > 7) {
        dia_juliano = dia_juliano + DIA_JUL;
    }
    if (mes > 8) {
        dia_juliano = dia_juliano + DIA_AGOS;
    }
    if (mes > 9) {
        dia_juliano = dia_juliano + DIA_SEP;
    }
    if (mes > 10) {
        dia_juliano = dia_juliano + DIA_OCT;
    }
    if (mes > 11) {
        dia_juliano = dia_juliano + DIA_NOV;
    }
    if (mes > 12) {
        dia_juliano = dia_juliano + DIA_DIC;
    }
```

Figura 17: código del ejercicio 368, proceso



```c
    }
    if (mes > 12) {
        dia_juliano = dia_juliano + DIA_DIC;
    }

    mitad = dia_juliano / 2.0;

    printf("Fecha: %02d/%02d/%d\n", dia, mes, anio);
    printf("Dia juliano: %d \n", dia_juliano);
    printf("Mitad del dia juliano: %.2f \n", mitad);


}
```

Figura 18: código del ejercicio 368, salida

3.6. Salida del programa

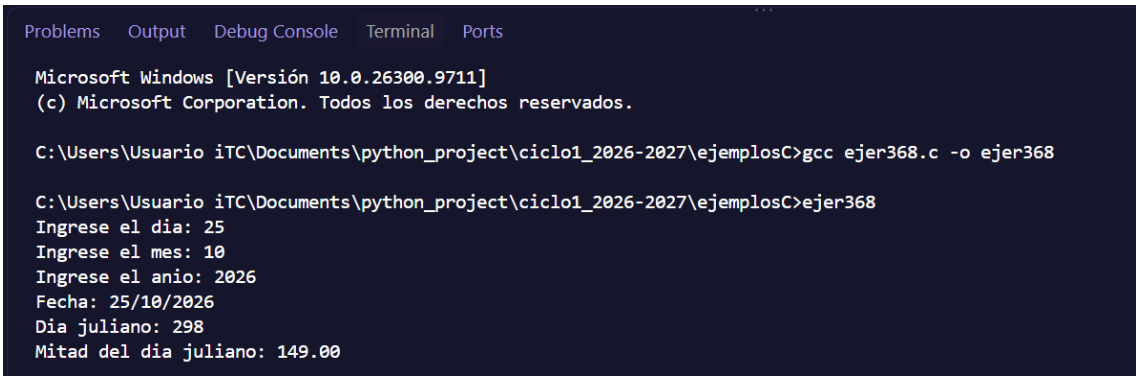


Figura 19: comprobación de la prueba 1

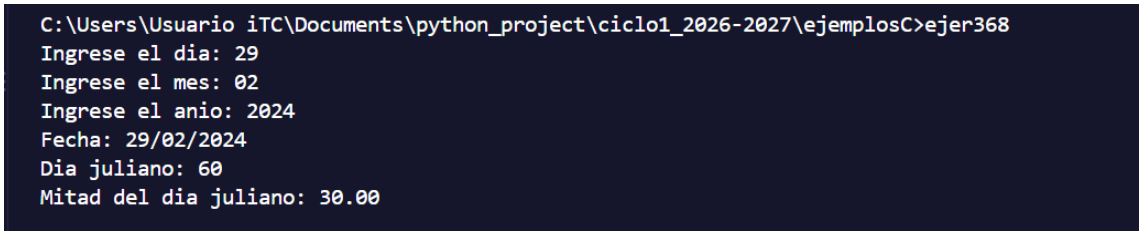


Figura 20: comprobación de la prueba 2

4. Enunciado del algoritmo

4) EJ-369

En una ferretería, cree un algoritmo que lea tres lados (reales) de un triángulo y determine si forman un triángulo valido. Evalúe la expresión lógica del teorema de la desigualdad triangular: (lado1 + lado2 > lado3 AND lado1 + lado3 > lado2 AND lado2 + lado3 > lado1). Si es válido, clasifíquelo usando expresiones lógicas: equilátero (los tres iguales), isósceles (al menos dos iguales), escaleno (todos diferentes). Asigne el tipo como cadena a una variable. Muestre por pantalla el tipo de triángulo. Adicionalmente, calcule la mitad del resultado obtenido. Documente cada paso del algoritmo con comentarios explicativos.

4.1. Análisis del problema

Datos de entrada:

- Lado 1, lado 2 y lado 3
- Expresión lógica del teorema de la desigualdad triangular

Proceso

- teorema = (lado1 + lado2 > lado3 && lado1 + lado3 > lado2 && lado2 + lado3 > lado1)
- perimetro = lado1 + lado2 + lado3
- mitad = perimetro / 2

Datos de salida:

- Tipo de triángulo
- Mitad del resultado

4.2. Diagrama de flujo

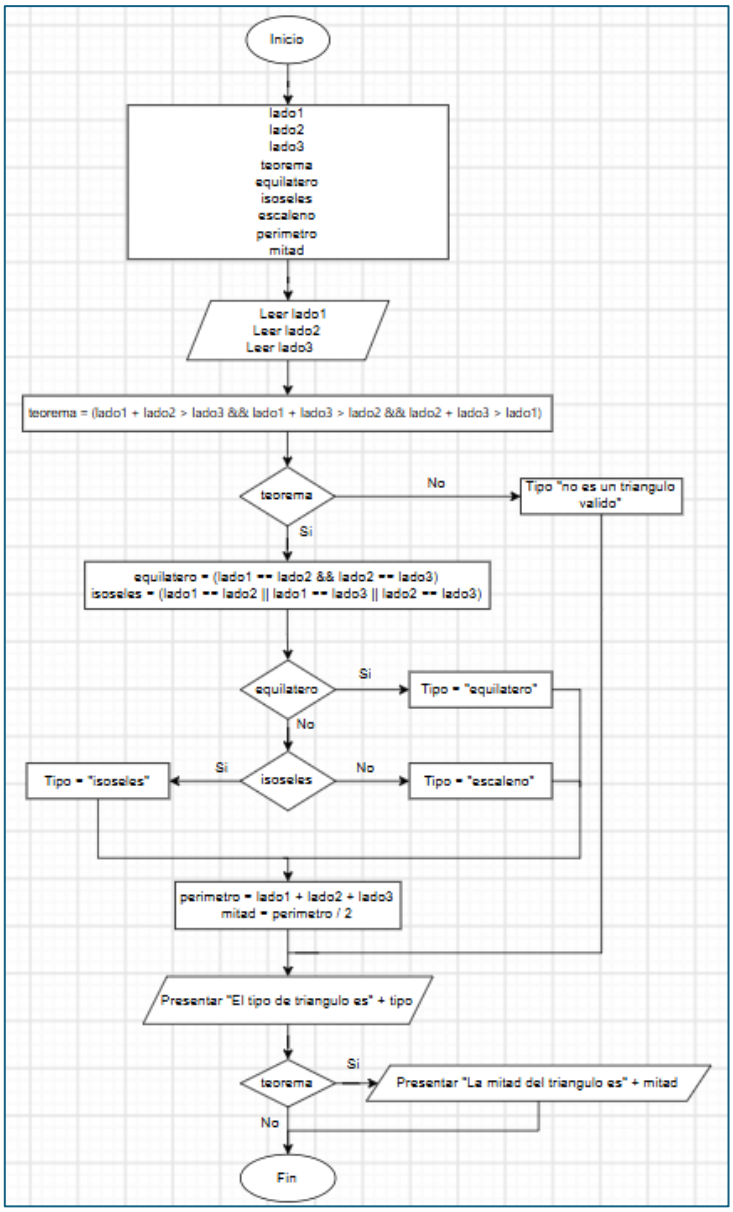


Figura 21: diagrama de flujo ejercicio 369 en Draw.io

4.3. Pseudocódigo

```
Inicio

lado1;
lado2;
lado3;
teorema;
equilatero;
isoseles;
escaleno;
perimetro;
mitad;

Leer lado1;
Leer lado2;
Leer lado3;

teorema = (lado1 + lado2 > lado3 && lado1 + lado3 > lado2 && lado2 + lado3 > lado1);

Si (teorema) entonces
equilatero = (lado1 == lado2 && lado2 == lado3);
isoseles = (lado1 == lado2 || lado1 == lado3 || lado2 == lado3);

Si (equilatero) entonces
Tipo = "equilatero";
Sino
Si (isoseles) entonces
Tipo = "isoseles";
Sino
Tipo = "escaleno";
Fin Si
Fin Si

perimetro = lado1 + lado2 + lado3;
mitad = perimetro / 2;

Sino
Tipo "no es un triangulo valido";
Fin Si

Presentar "El tipo de triangulo es" + tipo;

Si (teorema) entonces
Presentar "La mitad del triangulo es" + mitad;
Fin Si

Fin
```

Figura 22: pseudocódigo del ejercicio 369

4.4. Prueba de escritorio

Prueba 1: Triangulo equilátero

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	lado1 lado2 lado3		Ingrese el primer lado: 3 Ingrese el segundo lado: 3 Ingrese el tercer lado: 3
2	teorema	(lado1 + lado2 > lado3 && lado1 + lado3 > lado2 && lado2 + lado3 > lado1) (3+3 > 3) && (3+3 > 3) && (3+3 > 3) = TRUE	
3	equilatero	(lado1 == lado2 && lado2 == lado3) (3 == 3 && 3 == 3) = TRUE	
4	isoseles	(lado1 == lado2 || lado1 == lado3 || lado2 == lado3) (3 == 3 || 3 == 3 || 3 == 3) = TRUE	
5	tipo	equilatero	
6	perimetro	lado1 + lado2 + lado3 3 + 3 + 3 = 9	
7	mitad	perimetro / 2 9 / 2 = 4.5	
8	Presentar	teorema = TRUE	El tipo de triangulo es: equilatero La mitad del triangulo es: 4.5

Prueba 2: Triangulo isósceles

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	lado1 lado2 lado3		Ingrese el primer lado: 3 Ingrese el segundo lado: 3 Ingrese el tercer lado: 5
2	teorema	(lado1 + lado2 > lado3 && lado1 + lado3 > lado2 && lado2 + lado3 > lado1) (3+3 > 5) && (3+5 > 3) && (3+5 > 3) = TRUE	
3	equilatero	(lado1 == lado2 && lado2 == lado3) (3 == 3 && 3 == 5) = FALSE	
4	isoseles	(lado1 == lado2 || lado1 == lado3 || lado2 == lado3) (3 == 3 || 3 == 5 || 3 == 5) = TRUE	
5	tipo	isoseles	
6	perimetro	lado1 + lado2 + lado3 3 + 3 + 5 = 11	
7	mitad	perimetro / 2 11 / 2 = 5.5	
8	Presentar	teorema = TRUE	El tipo de triangulo es: isoseles La mitad del triangulo es: 5.5

Prueba 3: Triangulo escaleno

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	lado1 lado2 lado3		Ingrese el primer lado: 3 Ingrese el segundo lado: 4 Ingrese el tercer lado: 5
2	teorema	(lado1 + lado2 > lado3 && lado1 + lado3 > lado2 && lado2 + lado3 > lado1) (3+4 > 5) && (3+5 > 4) && (4+5 > 3) = TRUE	
3	equilatero	(lado1 == lado2 && lado2 == lado3) (3 == 4 && 4 == 5) = FALSE	
4	isoseles	(lado1 == lado2 || lado1 == lado3 || lado2 == lado3) (3 == 4 || 3 == 5 || 4 == 5) = FALSE	
5	tipo	escaleno	
6	perimetro	lado1 + lado2 + lado3 3 + 4 + 5 = 12	
7	mitad	perimetro / 2 12 / 2 = 6.0	
8	Presentar	teorema = TRUE	El tipo de triangulo es: escaleno La mitad del triangulo es: 6.0

4.5. Codificación en lenguaje C

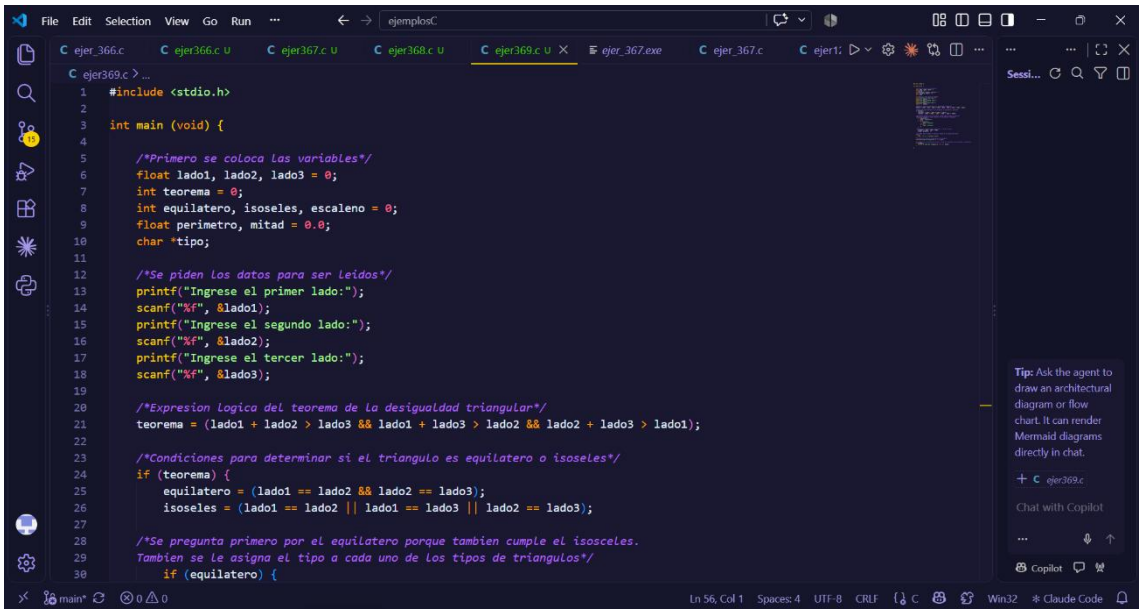
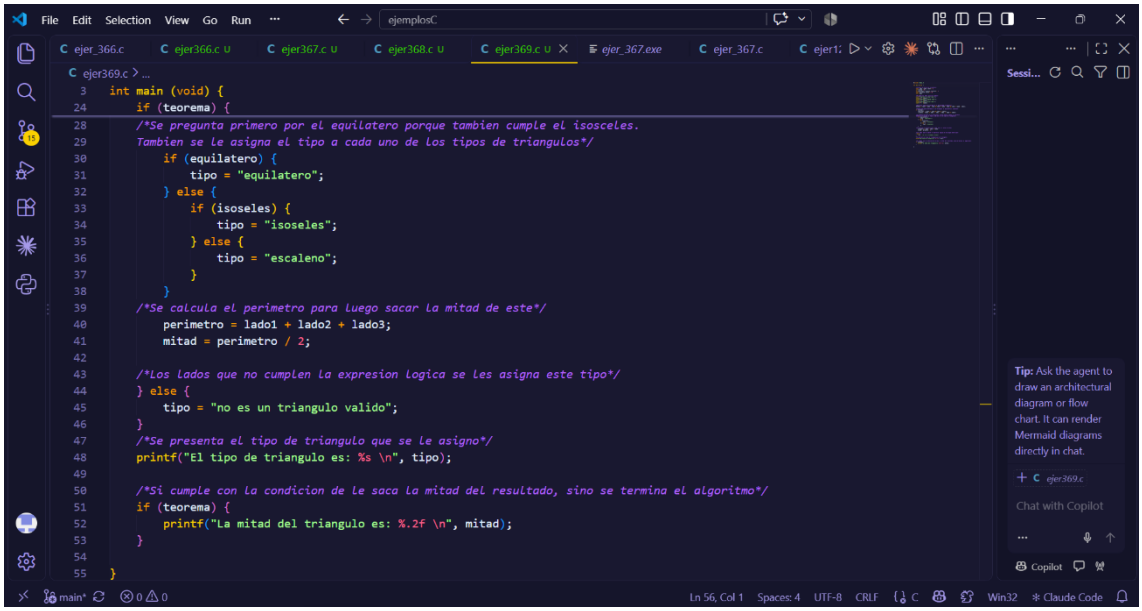


Figura 23: código del ejercicio 369, comentarios explicativos, variables y proceso



```c
int main (void) {
    if (teorema) {
    /*Se pregunta primero por el equilatero porque tambien cumple el isosceles.
    Tambien se Le asigna el tipo a cada uno de los tipos de triangulos*/
        if (equilatero) {
            tipo = "equilatero";
        } else {
            if (isoseles) {
                tipo = "isoseles";
            } else {
                tipo = "escaleno";
            }
        }
    /*Se calcula el perimetro para luego sacar la mitad de este*/
        perimetro = lado1 + lado2 + lado3;
        mitad = perimetro / 2;

    /*Los lados que no cumplen la expresion logica se les asigna este tipo*/
    } else {
        tipo = "no es un triangulo valido";
    }
    /*Se presenta el tipo de triangulo que se le asigno*/
    printf("El tipo de triangulo es: %s \n", tipo);

    /*Si cumple con la condicion de le saca la mitad del resultado, sino se termina el algoritmo*/
    if (teorema) {
        printf("La mitad del triangulo es: %.2f \n", mitad);
    }

}
```

Figura 24: código del ejercicio 369, comentarios explicativos, proceso y salida

4.6. Salida del programa

```
Problems  Output  Debug Console  Terminal  Ports

Microsoft Windows [Versión 10.0.26300.9711]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>gcc ejer369.c -o ejer369

C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>ejer369
Ingrese el primer lado:3
Ingrese el segundo lado:3
Ingrese el tercer lado:3
El tipo de triangulo es: equilatero
La mitad del triangulo es: 4.50
```

Figura 25: comprobación de la prueba 1

```
C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>ejer369
Ingrese el primer lado:3
Ingrese el segundo lado:3
Ingrese el tercer lado:5
El tipo de triangulo es: isoseles
La mitad del triangulo es: 5.50
```

Figura 26: comprobación de la prueba 2

```
C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>ejer369
Ingrese el primer lado:3
Ingrese el segundo lado:4
Ingrese el tercer lado:5
El tipo de triangulo es: escaleno
La mitad del triangulo es: 6.00
```

Figura 27: comprobación de la prueba 3

5. Enunciado del algoritmo

5) EJ-370

En una clínica veterinaria, diseñe un algoritmo que lea la cantidad de artículos (entero) y el precio unitario (real). Use constantes simbólicas para descuentos por volumen: DESC_10 = 0.05, DESC_50 = 0.10, DESC_100 = 0.15, UMBRAL_10 = 10, UMBRAL_50 = 50, UMBRAL_100 = 100, IVA = 0.15. Use expresiones lógicas relacionales para seleccionar el descuento. Calcule subtotal, descuento, total con IVA. Asigne los resultados a variables reales. Muestre por pantalla una factura detallada. Adicionalmente, calcule la mitad del resultado obtenido. Agregue un mensaje de bienvenida personalizado al iniciar el algoritmo.

5.1. Análisis del problema

Datos de entrada:

- Cantidad de artículos
- Precio unitario
- Descuentos por volumen

Proceso

- subtotal = cantidad_art * precio_uni
- descuento = subtotal * porcentaje_desc
- base_imponible = subtotal - descuento
- monto_iva = base_imponible * IVA
- total = base_imponible + monto_iva
- mitad = total / 2

Datos de salida:

- Subtotal
- Descuento
- Total con IVA
- Mitad

5.2. Diagrama de flujo

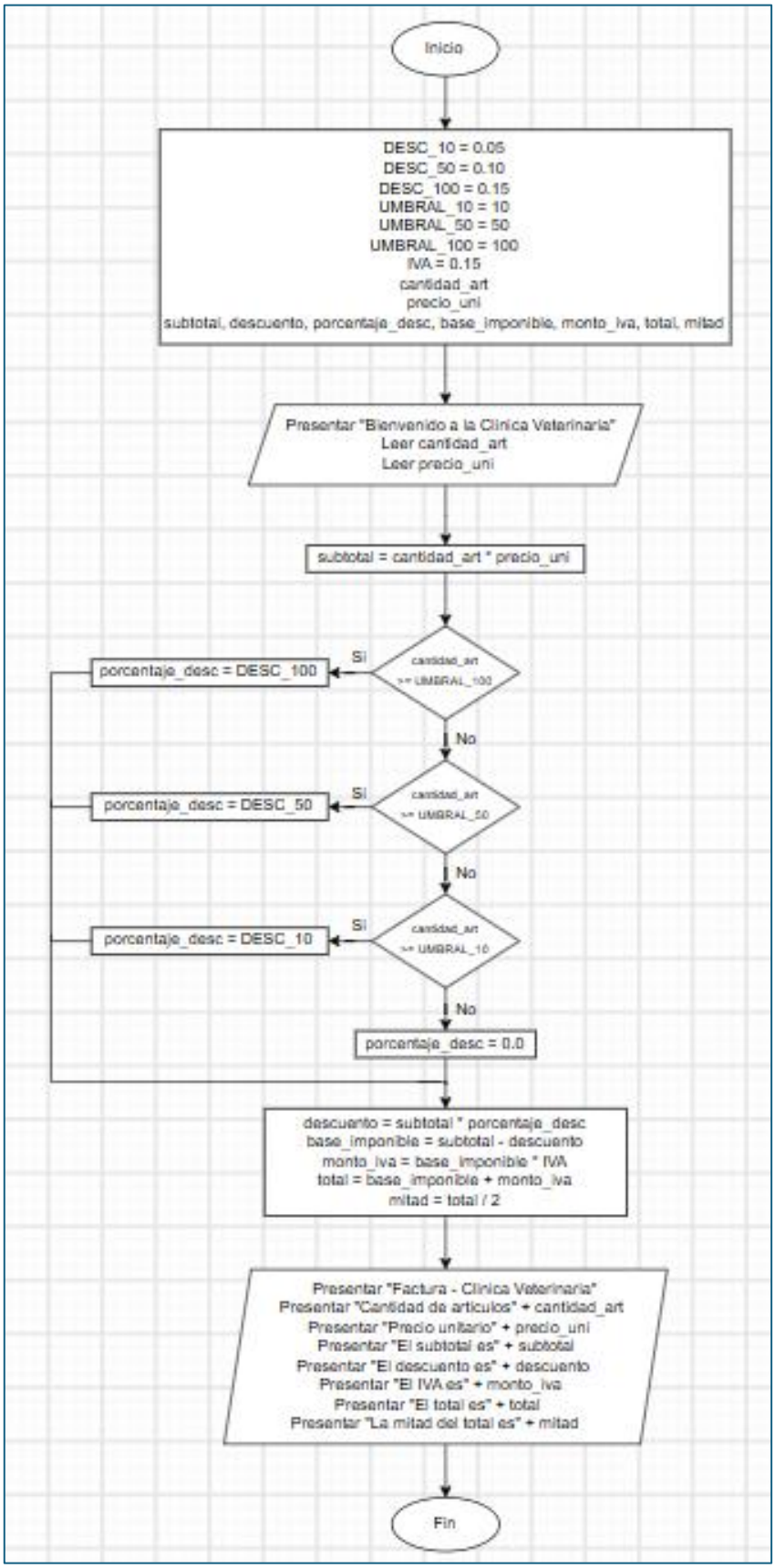


Figura 28: diagrama de flujo ejercicio 370 en Draw.io

5.3. Pseudocódigo

```
Inicio

DESC_10 = 0.05;
DESC_50 = 0.10;
DESC_100 = 0.15;
UMBRAL_10 = 10;
UMBRAL_50 = 50;
UMBRAL_100 = 100;
IVA = 0.15;
cantidad_art;
precio_uni;
subtotal, descuento, porcentaje_desc, base_imponible, monto_iva, total, mitad;

Presentar "Bienvenido a la Clinica Veterinaria";
Leer cantidad_art;
Leer precio_uni;

subtotal = cantidad_art * precio_uni;

Si (cantidad_art >= UMBRAL_100) entonces
porcentaje_desc = DESC_100;
Sino
Si (cantidad_art >= UMBRAL_50) entonces
porcentaje_desc = DESC_50;
Sino
Si (cantidad_art >= UMBRAL_10) entonces
porcentaje_desc = DESC_10;
Sino
porcentaje_desc = 0.0;
Fin Si
Fin Si
Fin Si

descuento = subtotal * porcentaje_desc;
base_imponible = subtotal - descuento;
monto_iva = base_imponible * IVA;
total = base_imponible + monto_iva;
mitad = total / 2;

Presentar "Factura - Clinica Veterinaria";
Presentar "Cantidad de articulos";
Presentar "Precio unitario";
Presentar "El subtotal es";
Presentar "El descuento es";
Presentar "El IVA es";
Presentar "El total es";
Presentar "La mitad del total es";

Fin
```

Figura 29: pseudocódigo del ejercicio 370

5.4. Prueba de escritorio

Prueba 1: Descuento del 15%

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	cantidad_art precio_uni		Bienvenido a la Clinica Veterinaria Ingrese la cantidad de articulos: 124 Ingrese el precio unitario: 50.00
2	Subtotal	cantidad_art * precio_uni 124 * 50.00 = 6200.00	
3	porcentaje_desc	(cantidad_art >= UMBRAL_100) 124 >= 100 = TRUE porcentaje_desc = 0.15	
4	descuento	subtotal * porcentaje_desc 6200.00 * 0.15 = 930.00	
5	base_imponible	subtotal – descuento 6200.00 - 930.00 = 5270.00	
6	monto_iva	base_imponible * IVA 5270.00 * 0.15 = 790.50	
7	total	base_imponible + monto_iva 5270.00 + 790.50 = 6060.50	
8	mitad	total / 2 6060.50 / 2 = 3030.25	
9	Presentar		"Factura - Clinica Veterinaria" Cantidad de articulos: 124 Precio unitario: $ 50.00 El subtotal es: $ 6200.00 El descuento es: $ 930.00 El IVA es: $ 790.50 El total es: $ 6060.50 La mitad del total es: $ 3030.25

Prueba 2: Descuento del 5%

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	cantidad_art precio_uni		Bienvenido a la Clinica Veterinaria Ingrese la cantidad de articulos: 20 Ingrese el precio unitario: 15.90
2	Subtotal	cantidad_art * precio_uni 20 * 15.90 = 318.00	
3	porcentaje_desc	(cantidad_art >= UMBRAL_100) 20 >= 100 = FALSE	
4	porcentaje_desc	(cantidad_art >= UMBRAL_50) 20 >= 50 = FALSE	

5	porcentaje_desc	(cantidad_art >= UMBRAL_10) 20 >= 10 = TRUE porcentaje_desc = 0.05	
6	descuento	subtotal * porcentaje_desc 318.00 * 0.05 = 15.90	
7	base_imponible	subtotal – descuento 318.00 - 15.90 = 302.10	
8	monto_iva	base_imponible * IVA 302.10 * 0.15 = 45.315	
9	total	base_imponible + monto_iva 302.10 + 45.315 = 347.415	
10	mitad	total / 2 347.42 / 2 = 173.71	
11	Presentar		"Factura - Clinica Veterinaria" Cantidad de articulos: 20 Precio unitario: $ 15.90 El subtotal es: $ 318.00 El descuento es: $ 15.90 El IVA es: $ 45.315 El total es: $ 347.42 La mitad del total es: $ 173.71

5.5. Codificación en lenguaje C

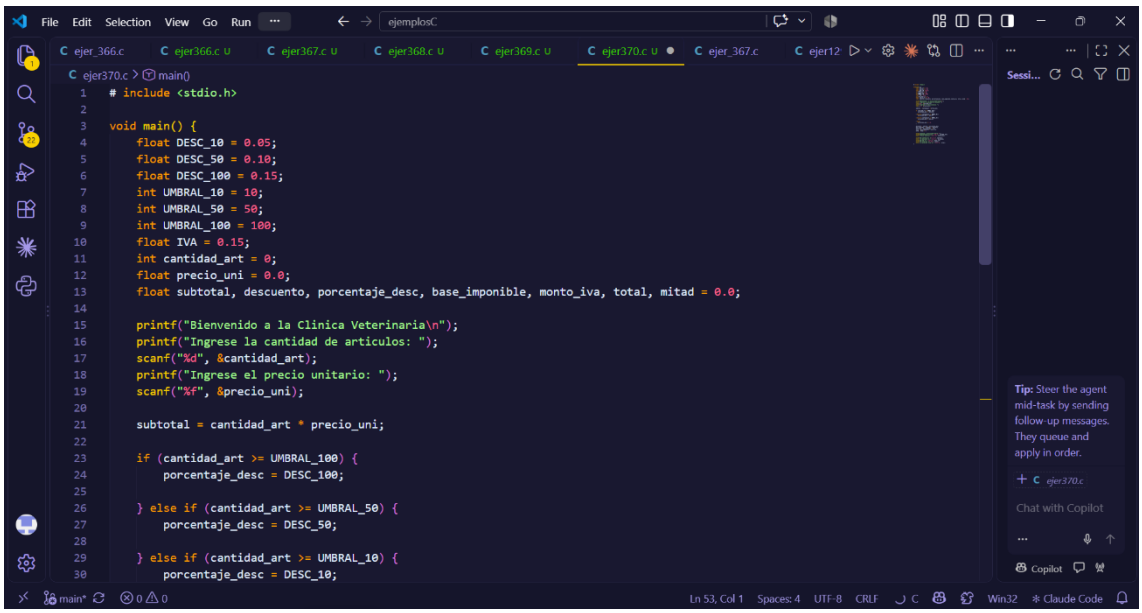
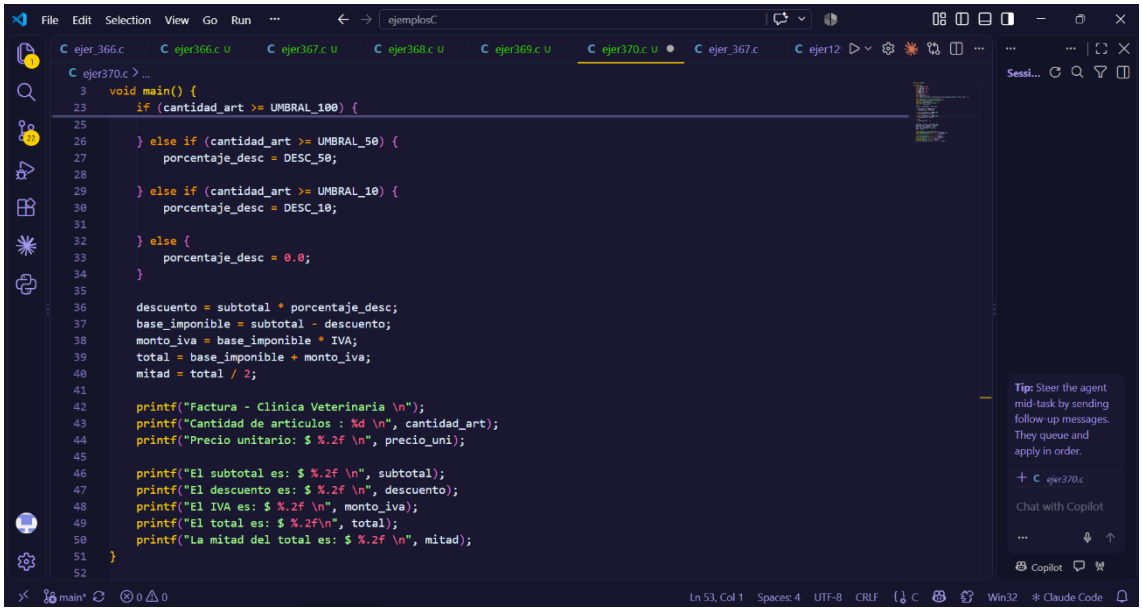


Figura 30: código del ejercicio 370, variables y proceso



```c
void main() {
    if (cantidad_art >= UMBRAL_100) {

    } else if (cantidad_art >= UMBRAL_50) {
        porcentaje_desc = DESC_50;

    } else if (cantidad_art >= UMBRAL_10) {
        porcentaje_desc = DESC_10;

    } else {
        porcentaje_desc = 0.0;
    }

    descuento = subtotal * porcentaje_desc;
    base_imponible = subtotal - descuento;
    monto_iva = base_imponible * IVA;
    total = base_imponible + monto_iva;
    mitad = total / 2;

    printf("Factura - Clinica Veterinaria \n");
    printf("Cantidad de articulos : %d \n", cantidad_art);
    printf("Precio unitario: $ %.2f \n", precio_uni);

    printf("El subtotal es: $ %.2f \n", subtotal);
    printf("El descuento es: $ %.2f \n", descuento);
    printf("El IVA es: $ %.2f \n", monto_iva);
    printf("El total es: $ %.2f\n", total);
    printf("La mitad del total es: $ %.2f \n", mitad);
}
```

Figura 31: código del ejercicio 370, proceso y salida

5.6. Salida del programa

```
Problems  Output  Debug Console  Terminal  Ports

Microsoft Windows [Versión 10.0.26300.9711]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>gcc ejer370.c -o ejer370

C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>ejer370
Bienvenido a la Clinica Veterinaria
Ingrese la cantidad de articulos: 124
Ingrese el precio unitario: 50
Factura - Clinica Veterinaria
Cantidad de articulos : 124
Precio unitario: $ 50.00
El subtotal es: $ 6200.00
El descuento es: $ 930.00
El IVA es: $ 790.50
El total es: $ 6060.50
La mitad del total es: $ 3030.25
```

Figura 32: comprobación de la prueba 1

```
C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>ejer370
Bienvenido a la Clinica Veterinaria
Ingrese la cantidad de articulos: 20
Ingrese el precio unitario: 15.90
Factura - Clinica Veterinaria
Cantidad de articulos : 20
Precio unitario: $ 15.90
El subtotal es: $ 318.00
El descuento es: $ 15.90
El IVA es: $ 45.32
El total es: $ 347.42
La mitad del total es: $ 173.71
```

Figura 33: comprobación de la prueba 2

6. Enunciado del algoritmo

15 ejercicios que se pueden repetir (anteriores ejercicios)

6) EJ-366

Diseñe un algoritmo que lea el código de un producto (entero de 4 dígitos) y valide que cumpla con un dígito verificador. Use constantes literales 10, 100, 1000 y operadores módulo y división entera para extraer los 4 dígitos. Calcule el dígito verificador esperado con la expresión aritmética: (d1 * 1 + d2 * 2 + d3 * 3 + d4 * 4) % 10. Evalúe la expresión relacional (digito_verificador == d4). Asigne el resultado a una variable booleana codigo_valido. Muestre por pantalla los dígitos y la validación.

6.1. Análisis del problema

Datos de entrada:

- Divisores 10, 100, 1000.
- Pesos de la expresión aritmética 1, 2, 3, 4.
- d1, d2, d3, d4.
- Código del producto.

Proceso

- d1 = codigo / 1000
- d2 = (codigo / 100) % 10
- d3 = (codigo / 10) % 10
- d4 = codigo % 10
- digito_veri = (d1 * 1 + d2 * 2 + d3 * 3 + d4 * 4) % 10
- codigo_vali = (digito_veri == d4)

Datos de salida:

- Primer dígito del código (d1).
- Segundo dígito del código (d2).
- Tercer dígito del código (d3).
- Cuarto dígito del código (d4).
- Dígito verificador esperado.
- Resultado de la validación del código (verdadero o falso).

6.2. Diagrama de flujo

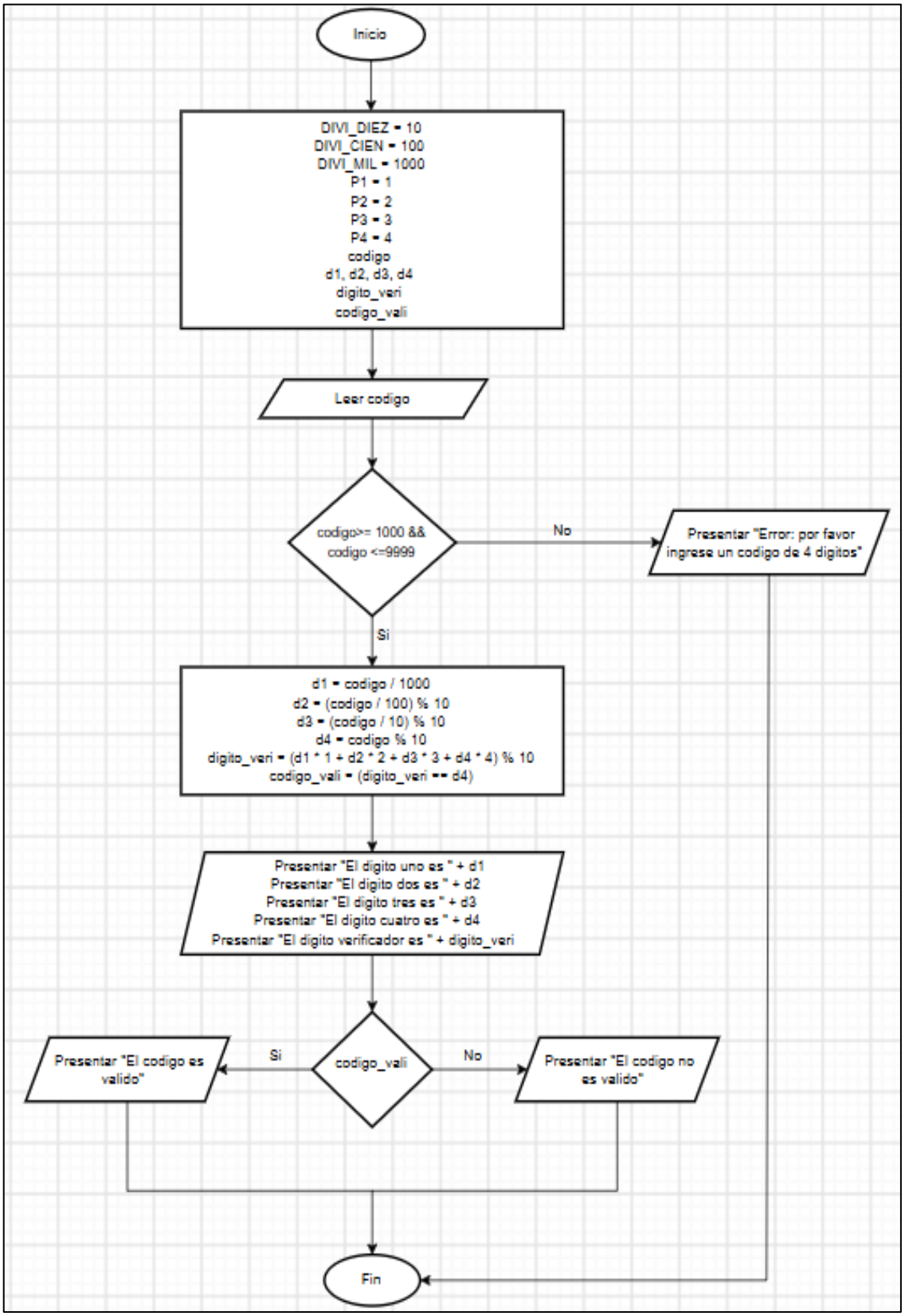


Figura 34: diagrama de flujo ejercicio anterior 366 en Draw.io

6.3. Pseudocódigo

```
Inicio

DIVI_DIEZ = 10;
DIVI_CIEN = 100;
DIVI_MIL = 1000;
P1 = 1;
P2 = 2;
P3 = 3;
P4 = 4;
codigo = 0;
d1 = 0, d2 = 0, d3 = 0, d4 = 0;
digito_veri = 0;
codigo_vali = 0;
Leer codigo = 0;

Si (codigo>= 1000 && codigo <=9999) entonces
d1 = codigo / 1000;
d2 = (codigo / 100) % 10;
d3 = (codigo / 10) % 10;
d4 = codigo % 10;
digito_veri = (d1 * 1 + d2 * 2 + d3 * 3 + d4 * 4) % 10;
codigo_vali = (digito_veri == d4);

Presentar "El digito uno es " + d1;
Presentar "El digito dos es " + d2;
Presentar "El digito tres es " + d3;
Presentar "El digito cuatro es " + d4;
Presentar "El digito verificador es " + digito_veri;

Si (codigo_vali) entonces
Presentar "El codigo es valido";
Sino
Presentar "El codigo no es valido";
Fin Si

Sino
Presentar "Error: el codigo debe tener 4 digitos";
Fin Si

Fin
```

Figura 35: pseudocódigo del ejercicio anterior 366

6.4. Prueba de escritorio

Prueba 1: Código válido

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	codigo		Por favor ingrese el codigo: 4557
2	d1 = 4, d2 = 5, d3 = 5, d4 = 7	(codigo >= 1000 && codigo <= 9999) = TRUE	
3	digito_veri	(4*1 + 5*2 + 5*3 + 7*4) = 57 % 10 = **7**	
4	codigo_vali	(**7 == 7**) = TRUE	
5	Presentar	codigo_valido = TRUE	El digito uno es: 4 El digito dos es: 5 El digito tres es: 5 El digito cuatro es: 7 El digito verificador es: 7 El codigo es valido.

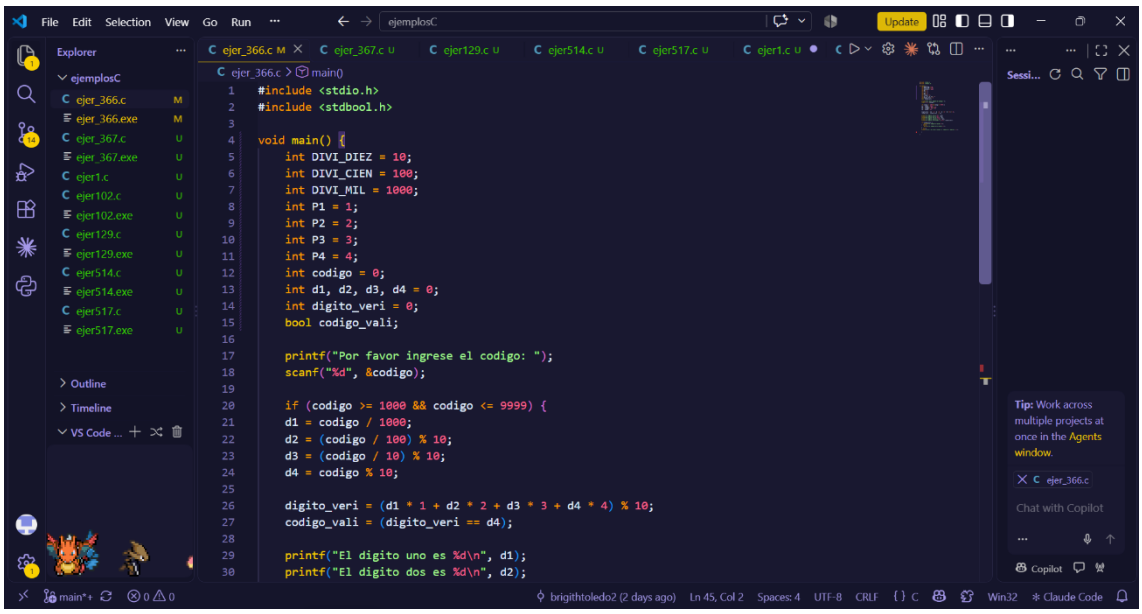
Prueba 2: Código no válido

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	codigo		Por favor ingrese el codigo: 1234
2	d1 = 1, d2 = 2, d3 = 3, d4 = 4	(codigo >= 1000 && codigo <= 9999) = TRUE	
3	digito_veri	(1*1 + 2*2 + 3*3 + 4*4) % 10 = **0**	
4	codigo_vali	(**0 == 4**) = FALSE	
5	Presentar	codigo_valido = FALSE	El digito verificador es: 0 El codigo no es valido

Prueba 3: Código sin los 4 dígitos

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	codigo		Por favor ingrese el codigo: 80
2	d1 = 8, d2 = 0, d3 = ?, d4 = ?	(codigo >= 1000 && codigo <= 9999) = FALSE	Error: el codigo debe tener 4 digitos

6.5. Codificación en lenguaje C



```c
#include <stdio.h>
#include <stdbool.h>

void main() {
    int DIVI_DIEZ = 10;
    int DIVI_CIEN = 100;
    int DIVI_MIL = 1000;
    int P1 = 1;
    int P2 = 2;
    int P3 = 3;
    int P4 = 4;
    int codigo = 0;
    int d1, d2, d3, d4 = 0;
    int digito_veri = 0;
    bool codigo_vali;

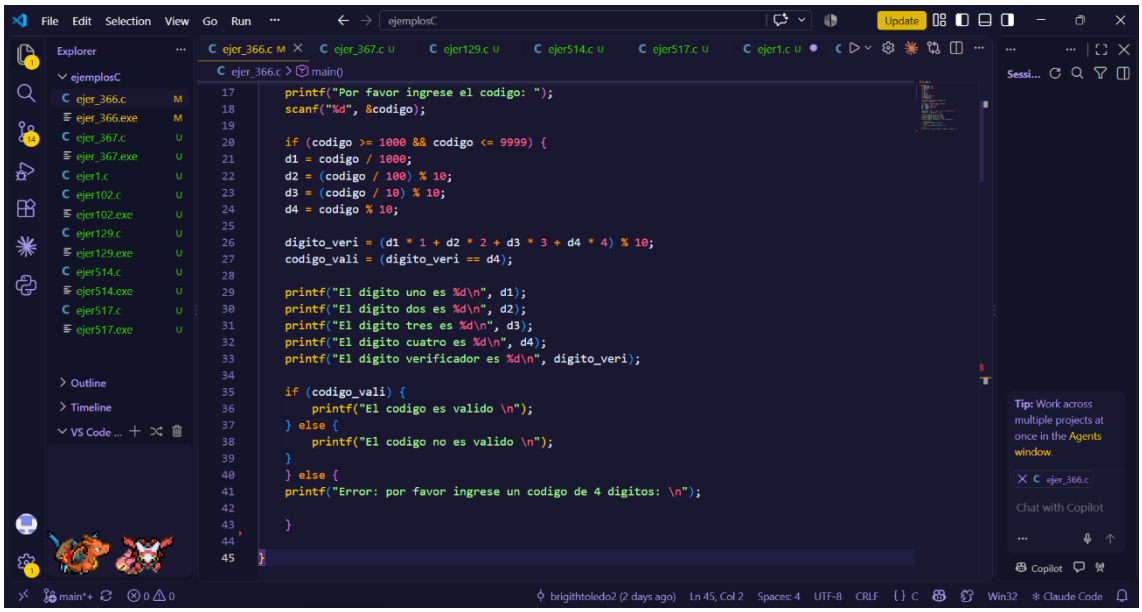
    printf("Por favor ingrese el codigo: ");
    scanf("%d", &codigo);

    if (codigo >= 1000 && codigo <= 9999) {
    d1 = codigo / 1000;
    d2 = (codigo / 100) % 10;
    d3 = (codigo / 10) % 10;
    d4 = codigo % 10;

    digito_veri = (d1 * 1 + d2 * 2 + d3 * 3 + d4 * 4) % 10;
    codigo_vali = (digito_veri == d4);

    printf("El digito uno es %d\n", d1);
    printf("El digito dos es %d\n", d2);
```

Figura 36: código del ejercicio anterior 366, variables y proceso



```c
    printf("Por favor ingrese el codigo: ");
    scanf("%d", &codigo);

    if (codigo >= 1000 && codigo <= 9999) {
    d1 = codigo / 1000;
    d2 = (codigo / 100) % 10;
    d3 = (codigo / 10) % 10;
    d4 = codigo % 10;

    digito_veri = (d1 * 1 + d2 * 2 + d3 * 3 + d4 * 4) % 10;
    codigo_vali = (digito_veri == d4);

    printf("El digito uno es %d\n", d1);
    printf("El digito dos es %d\n", d2);
    printf("El digito tres es %d\n", d3);
    printf("El digito cuatro es %d\n", d4);
    printf("El digito verificador es %d\n", digito_veri);

    if (codigo_vali) {
        printf("El codigo es valido \n");
    } else {
        printf("El codigo no es valido \n");
    }
    } else {
    printf("Error: por favor ingrese un codigo de 4 digitos: \n");

    }
}
```

Figura 37: código del ejercicio anterior 366, salida

6.6. Salida del programa

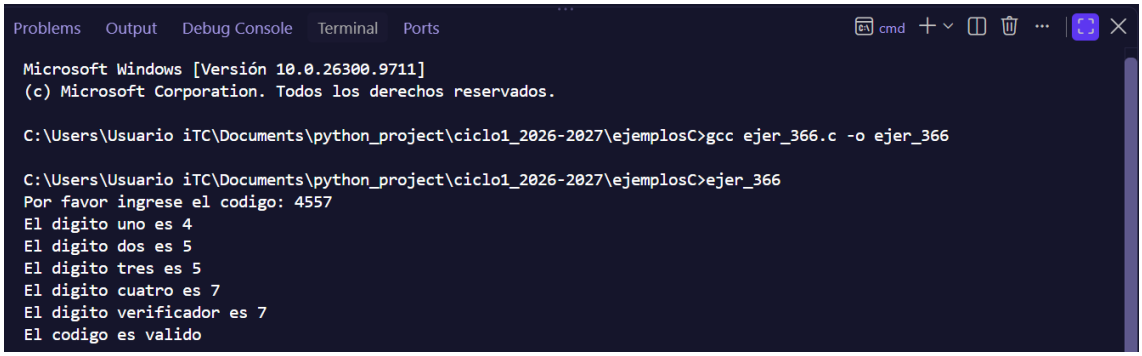


Figura 38: comprobación de la prueba 1

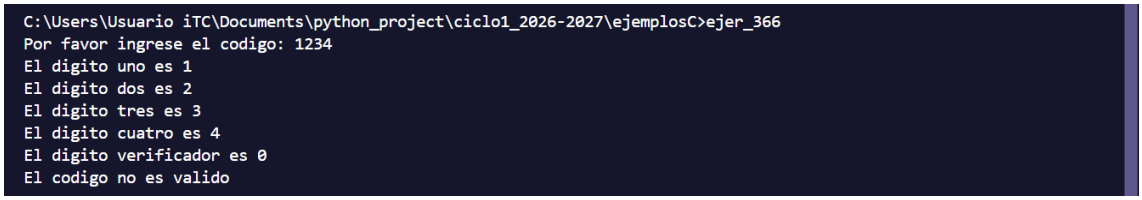


Figura 39: comprobación de la prueba 2

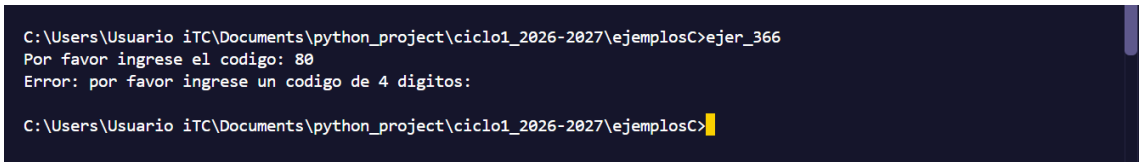


Figura 40: comprobación de la prueba 3

7. Enunciado del algoritmo

7) EJ-367

Construya un algoritmo que simule el cálculo de utilidades de una empresa. Lea los ingresos totales (real), los costos fijos (real), los costos variables (real) y el porcentaje de impuestos (real). Use constantes simbólicas: PORCENTAJE = 100, UMBRAL_UTILIDAD = 0.0, BONO_EMPLEADOS = 0.10. Calcule: utilidad_bruta = ingresos - (costos_fijos + costos_variables); impuestos = utilidad_bruta * impuestos / PORCENTAJE; utilidad_neta = utilidad_bruta - impuestos; bono = (utilidad_neta > UMBRAL_UTILIDAD) ? utilidad_neta * BONO_EMPLEADOS : 0. Evalúe la expresión lógica (utilidad_neta > UMBRAL_UTILIDAD). Muestre por pantalla el desglose.

7.1. Análisis del problema

Datos de entrada:

- Ingresos totales
- Costos fijos
- Costos variables
- Porcentaje de impuestos

Proceso

- utilidad_bruta = ingresos_totales - (costos_fijos + costos_variables)
- impuestos = utilidad_bruta * porcentaje_impuestos / PORCENTAJE
- utilidad_neta = utilidad_bruta - impuestos
- es_rentable = (utilidad_neta > UMBRAL_UTILIDAD)
- bono = (es_rentable) ? utilidad_neta * BONO_EMPLEADOS : 0

Datos de salida:

- Utilidad bruta
- Impuestos
- Utilidad Neta
- Bono

7.2. Diagrama de flujo

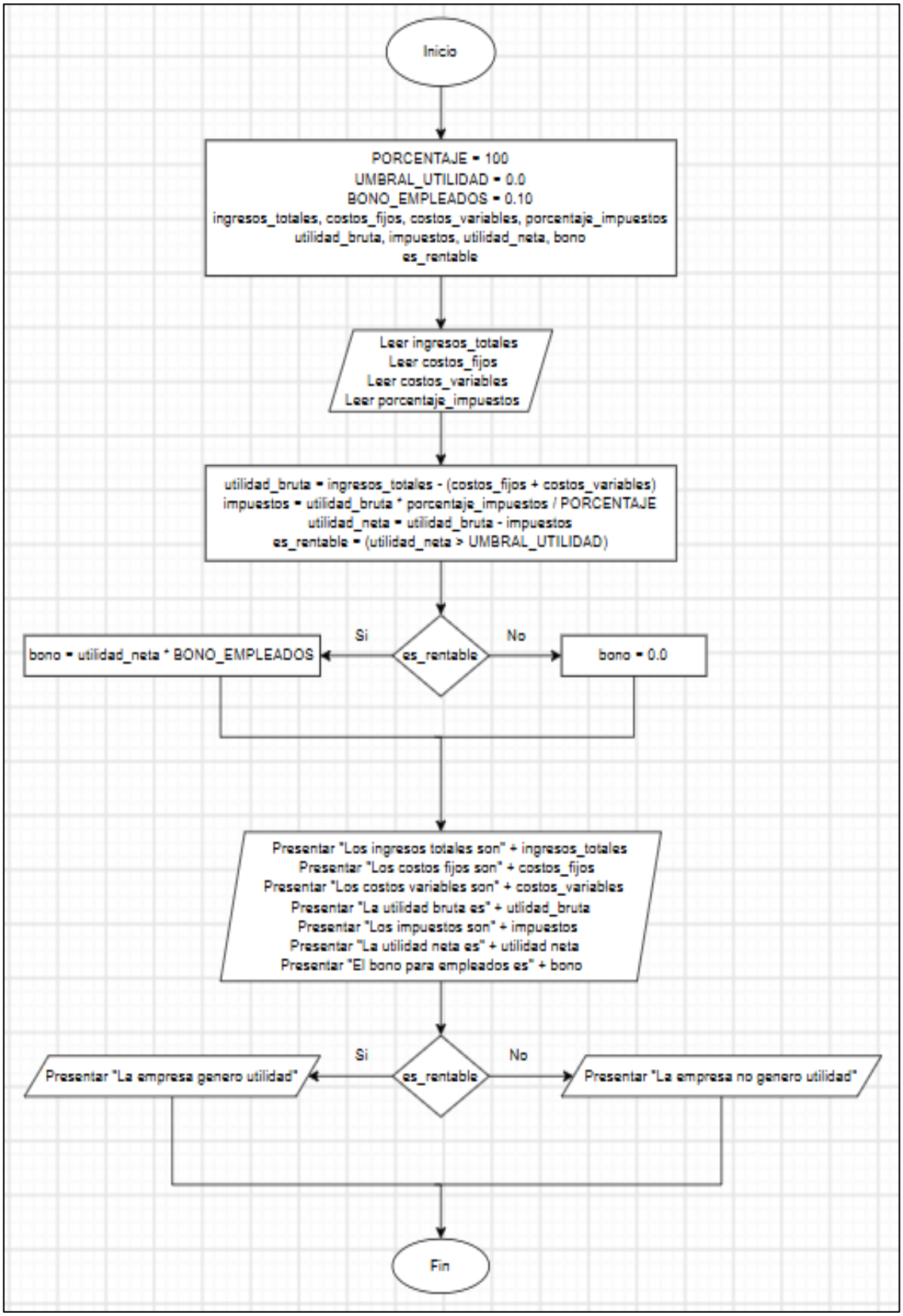


Figura 41: diagrama de flujo ejercicio anterior 367 en Draw.io

7.3. Pseudocódigo

```
Inicio

PORCENTAJE =100;
UMBRAL_UTILIDAD = 0.0;
BONO_EMPLEADOS = 0.10;
ingresos_totales, costos_fijos, costos_variables, porcentaje_impuestos;
utilidad_bruta, impuestos, utilidad_neta, bono;
es_rentable;

Leer ingresos_totales;
Leer costos_fijos;
Leer costos_variables;
Leer porcentaje_impuestos;

utilidad_bruta = ingresos_totales - (costos_fijos + costos_variables);
impuestos = utilidad_bruta * porcentaje_impuestos / PORCENTAJE;
utilidad_neta = utilidad_bruta - impuestos;
es_rentable = (utilidad_neta > UMBRAL_UTILIDAD);

Si (es_rentable) entonces
bono = utilidad_neta * BONO_EMPLEADOS;
Sino
bono = 0.0;
Fin Si

Presentar "Los ingresos totales son" + ingresos_totales;
Presentar "Los costos fijos son" + costos_fijos;
Presentar "Los costos variables son" + costos_variables;
Presentar "La utilidad bruta es" + utlidad_bruta;
Presentar "Los impuestos son" + impuestos;
Presentar "La utilidad neta es" + utilidad neta;
Presentar "El bono para empleados es" + bono;

Si (es_rentable) entonces
Presentar "La empresa genero utilidad";
Sino
Presentar "La empresa no genero utilidad";

Fin
```

Figura 42: pseudocódigo del ejercicio anterior 367

7.4. Prueba de escritorio

Prueba 1: La empresa genero utilidad

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	ingresos_totales costos_fijos costos_variables porcentaje_impuestos		Ingrese los ingresos totales: 1000 Ingrese los costos fijos: 400 Ingrese los costos variables: 300 Ingrese el porcentaje de impuestos: 0.15
2	utilidad_bruta	ingresos_totales - (costos_fijos + costos_variables) 1000 – (400 + 300) = 300.00	
3	impuestos	utilidad_bruta * porcentaje_impuestos / PORCENTAJE 300.00 * 0.15 / 100.00 = 0.45	
4	utilidad_neta	utilidad_bruta – impuestos 300.00 - 0.45 = 299.55	
5	es_rentable	(utilidad_neta > UMBRAL_UTILIDAD) 299.55 > 0.0 = TRUE	
6	bono	utilidad_neta * BONO_EMPLEADOS 299.55 * 0.10 = 29.955	
7	Presentar		Los ingresos totales son: $ 1000.00 Los costos fijos son: $ 400 Los costos variables son: $ 300.00 La utilidad bruta es: $ 300.00 Los impuestos son: $ 0.45 La utilidad neta es: $ 299.55 El bono para empleados es: $ 29.955 La empresa genero utilidad

Prueba 2: La empresa no genero utilidad

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	ingresos_totales costos_fijos costos_variables porcentaje_impuestos		Ingrese los ingresos totales: 500 Ingrese los costos fijos: 200 Ingrese los costos variables: 350 Ingrese el porcentaje de impuestos: 0.20
2	utilidad_bruta	ingresos_totales - (costos_fijos + costos_variables) 500 – (200 + 350) = -50.00	
3	impuestos	utilidad_bruta * porcentaje_impuestos / PORCENTAJE -50.00 * 0.20 / 100.00 = -0.10	
4	utilidad_neta	utilidad_bruta – impuestos -50.00 – (-0.10) = -49.90	
5	es_rentable	(utilidad_neta > UMBRAL_UTILIDAD) -49.90 > 0.0 = FALSE	
6	bono	0.0	
7	Presentar		Los ingresos totales son: $ 500.00 Los costos fijos son: $ 200 Los costos variables son: $ 350.00 La utilidad bruta es: $ -50.00 Los impuestos son: $ -0.10 La utilidad neta es: $ -49.90 El bono para empleados es: $ 0.00 La empresa no genero utilidad

7.5. Codificación en lenguaje C

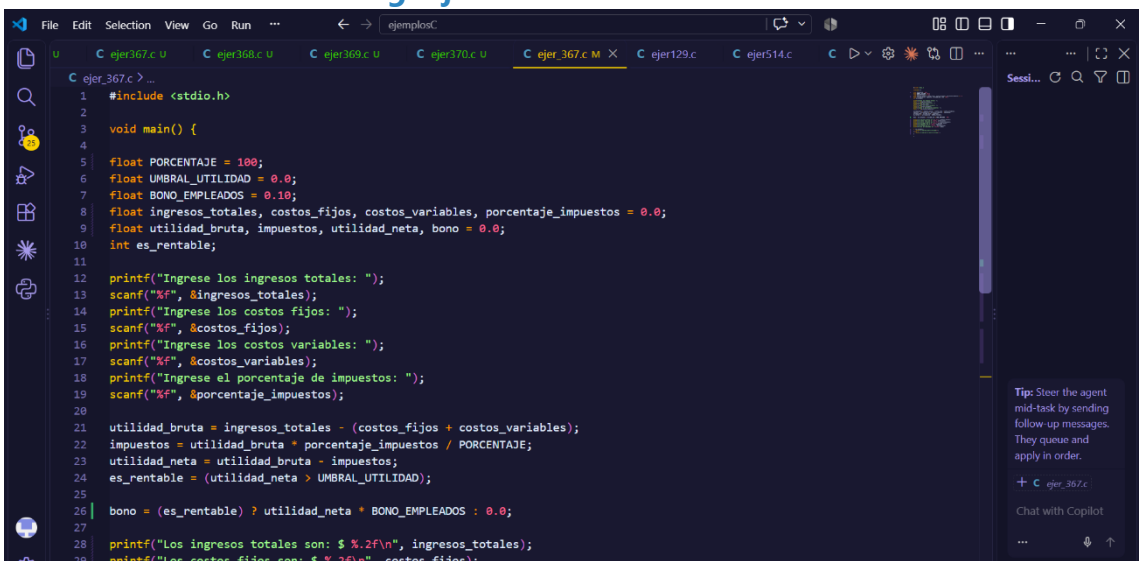


Figura 43: código del ejercicio anterior 367, variables y proceso

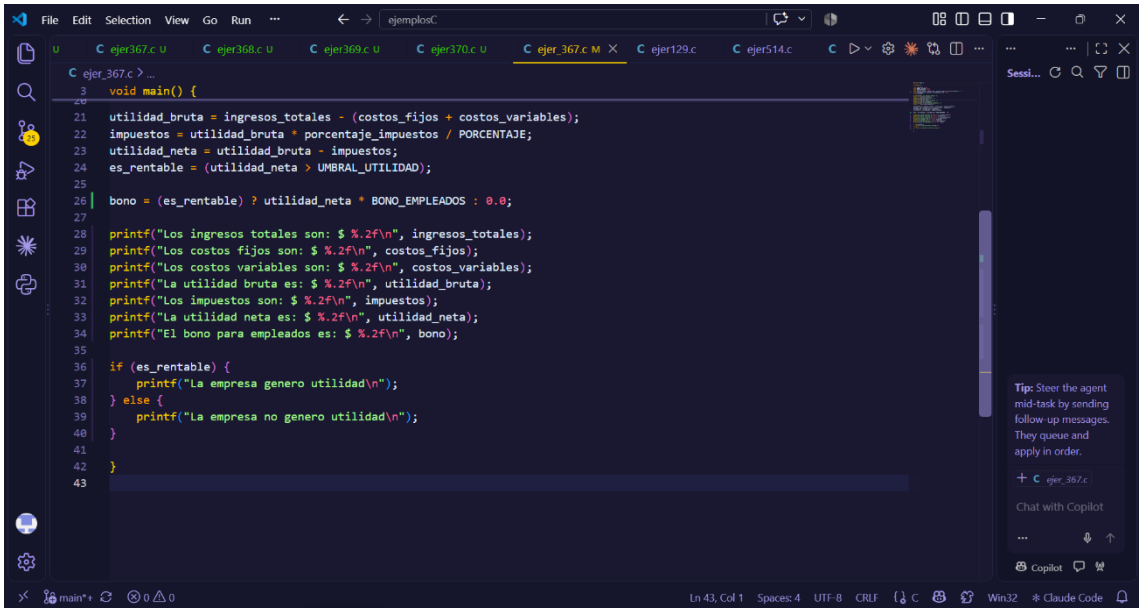


Figura 44: código del ejercicio anterior 367, salida

7.6. Salida del programa

```
Problems  Output  Debug Console  Terminal  Ports

Microsoft Windows [Versión 10.0.26300.9711]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>gcc ejer_367.c -o ejer_367

C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>ejer_367
Ingrese los ingresos totales: 1000
Ingrese los costos fijos: 400
Ingrese los costos variables: 300
Ingrese el porcentaje de impuestos: 0.15
Los ingresos totales son: $ 1000.00
Los costos fijos son: $ 400.00
Los costos variables son: $ 300.00
La utilidad bruta es: $ 300.00
Los impuestos son: $ 0.45
La utilidad neta es: $ 299.55
El bono para empleados es: $ 29.95
La empresa genero utilidad
```

Figura 45: comprobación de la prueba 1

```
C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>ejer_367
Ingrese los ingresos totales: 500
Ingrese los costos fijos: 200
Ingrese los costos variables: 350
Ingrese el porcentaje de impuestos: 0.20
Los ingresos totales son: $ 500.00
Los costos fijos son: $ 200.00
Los costos variables son: $ 350.00
La utilidad bruta es: $ -50.00
Los impuestos son: $ -0.10
La utilidad neta es: $ -49.90
El bono para empleados es: $ 0.00
La empresa no genero utilidad
```

Figura 46: comprobación de la prueba 2

8. Enunciado del algoritmo

8) EJ-129

Cree un algoritmo que calcule el área de un círculo. Declare una constante simbólica PI con valor 3.14159 y una variable real radio. Lea el radio desde teclado. Calcule el área usando la expresión aritmética (PI * radio * radio) y asígnela a la variable area. Muestre por pantalla el resultado con un mensaje informativo. Justifique por qué se usa una constante en lugar de una variable para PI.

Justificación: PI se usa como constante ya que su valor es fijo, o sea que no cambiar, si fuera una variable tendría diferentes resultados siempre, algo que daría un error en el programa.

8.1. Análisis del problema

Datos de entrada:

- PI (3.14159)
- Radio

Proceso

- Radio <= 0
- area = (PI * radio * radio)

Datos de salida:

- Área del circulo

8.2. Diagrama de flujo

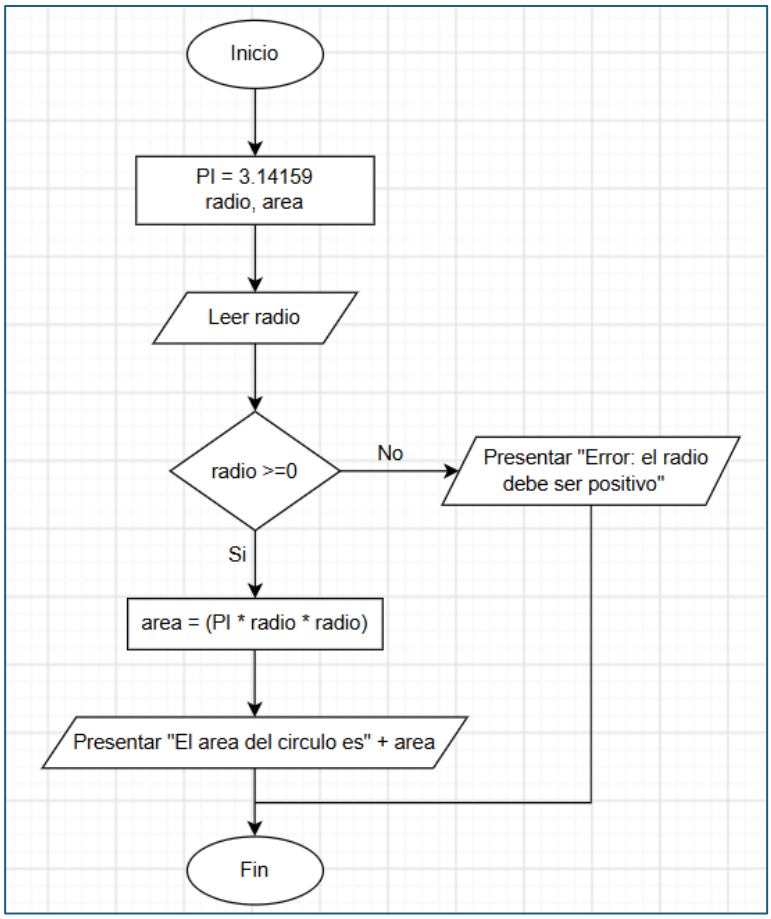


Figura 47: diagrama de flujo ejercicio anterior 129 en Draw.io

8.3. Pseudocódigo

```
Inicio

PI = 3.14159;
radio, area;

Leer radio;

Si (radio >=0) entonces
area = (PI * radio * radio);
Presentar "El area del circulo es" + area;

Sino
Presentar "Error: el radio debe ser positivo";
Fin Si

Fin
```

Figura 48: pseudocódigo del ejercicio anterior 129

8.4. Prueba de escritorio

Prueba 1: El radio es mayor que 0

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1			Ingrese el radio del circulo: 8
2	radio	(radio <=0) 8 <= 0 = TRUE	
3	area	(PI * radio * radio) 3.14159 * 8 * 8 = 201.0617	
4	Presentar		El area del circulo es: 201.0617

Prueba 2: El radio no es mayor que 0

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1			Ingrese el radio del circulo: -4
2	radio	(radio <=0) -4 <= 0 = FALSE	
4	Presentar		Error: el radio debe ser positivo

8.5. Codificación en lenguaje C

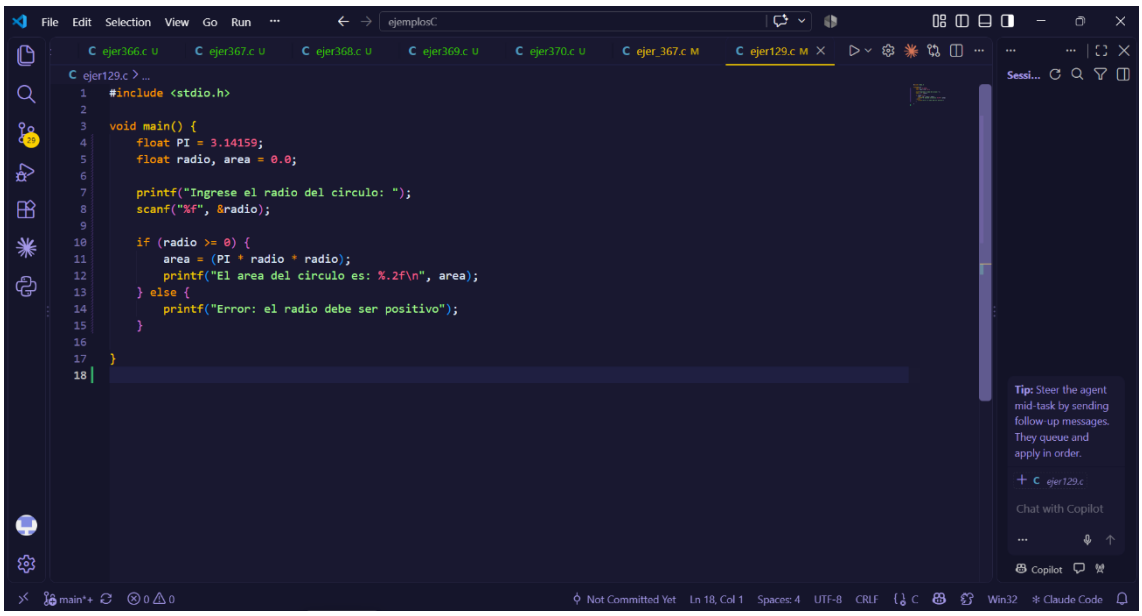


Figura 49: código del ejercicio anterior 129, variables, proceso y salida

8.6. Salida del programa

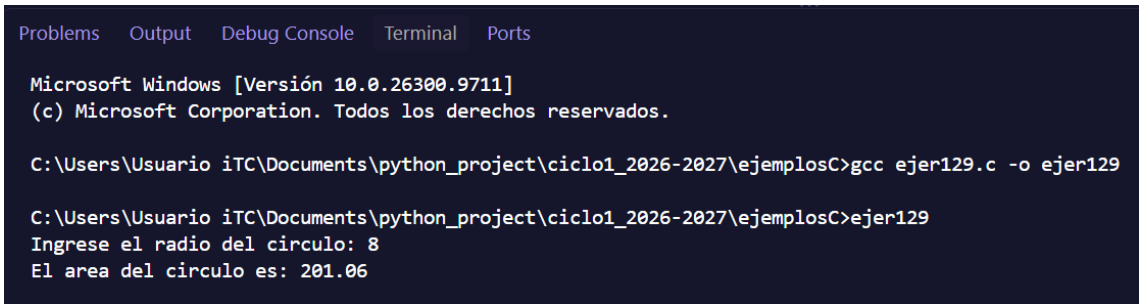


Figura 50: comprobación de la prueba 1

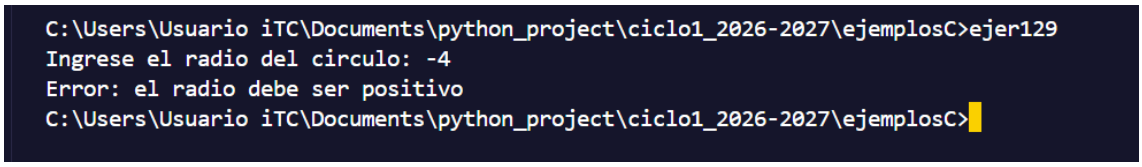


Figura 51: comprobación de la prueba 2

9. Enunciado del algoritmo

9) EJ-514

Declare las variables x (entero) y a (real). Lea ambos valores desde teclado. Use la constante literal 100 para calcular el porcentaje: evalúe la expresión aritmética (x * a / 100) y asígnela a la variable porcentaje. Evalúe la expresión lógica (porcentaje > 50 AND porcentaje < 90). Muestre por pantalla el valor del porcentaje y el resultado booleano de la expresión lógica.

9.1. Análisis del problema

Datos de entrada:

- Números x y a
- División 100

Proceso

- porcentaje = (x * a / 100)
- condicion = (porcentaje > 50 && porcentaje < 90)

Datos de salida:

- Porcentaje
- Resultado de la expresión lógica

9.2. Diagrama de flujo

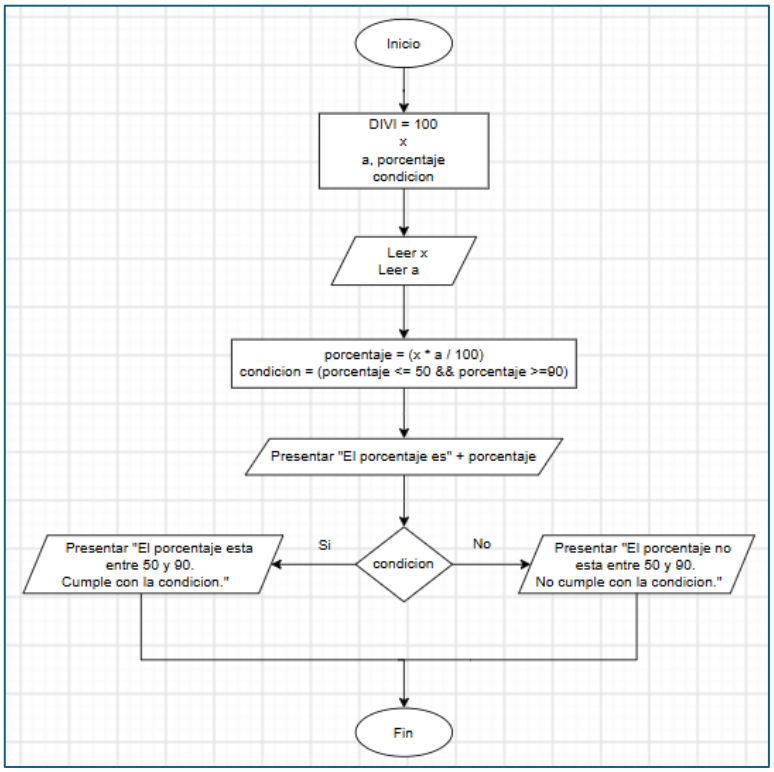


Figura 52: diagrama de flujo ejercicio anterior 514 en Draw.io

9.3. Pseudocódigo

```
Inicio

DIVI = 100;
x;
a, porcentaje;
condición;

Leer x;
Leer a;

porcentaje = (x * a / 100);
condicion = (porcentaje <= 50 && porcentaje >=90);

Presentar "El porcentaje es" + porcentaje;

Si (condición) entonces
Presentar “El porcentaje esta entre 50 y 90. Cumple con la condicion.”;
Sino
Presentar "El porcentaje no esta entre 50 y 90. No cumple con la condicion.";

Fin
```

Figura 53: pseudocódigo del ejercicio anterior 514

9.4. Prueba de escritorio

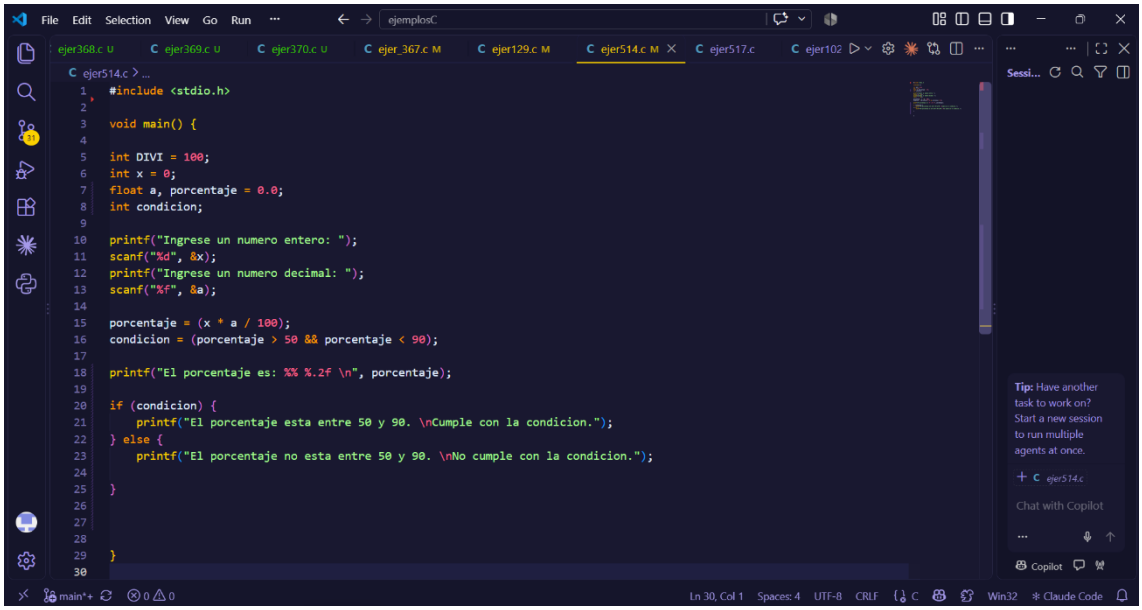
Prueba 1: El porcentaje cumple la expresión lógica

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	x a		Ingrese un numero entero: 100 Ingrese un numero decimal: 70.0
2	porcentaje	(x * a / 100) (100 * 70.0 / 100) = 70	
3	condicion	(porcentaje <= 50 && porcentaje >=90) (70 <= 50 && 70 >=90) = TRUE	
4	Presentar		El porcentaje es: % 70 El porcentaje esta entre 50 y 90. Cumple con la condicion.

Prueba 2: El porcentaje no cumple la expresión lógica

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	x a		Ingrese un numero entero: 20 Ingrese un numero decimal: 50.0
2	porcentaje	(x * a / 100) (20 * 50.0 / 100) = 10	
3	condicion	(porcentaje <= 50 && porcentaje >=90) (10 <= 50 && 10 >=90) = FALSE	
4	Presentar		El porcentaje es: % 10 El porcentaje no esta entre 50 y 90. No cumple con la condicion.

9.5. Codificación en lenguaje C



```c
#include <stdio.h>

void main() {

int DIVI = 100;
int x = 0;
float a, porcentaje = 0.0;
int condicion;

printf("Ingrese un numero entero: ");
scanf("%d", &x);
printf("Ingrese un numero decimal: ");
scanf("%f", &a);

porcentaje = (x * a / 100);
condicion = (porcentaje > 50 && porcentaje < 90);

printf("El porcentaje es: %% %.2f \n", porcentaje);

if (condicion) {
    printf("El porcentaje esta entre 50 y 90. \nCumple con la condicion.");
} else {
    printf("El porcentaje no esta entre 50 y 90. \nNo cumple con la condicion.");

}


}
```

Figura 54: código del ejercicio anterior 514, variables, proceso y salida

9.6. Salida del programa

```
Problems  Output  Debug Console  Terminal  Ports

Microsoft Windows [Versión 10.0.26300.9711]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>gcc ejer514.c -o ejer514

C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>ejer514
Ingrese un numero entero: 100
Ingrese un numero decimal: 70.0
El porcentaje es: % 70.00
El porcentaje esta entre 50 y 90.
Cumple con la condicion.
```

Figura 55: comprobación de la prueba 1

```
C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>ejer514
Ingrese un numero entero: 20
Ingrese un numero decimal: 50.0
El porcentaje es: % 10.00
El porcentaje no esta entre 50 y 90.
No cumple con la condicion.
```

Figura 56: comprobación de la prueba 2

10. Enunciado del algoritmo

10) EJ-517

Construya un algoritmo que lea el nombre de un estudiante (cadena) y tres notas (reales). Use una constante simbólica APROBADO = 7.0 como nota mínima de aprobación. Calcule el promedio con la expresión aritmética ((nota1 + nota2 + nota3) / 3). Evalúe la expresión relacional (promedio >= APROBADO). Muestre por pantalla un mensaje que incluya el nombre del estudiante, su promedio y su condición (aprobado o reprobado).

10.1. Análisis del problema

Datos de entrada:

- Nombre del estudiante
- Notas
- Nota mínima de aprobación (7.0)

Proceso

- promedio = (nota1 + nota2 + nota3) / 3
- condicion = (promedio >= APROBADO)

Datos de salida:

- El promedio de las notas
- Expresión racional de la aprobación o no del estudiante

10.2. Diagrama de flujo

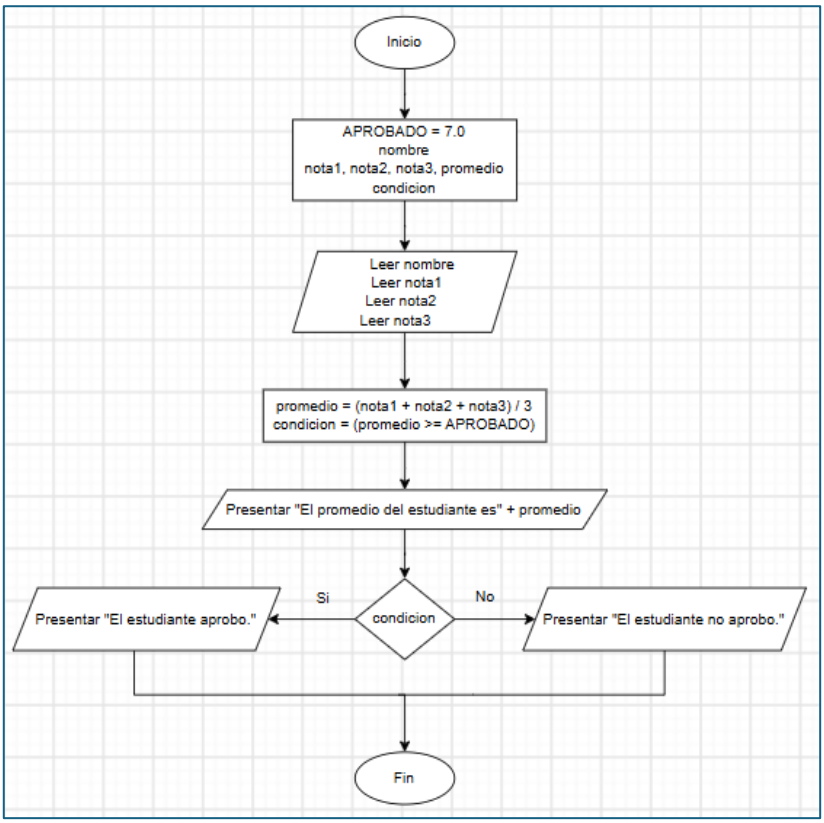


Figura 57: diagrama de flujo ejercicio anterior 517 en Draw.io

10.3. Pseudocódigo

```
Inicio

APROBADO = 7.0;
nombre;
nota1, nota2, nota3, promedio;
condicion;

Leer nombre;
Leer nota1;
Leer nota2;
Leer nota3;

promedio = (nota1 + nota2 + nota3) / 3;
condicion = (promedio >= APROBADO);

Presentar "El promedio del estudiante es" + promedio;

Si (condición) entonces
Presentar "El estudiante aprobo.";

Sino
Presentar " El estudiante no aprobo.";

Fin
```

Figura 58: pseudocódigo del ejercicio anterior 517

10.4. Prueba de escritorio

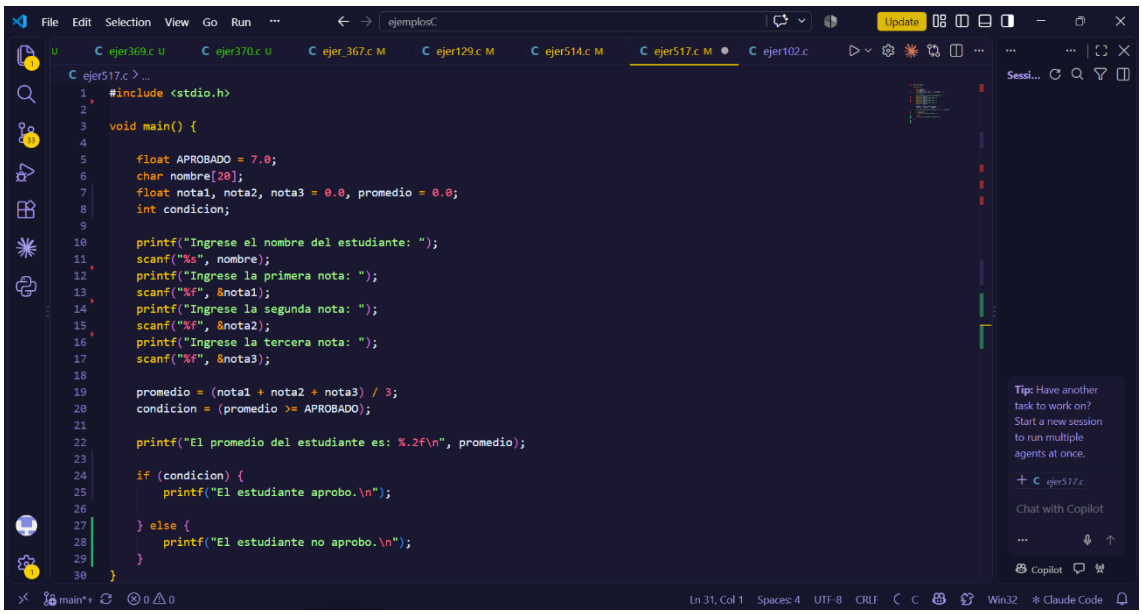
Prueba 1: El estudiante aprobó

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	nombre nota1 nota2 nota3		Ingrese el nombre del estudiante: Samuel Ingrese la primera nota: 10 Ingrese la segunda nota: 8.5 Ingrese la tercera nota: 7
2	promedio	(nota1 + nota2 + nota3) / 3 (10 + 8.5 + 7) / 3 = 8.50	
3	condicion	(promedio >= APROBADO) (8.5 >= 7.0) = TRUE	
4	Presentar		El promedio del estudiante es: 8.50 El estudiante aprobo.

Prueba 2: El estudiante no aprobó

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	nombre nota1 nota2 nota3		Ingrese el nombre del estudiante: Marta Ingrese la primera nota: 8.5 Ingrese la segunda nota: 6.8 Ingrese la tercera nota: 5
2	promedio	(nota1 + nota2 + nota3) / 3 (8.5 + 6.8 + 5) / 3 = 6.766	
3	condicion	(promedio >= APROBADO) (6.766 >= 7.0) = FALSE	
4	Presentar		El promedio del estudiante es: 6.766 El estudiante no aprobo.

10.5. Codificación en lenguaje C



```c
#include <stdio.h>

void main() {

    float APROBADO = 7.0;
    char nombre[20];
    float nota1, nota2, nota3 = 0.0, promedio = 0.0;
    int condicion;

    printf("Ingrese el nombre del estudiante: ");
    scanf("%s", nombre);
    printf("Ingrese la primera nota: ");
    scanf("%f", &nota1);
    printf("Ingrese la segunda nota: ");
    scanf("%f", &nota2);
    printf("Ingrese la tercera nota: ");
    scanf("%f", &nota3);

    promedio = (nota1 + nota2 + nota3) / 3;
    condicion = (promedio >= APROBADO);

    printf("El promedio del estudiante es: %.2f\n", promedio);

    if (condicion) {
        printf("El estudiante aprobo.\n");

    } else {
        printf("El estudiante no aprobo.\n");
    }
}
```

Figura 59: código del ejercicio anterior 517, variables, proceso y salida

10.6. Salida del programa

```
Problems  Output  Debug Console  Terminal  Ports

Microsoft Windows [Versión 10.0.26300.9711]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>gcc ejer517.c -o ejer517

C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>ejer517
Ingrese el nombre del estudiante: Samuel
Ingrese la primera nota: 10
Ingrese la segunda nota: 8.5
Ingrese la tercera nota: 7
El promedio del estudiante es: 8.50
El estudiante aprobo.
```

Figura 60: comprobación de la prueba 1

```
C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>ejer517
Ingrese el nombre del estudiante: Marta
Ingrese la primera nota: 8.5
Ingrese la segunda nota: 6.8
Ingrese la tercera nota: 5
El promedio del estudiante es: 6.77
El estudiante no aprobo.
```

Figura 61: comprobación de la prueba 2

11. Enunciado del algoritmo

11) EJ-102

En una tienda de electrónica, cree un algoritmo que lea dos números enteros z y s por teclado. Evalúe la expresión lógica (s ! = 0) para evitar división por cero. Calcule el cociente entero con (z / s) y el residuo con (z % s). Asigne los resultados a cociente y residuo. Muestre por pantalla ambos valores. Adicionalmente, calcule la diferencia entre el resultado y el promedio. Muestre al inicio una lista de las variables que se utilizaran.

11.1. Análisis del problema

Datos de entrada:

- Números z y s

Proceso

- condicion = (s ! = 0)
- cociente = (z / s)
- residuo = (z % s)
- promedio = (z + s) / 2.0
- diferencia = cociente – promedio

Datos de salida:

- Promedio
- Diferencia entre el resultado
- Cociente
- Residuo

11.2. Diagrama de flujo

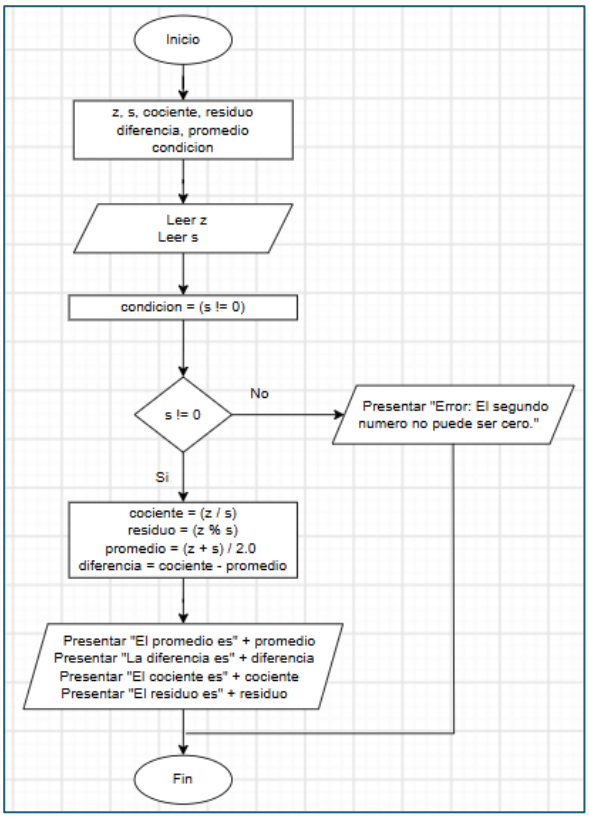


Figura 62: diagrama de flujo del ejercicio anterior 102 en Draw.io

11.3. Pseudocódigo

```
Inicio

z, s, cociente, residuo;
diferencia, promedio;
condicion;

Leer z;
Leer s;

condicion = (s != 0);

Si (s != 0) entonces
cociente = (z / s);
residuo = (z % s);
promedio = (z + s) / 2.0;
diferencia = cociente – promedio;

Presentar "El promedio es" + promedio;
Presentar "La diferencia es" + diferencia;
Presentar "El cociente es" + cociente;
Presentar "El residuo es" + residuo;

Sino
Presentar "Error: El segundo numero no puede ser cero.";
Fin Si

Fin
```

Figura 63: pseudocódigo del ejercicio anterior 102

11.4. Prueba de escritorio

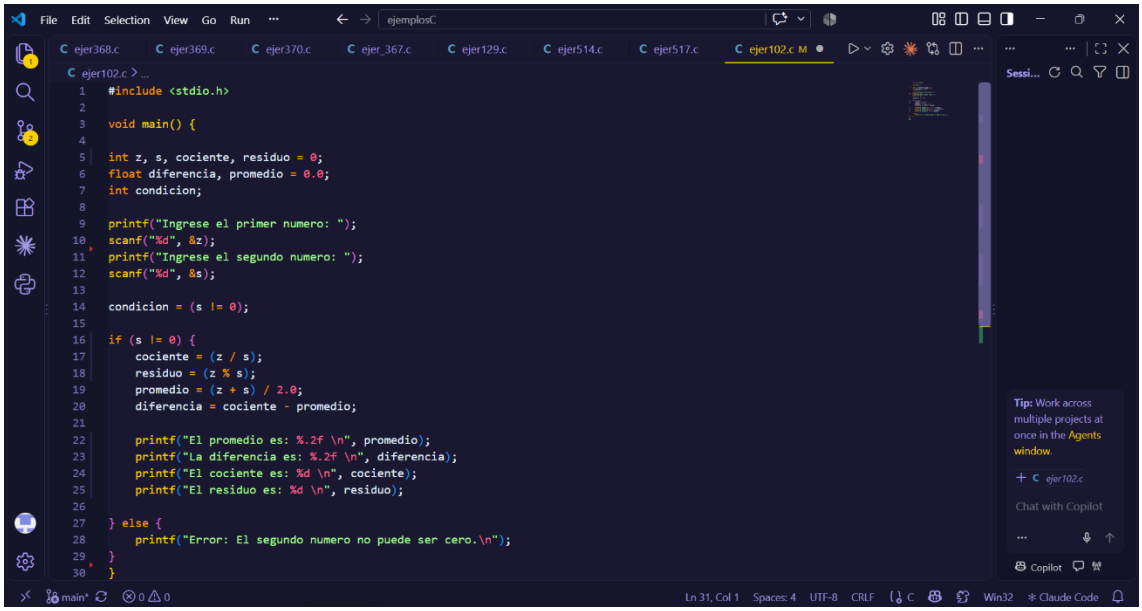
Prueba 1: Evita la división por 0

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	z s		Ingrese el primer numero: 8 Ingrese el segundo numero: 10
2	condicion	(s != 0) (10 != 0) = TRUE	
3	cociente	(z / s) (8 / 10) = 0.8 = 0 (int)	
4	residuo	(z % s) (8 % 10) = 8	
5	promedio	(z + s) / 2.0 (8 + 10) = 9.00	
6	diferencia	cociente – promedio 0 – 9 = -9.00	
7	Presentar		El promedio es: 9.00 La diferencia es: -9.00 El cociente es: 0 El residuo es: 8

Prueba 2: Se divide por 0

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	z s		Ingrese el primer numero: 10 Ingrese el segundo numero: 0
2	condicion	(s != 0) (0 != 0) = FALSE	
3	Presentar		Error: El segundo numero no puede ser cero

11.5. Codificación en lenguaje C



```c
#include <stdio.h>

void main() {

int z, s, cociente, residuo = 0;
float diferencia, promedio = 0.0;
int condicion;

printf("Ingrese el primer numero: ");
scanf("%d", &z);
printf("Ingrese el segundo numero: ");
scanf("%d", &s);

condicion = (s != 0);

if (s != 0) {
    cociente = (z / s);
    residuo = (z % s);
    promedio = (z + s) / 2.0;
    diferencia = cociente - promedio;

    printf("El promedio es: %.2f \n", promedio);
    printf("La diferencia es: %.2f \n", diferencia);
    printf("El cociente es: %d \n", cociente);
    printf("El residuo es: %d \n", residuo);

} else {
    printf("Error: El segundo numero no puede ser cero.\n");
}
}
```

Figura 64: código del ejercicio anterior 102, variables, proceso y salida

11.6. Salida del programa

```
Problems  Output  Debug Console  Terminal  Ports

Microsoft Windows [Versión 10.0.26300.9711]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>gcc ejer102.c -o ejer102

C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>ejer102
Ingrese el primer numero: 8
Ingrese el segundo numero: 10
El promedio es: 9.00
La diferencia es: -9.00
El cociente es: 0
El residuo es: 8
```

Figura 65: comprobación de la prueba 1

```
C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>ejer102
Ingrese el primer numero: 10
Ingrese el segundo numero: 0
Error: El segundo numero no puede ser cero.
```

Figura 66: comprobación de la prueba 2

12. Enunciado del algoritmo

Nuevos ejercicios

12) EJ-406

En una zapatería, construya un algoritmo que lea un peso en onzas (real). Use una constante simbólica ONZA_GRAMO = 28.3495. Calcule la equivalencia en gramos con la expresión aritmética (onzas * ONZA_GRAMO) y asígnela a gramos. Evalúe la expresión relacional (gramos > 0). Muestre por pantalla 'X onzas equivalen a Y gramos'. Además, calcule el inverso multiplicativo del resultado si no es cero. Muestre una tabla simulada con los datos y resultados al final.

12.1. Análisis del problema

Datos de entrada:

- Peso en onzas
- Onza en gramo (28.3495)

Proceso

- gramos = (onzas * ONZA_GRAMO)
- condicion = (gramos > 0)
- inverso = (1 / gramos)

Datos de salida:

- 'X onzas equivalen a Y gramos'
- Inverso multiplicativo del resultado si no es cero

12.2. Diagrama de flujo

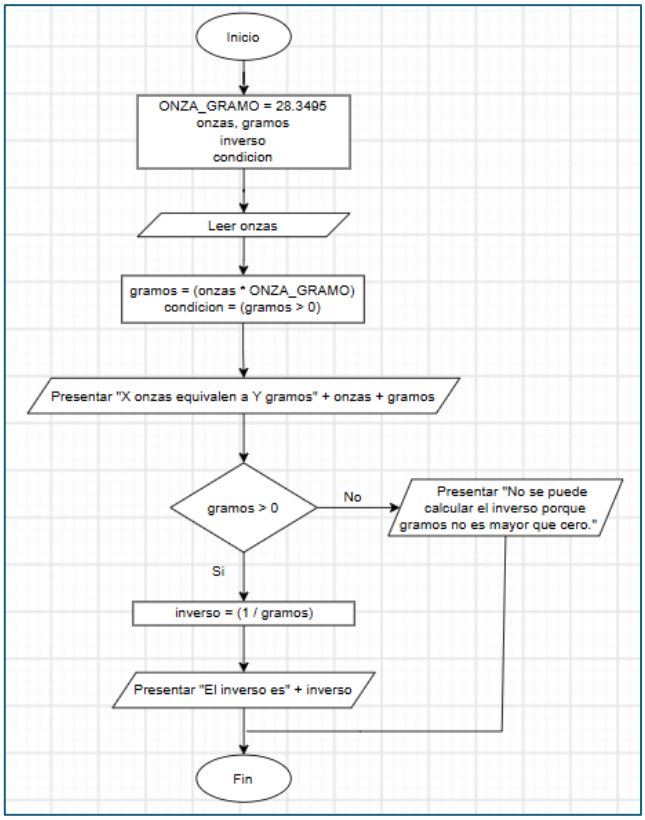


Figura 67: diagrama de flujo del ejercicio 406 en Draw.io

12.3. Pseudocódigo

```
Inicio

ONZA_GRAMO = 28.3495;
onzas, gramos;
inverso;
condición;

Leer onzas;

gramos = (onzas * ONZA_GRAMO);
condicion = (gramos > 0);

Presentar "X onzas equivalen a Y gramos" + onzas + gramos;

Si (gramos > 0) entonces
inverso = (1 / gramos);
Presentar "El inverso es" + inverso;

Sino
Presentar "No se puede calcular el inverso porque gramos no es mayor que cero.";
Fin Si

Fin
```

Figura 68: pseudocódigo del ejercicio 406

12.4. Prueba de escritorio

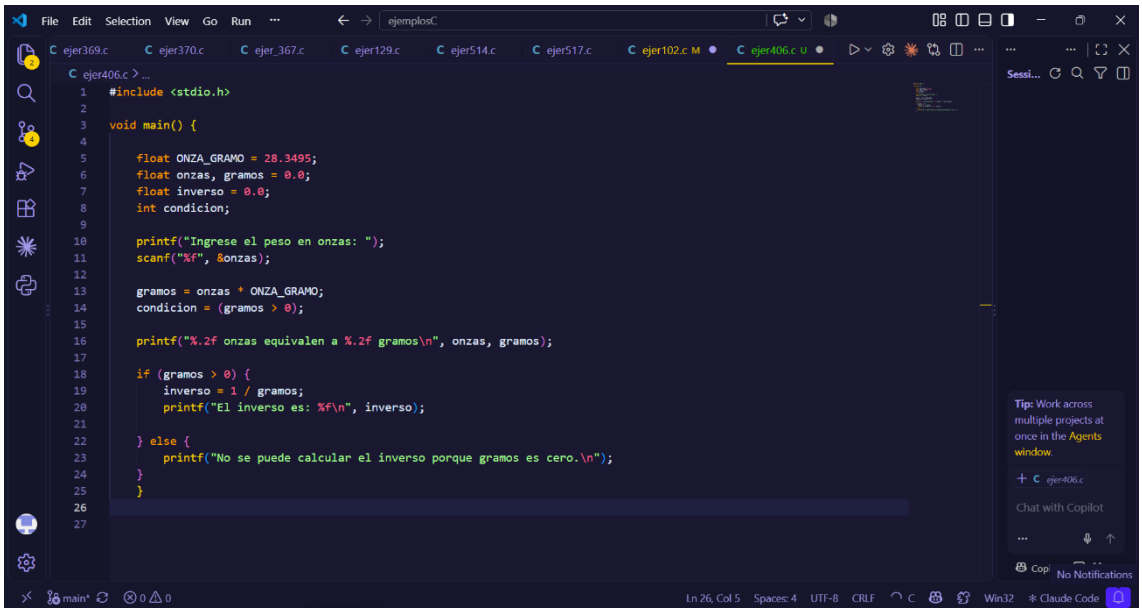
Prueba 1: Peso normal

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	onzas		Ingrese el peso en onzas: 5
2	Gramos	(onzas * ONZA_GRAMO) (5 * 28.3495) = 141.7475	
3	condicion	(gramos > 0) (141.7475 > 0) = TRUE	
4	Presentar		5.00 onzas equivalen a 141.74 gramos
5	inverso	(1 / gramos) (1 / 141.74) = 0.007055	
6	Presentar		El inverso es: 9.00

Prueba 2: Peso son resultado 0

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	onzas		Ingrese el peso en onzas: 0
2	Gramos	(onzas * ONZA_GRAMO) (0 * 28.3495) = 0.00	
3	condicion	(gramos > 0) (0 > 0) = FALSE	
4	Presentar		0.00 onzas equivalen a 0.00 gramos
5	Presentar		No se puede calcular el inverso porque gramos no es mayor que cero.

12.5. Codificación en lenguaje C



```c
#include <stdio.h>

void main() {

    float ONZA_GRAMO = 28.3495;
    float onzas, gramos = 0.0;
    float inverso = 0.0;
    int condicion;

    printf("Ingrese el peso en onzas: ");
    scanf("%f", &onzas);

    gramos = onzas * ONZA_GRAMO;
    condicion = (gramos > 0);

    printf("%.2f onzas equivalen a %.2f gramos\n", onzas, gramos);

    if (gramos > 0) {
        inverso = 1 / gramos;
        printf("El inverso es: %f\n", inverso);

    } else {
        printf("No se puede calcular el inverso porque gramos es cero.\n");
    }
    }
```

Figura 69: código del ejercicio 406, variables, proceso y salida

12.6. Salida del programa

```
Problems  Output  Debug Console  Terminal  Ports

Microsoft Windows [Versión 10.0.26300.9711]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>gcc ejer406.c -o ejer406

C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>ejer406
Ingrese el peso en onzas: 5
5.00 onzas equivalen a 141.75 gramos
El inverso es: 0.007055
```

Figura 70: comprobación de la prueba 1

```
C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>ejer406
Ingrese el peso en onzas: 0
0.00 onzas equivalen a 0.00 gramos
No se puede calcular el inverso porque gramos es cero.
```

Figura 71: comprobación de la prueba 2

13. Enunciado del algoritmo

13) EJ-001

En un taller mecánico, declare tres variables enteras llamadas x, u y val2. Solicite por teclado tres números enteros (lectura) y almacénelos en esas variables. Calcule la suma, el producto y el promedio (expresión aritmética) usando una constante literal 3 como divisor. Asigne los resultados a variables suma, producto y promedio. Muestre por pantalla (escritura) un mensaje con los tres resultados. Justifique porque promedio debe ser tipo real mientras que suma y producto son enteros. Además del cálculo principal, determine si el resultado es positivo o negativo. Agregue un mensaje indicando el tipo de operación realizada.

13.1. Análisis del problema

Datos de entrada:

- Divisor (3.0)
- x
- u
- val2

Proceso

- suma = x + u + val2
- producto = x * u * val2
- promedio = suma / 3.0

Datos de salida:

- Suma
- Producto
- Promedio
- La suma es positiva o negativa

13.2. Diagrama de flujo

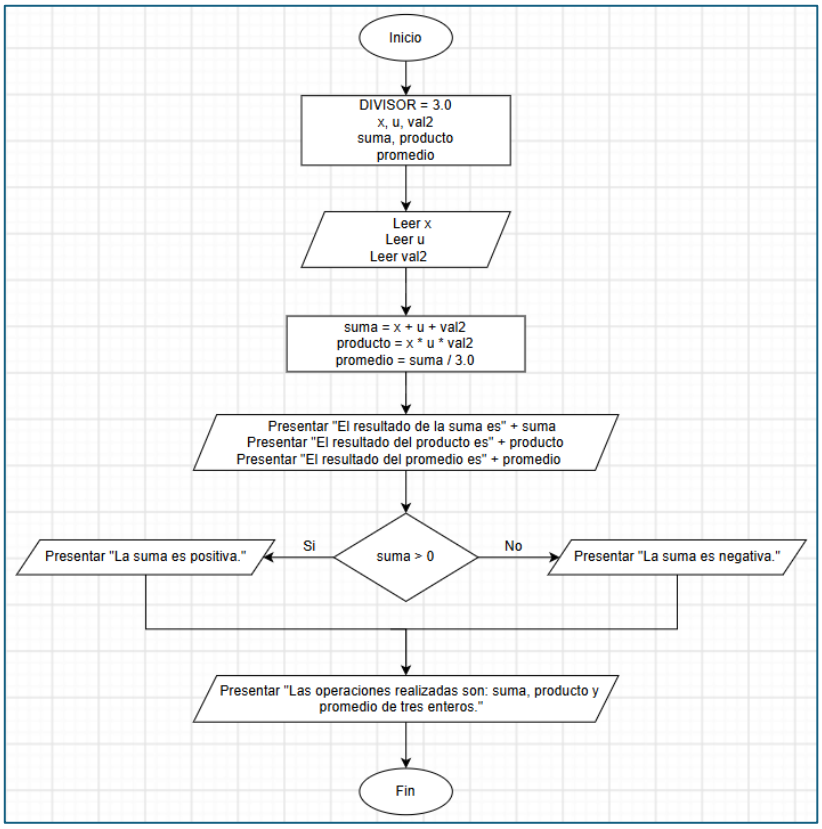


Figura 72: diagrama de flujo del ejercicio 001 en Draw.io

13.3. Pseudocódigo

```
Inicio

DIVISOR = 3.0;
x, u, val2;
suma, producto;
promedio;

Leer x;
Leer u;
Leer val2;

suma = x + u + val2;
producto = x * u * val2;
promedio = suma / 3.0;

Presentar "El resultado de la suma es" + suma;
Presentar "El resultado del producto es" + producto;
Presentar "El resultado del promedio es" + promedio;

Si (suma > 0) entonces
Presentar "La suma es positiva.";
Sino
Presentar "La suma es negativa.";
Fin Si

Presentar "Las operaciones realizadas son: suma, producto y promedio de tres enteros.";

Fin
```

Figura 73: pseudocódigo del ejercicio 001

13.4. Prueba de escritorio

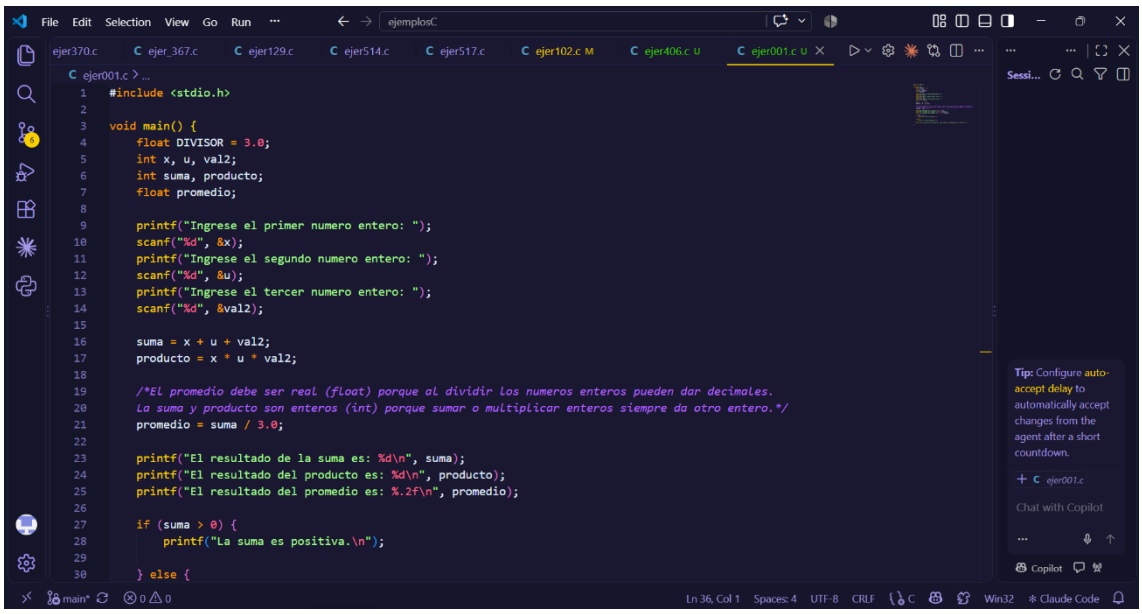
Prueba 1: La suma es positiva

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	x u val2		Ingrese el primer numero entero: 5 Ingrese el segundo numero entero: 6 Ingrese el tercer numero entero: 8
2	suma	x + u + val2 5 + 6 + 8 = 19	
3	producto	x * u * val2 5 * 6 * 8 = 240	
4	promedio	suma / 3.0 19 / 3.0 = 6.33	
5	Presentar		El resultado de la suma es: 19 El resultado del producto es: 240 El resultado del promedio es: 6.33
6		(suma > 0) (19 > 0) = TRUE	
7	Presentar		La suma es positiva Las operaciones realizadas son: suma, producto y promedio de tres enteros.

Prueba 2: La suma es negativa

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	x u val2		Ingrese el primer numero entero: -5 Ingrese el segundo numero entero: 2 Ingrese el tercer numero entero: -6
2	suma	x + u + val2 -5 + 2 + (-6) = -9	
3	producto	x * u * val2 -5 * 2 * (-6) = 60	
4	promedio	suma / 3.0 -9 / 3.0 = -3.00	
5	Presentar		El resultado de la suma es: -9 El resultado del producto es: 60 El resultado del promedio es: -3.00
6		(suma > 0) (-9 > 0) = FALSE	
7	Presentar		La suma es negativa Las operaciones realizadas son: suma, producto y promedio de tres enteros.

13.5. Codificación en lenguaje C



```c
#include <stdio.h>

void main() {
    float DIVISOR = 3.0;
    int x, u, val2;
    int suma, producto;
    float promedio;

    printf("Ingrese el primer numero entero: ");
    scanf("%d", &x);
    printf("Ingrese el segundo numero entero: ");
    scanf("%d", &u);
    printf("Ingrese el tercer numero entero: ");
    scanf("%d", &val2);

    suma = x + u + val2;
    producto = x * u * val2;

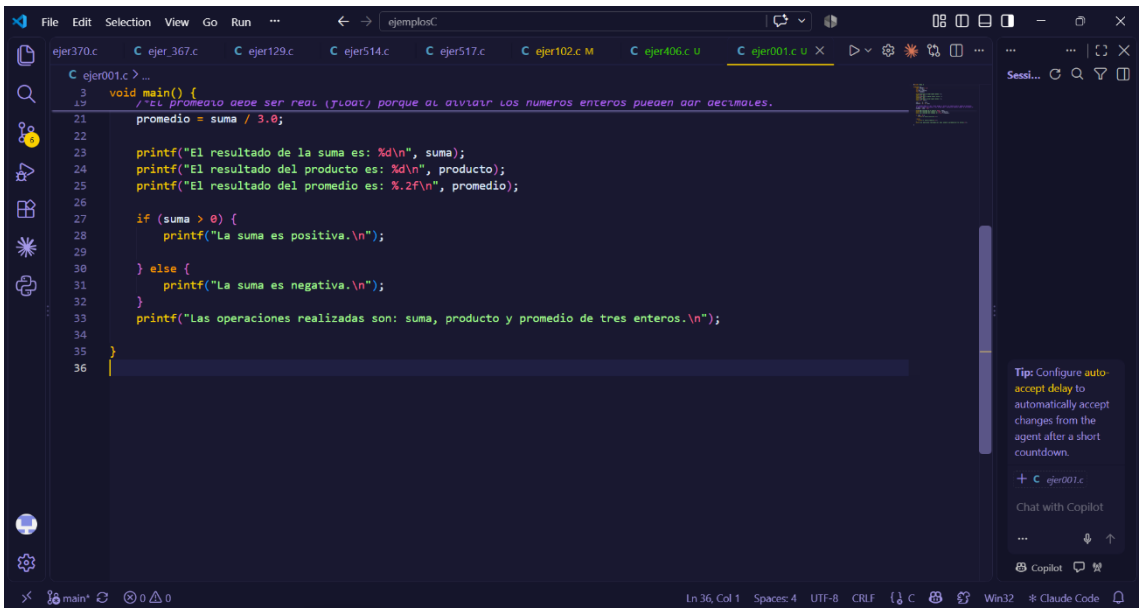
    /*El promedio debe ser real (float) porque al dividir los numeros enteros pueden dar decimales.
    La suma y producto son enteros (int) porque sumar o multiplicar enteros siempre da otro entero.*/
    promedio = suma / 3.0;

    printf("El resultado de la suma es: %d\n", suma);
    printf("El resultado del producto es: %d\n", producto);
    printf("El resultado del promedio es: %.2f\n", promedio);

    if (suma > 0) {
        printf("La suma es positiva.\n");

    } else {
```

Figura 74: código del ejercicio 001, variables y proceso



```c
void main() {
    /*El promedio debe ser real (float) porque al dividir los numeros enteros pueden dar decimales.
    promedio = suma / 3.0;

    printf("El resultado de la suma es: %d\n", suma);
    printf("El resultado del producto es: %d\n", producto);
    printf("El resultado del promedio es: %.2f\n", promedio);

    if (suma > 0) {
        printf("La suma es positiva.\n");

    } else {
        printf("La suma es negativa.\n");
    }
    printf("Las operaciones realizadas son: suma, producto y promedio de tres enteros.\n");

}
```

Figura 75: código del ejercicio 001, salida

13.6. Salida del programa

```
Problems  Output  Debug Console  Terminal  Ports

Microsoft Windows [Versión 10.0.26300.9711]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>gcc ejer001.c -o ejer001

C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>ejer001
Ingrese el primer numero entero: 5
Ingrese el segundo numero entero: 6
Ingrese el tercer numero entero: 8
El resultado de la suma es: 19
El resultado del producto es: 240
El resultado del promedio es: 6.33
La suma es positiva.
Las operaciones realizadas son: suma, producto y promedio de tres enteros.
```

Figura 76: comprobación de la prueba 1

```
C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>ejer001
Ingrese el primer numero entero: -5
Ingrese el segundo numero entero: 2
Ingrese el tercer numero entero: -6
El resultado de la suma es: -9
El resultado del producto es: 60
El resultado del promedio es: -3.00
La suma es negativa.
Las operaciones realizadas son: suma, producto y promedio de tres enteros.
```

Figura 77: comprobación de la prueba 2

14. Enunciado del algoritmo

14) EJ-217

En un taller mecánico, diseñe un algoritmo que lea una medida en centímetros (real). Use una constante simbólica FACTOR = 0.393701. Calcule la equivalencia en pulgadas con la expresión aritmética (cm * FACTOR) y asígnela a pulgadas. Evalúe la expresión relacional (pulgadas > 0). Muestre por pantalla 'X cm equivalen a Y pulgadas'. Además del cálculo principal, calcule el resultado elevado al cubo. Asegúrese de que las expresiones lógicas estén bien parentizadas.

14.1. Análisis del problema

Datos de entrada:

- Medida en centímetros
- Factor

Proceso

- pulgadas = (cm * FACTOR)
- condicion = (pulgadas > 0)
- cubo = (pulgadas * pulgadas * pulgadas)

Datos de salida:

- X cm equivalen a Y pulgadas
- Resultado elevado al cubo
- Evaluar la condición

14.2. Diagrama de flujo

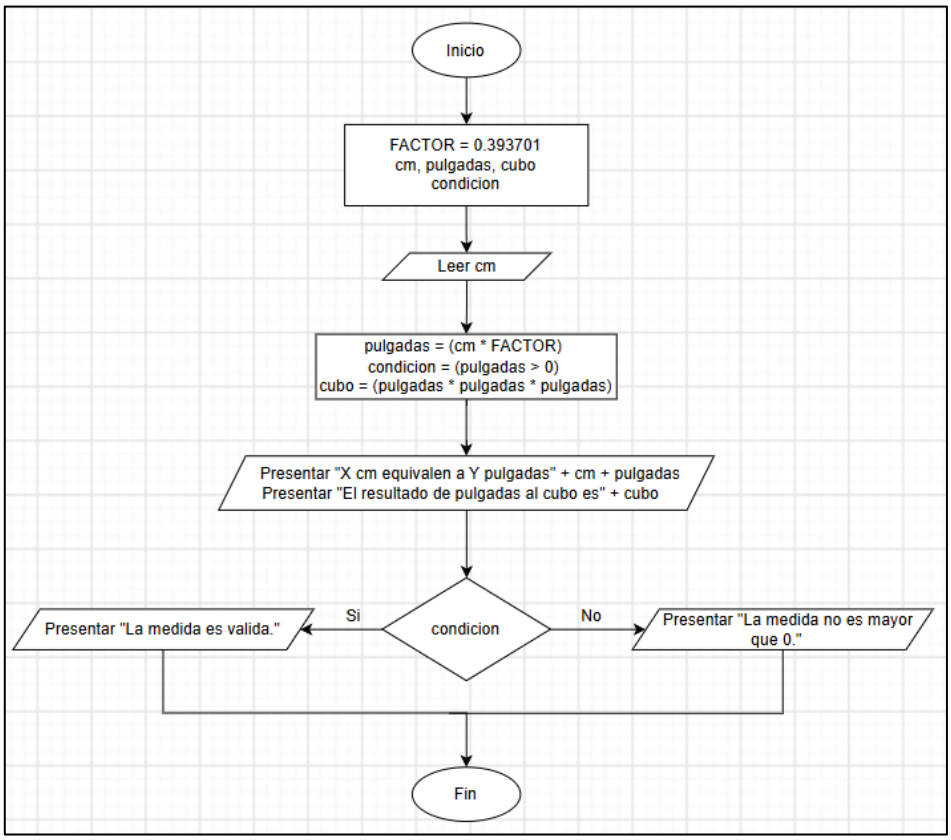


Figura 78: diagrama de flujo del ejercicio 217 en Draw.io

14.3. Pseudocódigo

```
Inicio

FACTOR = 0.393701;
cm, pulgadas, cubo;
condición;

Leer cm;

pulgadas = (cm * FACTOR);
condicion = (pulgadas > 0);
cubo = (pulgadas * pulgadas * pulgadas);

Presentar "X cm equivalen a Y pulgadas" + cm + pulgadas;
Presentar "El resultado de pulgadas al cubo es" + cubo;

Si (condicion) entonces
Presentar "La medida es valida.";
Sino
Presentar "La medida no es mayor que 0.";
Fin Si

Fin
```

Figura 79: pseudocódigo del ejercicio 217

14.4. Prueba de escritorio

Prueba 1: La medida es válida

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	cm		Ingrese la medida en centimetros: 10
2	pulgadas	(cm * FACTOR) (10 * 0.393701) = 3.94	
3	condicion	(pulgadas > 0) (3.93 > 0) = TRUE	
4	cubo	(pulgadas * pulgadas * pulgadas) (3.94 * 3.94 * 3.94) = 61.02	
5	Presentar		10.00 cm equivalen a 3.94 pulgadas El resultado de pulgadas al cubo es: 61.02 La medida es valida.

Prueba 2: La medida no es mayor que cero

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	cm		Ingrese la medida en centimetros: -6
2	pulgadas	(cm * FACTOR) (-6 * 0.393701) = -2.36	
3	condicion	(pulgadas > 0) (-2.36 > 0) = FALSE	
4	cubo	(pulgadas * pulgadas * pulgadas) (-2.36 * (-2.36) * (-2.36)) = -13.18	
5	Presentar		-6.00 cm equivalen a -2.36 pulgadas El resultado de pulgadas al cubo es: -13.18 La medida no es mayor que 0

14.5. Codificación en lenguaje C

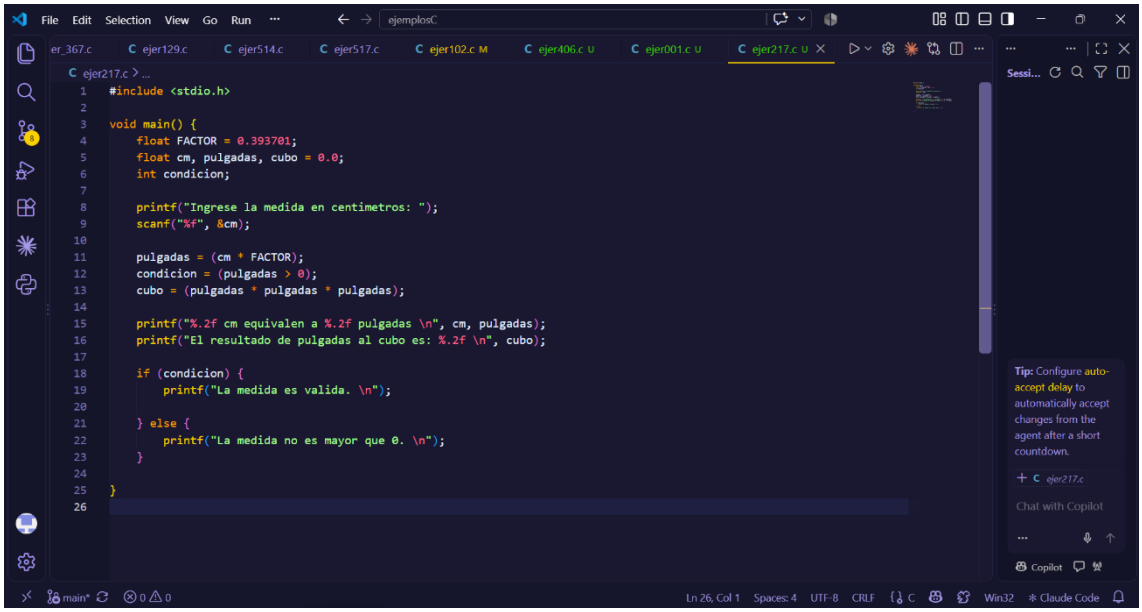


Figura 80: código del ejercicio 217, variables, proceso y salida

14.6. Salida del programa

```
Problems  Output  Debug Console  Terminal  Ports

Microsoft Windows [Versión 10.0.26300.9711]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>gcc ejer217.c -o ejer217

C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>ejer217
Ingrese la medida en centimetros: 10
10.00 cm equivalen a 3.94 pulgadas
El resultado de pulgadas al cubo es: 61.02
La medida es valida.
```

Figura 81: comprobación de la prueba 1

```
C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>ejer217
Ingrese la medida en centimetros: -6
-6.00 cm equivalen a -2.36 pulgadas
El resultado de pulgadas al cubo es: -13.18
La medida no es mayor que 0.
```

Figura 82: comprobación de la prueba 2

15. Enunciado del algoritmo

15) EJ-238

En un hospital, diseñe un algoritmo que lea una distancia en yardas (real). Use la constante literal 3. Calcule la equivalencia en pies con la expresión aritmética (yardas * 3) y asígnela a pies. Evalúe la expresión relacional (pies > 0). Muestre por pantalla 'X yardas equivalen a Y pies'. Además del cálculo principal, determine si el resultado es positivo o negativo. Asegúrese de inicializar todas las variables antes de usarlas.

15.1. Análisis del problema

Datos de entrada:

- Distancia en yardas
- Multiplicador para la expresión aritmética (3)

Proceso

- pies = (yardas * 3);
- condicion = (pies > 0)

Datos de salida:

- X yardas equivalen a Y pies
- El resultado es positivo o negativo

15.2. Diagrama de flujo



Figura 83: diagrama de flujo del ejercicio 238 en Draw.io

15.3. Pseudocódigo

```
Inicio

MULTI = 3;
yardas, pies;
condición;

Leer yardas;

pies = (yardas * 3);
condicion = (pies > 0);

Presentar "X yardas equivalen a Y pies" + yardas + pies;

Si (condicion) entonces
Presentar "El resultado es positivo.";
Sino
Presentar "El resultado es negativo.";
Fin Si

Fin
```

Figura 84: pseudocódigo del ejercicio 238

15.4. Prueba de escritorio

Prueba 1: El resultado es positivo

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	yardas		Ingrese la distancia en yardas: 12.6
2	pies	(yardas * 3) (12.6 * 3) = 37.80	
3	condicion	(pies > 0) (37.80 > 0) = TRUE	
4	Presentar		12.60 yardas equivalen a 37.80 pies El resultado es positivo.

Prueba 2: El resultado es negativo

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	yardas		Ingrese la distancia en yardas: -8
2	pies	(yardas * 3) (-8 * 3) = -24	
3	condicion	(pies > 0) (-24 > 0) = FALSE	
4	Presentar		-8.00 yardas equivalen a -24.00 pies El resultado es negativo.

15.5. Codificación en lenguaje C

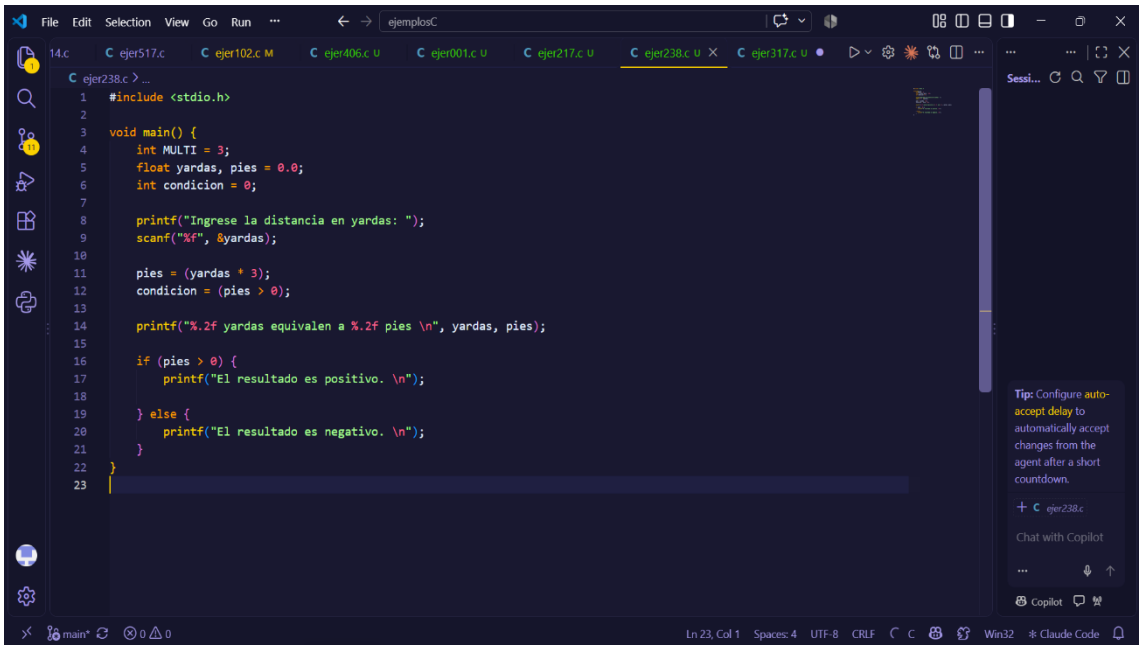


Figura 85: código del ejercicio 238, variables, proceso y salida

15.6. Salida del programa



Figura 86: comprobación de la prueba 1



Figura 87: comprobación de la prueba 2

16. Enunciado del algoritmo

16) EJ-317

En una universidad, construya un algoritmo que lea un monto en euros (real). Use una constante simbólica TASA_CAMBIO = 1.08. Calcule el equivalente en dólares con la expresión aritmética (euros * TASA_CAMBIO) y asígnelo a dólares. Evalúe la expresión relacional (euros > 0). Muestre por pantalla 'X euros equivalen a Y dólares'. Adicionalmente, calcule la raíz cuadrada del valor absoluto del resultado. Agregue una variable de estado con el código de éxito (0) al final.

16.1. Análisis del problema

Datos de entrada:

- Tasa de cambio
- Monto en euros

Proceso

- dolares = (euros * TASA_CAMBIO)
- condicion = (euros > 0)

Datos de salida:

- X euros equivalen a Y dólares
- La raíz cuadrada del valor absoluto del resultado
- El estado con el código de éxito (0)

16.2. Diagrama de flujo

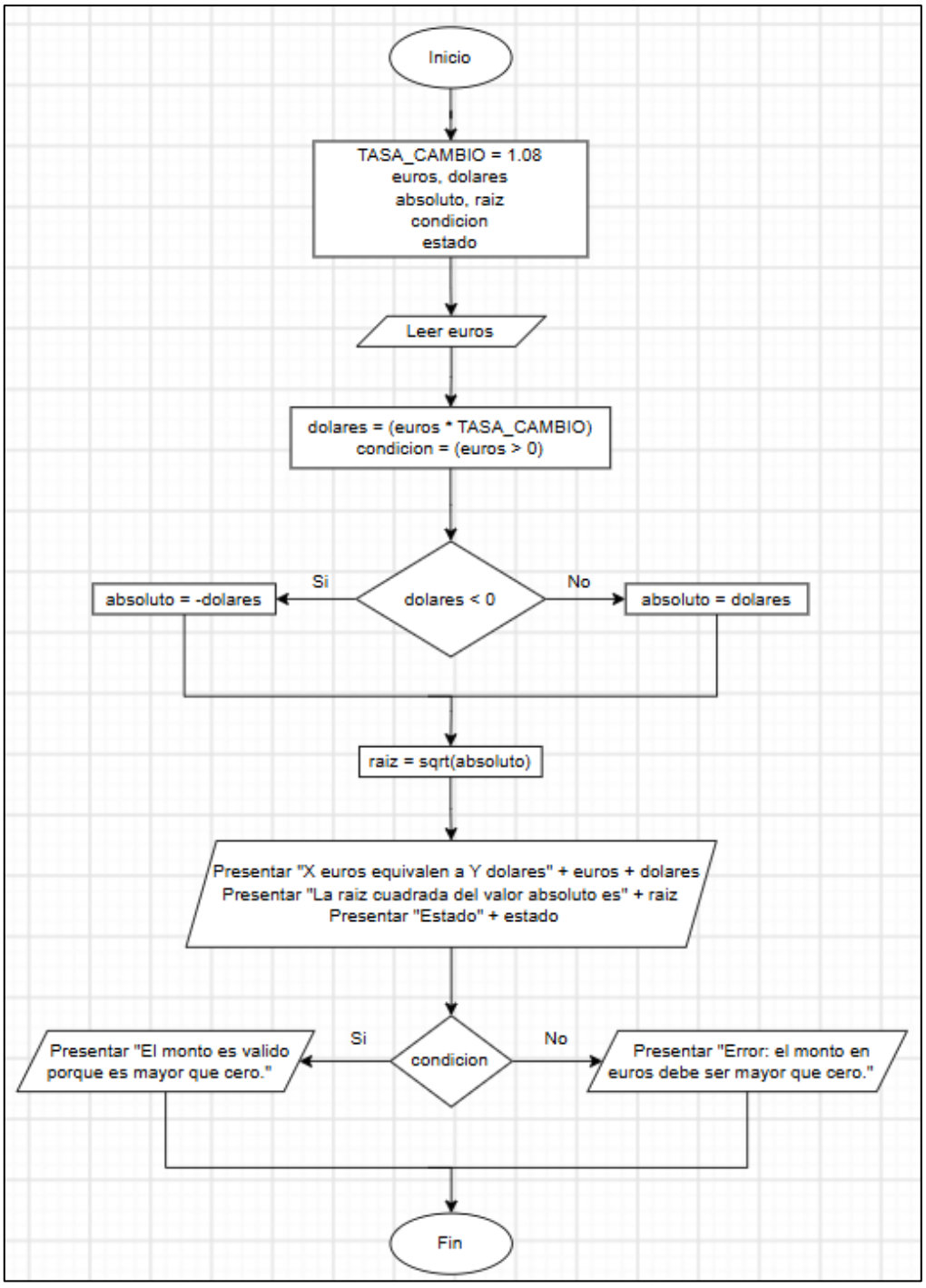


Figura 88: diagrama de flujo del ejercicio 317 en Draw.io

16.3. Pseudocódigo

```
Inicio

TASA_CAMBIO = 1.08;
euros, dolares;
absoluto, raiz;
condicion;
estado;

Leer euros;

dolares = (euros * TASA_CAMBIO);
condicion = (euros > 0);

Si (dolares < 0) entonces
absoluto = -dolares;
Sino
absoluto = dolares;
Fin Si

raiz = sqrt(absoluto);

Presentar "X euros equivalen a Y dolares" + euros + dolares;
Presentar "La raiz cuadrada del valor absoluto es" + raiz;
Presentar "Estado" + estado;

Si (condicion) entonces
Presentar "El monto es valido porque es mayor que cero.";
Sino
Presentar "Error: el monto en euros debe ser mayor que cero.";
Fin Si

Fin
```

Figura 89: pseudocódigo del ejercicio 317

16.4. Prueba de escritorio

Prueba 1: El monto es válido

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	euros		Ingrese el monto en euros: 100
2	dolares	(euros * TASA_CAMBIO) (100 * 1.08) = 108	
3	condicion	(euros > 0) (100 > 0) = TRUE	
4		(dolares < 0) (108 < 0) = FALSE absoluto = dolares	

5	raiz	sqrt(absoluto) sqrt(108) = 10.39	
6	Presentar		100.00 euros equivalen a 108.00 dolares La raiz cuadrada del valor absoluto es: 10.39 Estado: 0 El monto es valido porque es mayor que cero.

Prueba 2: El monto es menor que cero

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	euros		Ingrese el monto en euros: -45
2	dolares	(euros * TASA_CAMBIO) (-45 * 1.08) = -48.6	
3	condicion	(euros > 0) (-48.6 > 0) = FALSE	
4		(dolares < 0) (-48.6 < 0) = TRUE absoluto = -dolares	
5	raiz	sqrt(absoluto) sqrt(48.6) = 6.97	
6	Presentar		-45.00 euros equivalen a -48.60 dolares La raiz cuadrada del valor absoluto es: 6.97 Estado: 0 Error: el monto en euros debe ser mayor que cero.

16.5. Codificación en lenguaje C

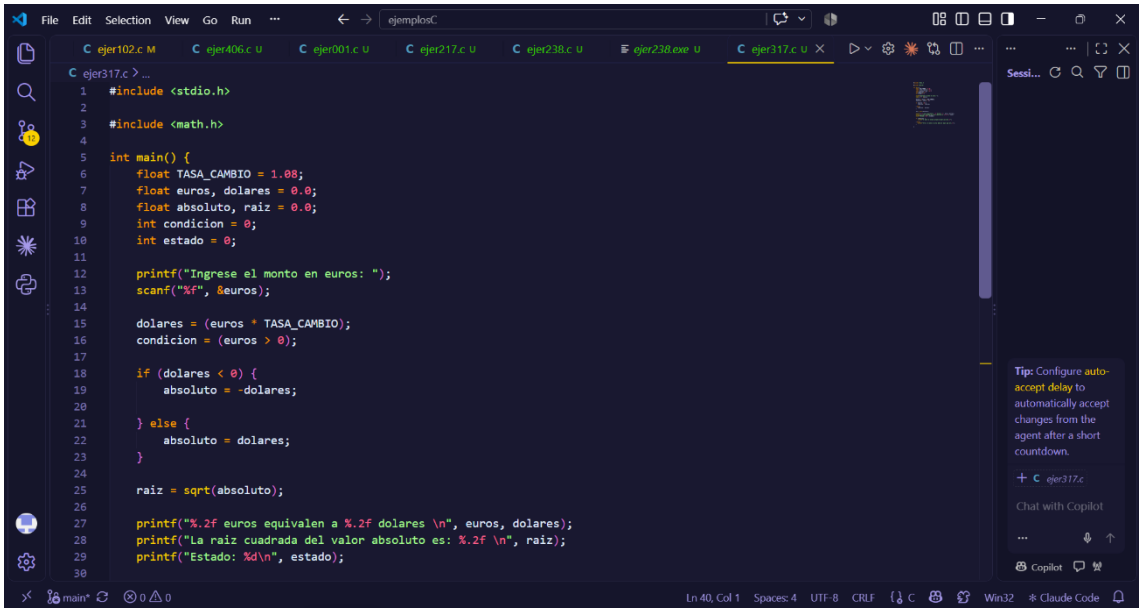


Figura 90: código del ejercicio 317, variables y proceso

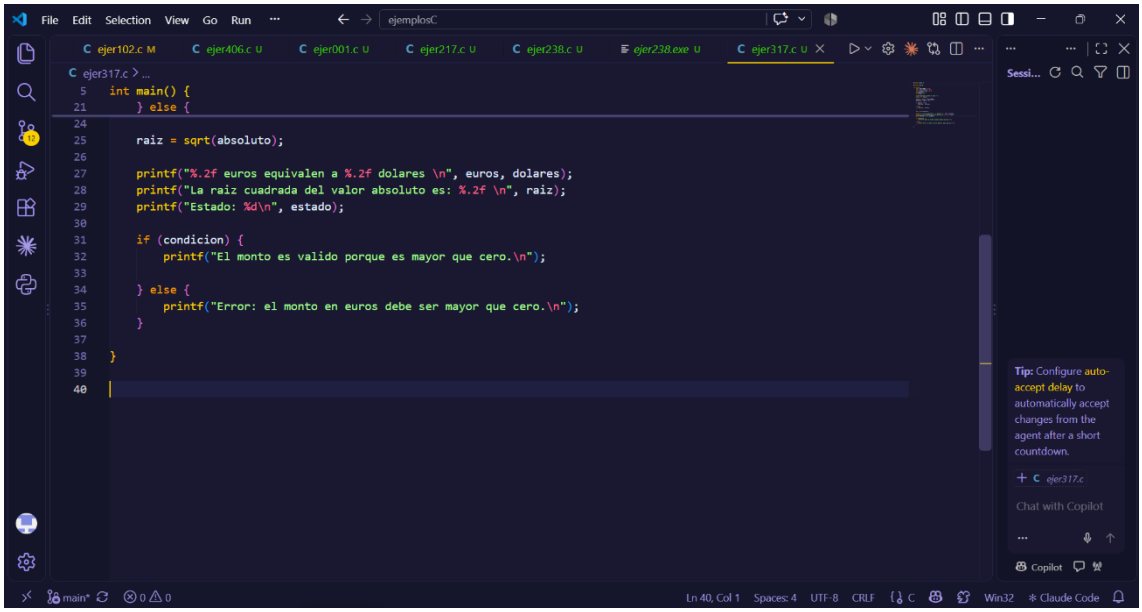


Figura 91: código del ejercicio 317, salida

16.6. Salida del programa

```
Problems  Output  Debug Console  Terminal  Ports

Microsoft Windows [Versión 10.0.26300.9711]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>gcc ejer317.c -o ejer317

C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>ejer317
Ingrese el monto en euros: 100
100.00 euros equivalen a 108.00 dolares
La raiz cuadrada del valor absoluto es: 10.39
Estado: 0
El monto es valido porque es mayor que cero.
```

Figura 92: comprobación de la prueba 1

```
C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>ejer317
Ingrese el monto en euros: -45
-45.00 euros equivalen a -48.60 dolares
La raiz cuadrada del valor absoluto es: 6.97
Estado: 0
Error: el monto en euros debe ser mayor que cero.
```

Figura 93: comprobación de la prueba 2

17. Enunciado del algoritmo

17) EJ-037

En un hospital, cree un algoritmo que lea un numero entero y por teclado. Use constantes literales 2 y 3 como factores. Calcule el doble con (2 * y) y el triple con (3 * y). Asigne los resultados a variables doble y triple. Evalúe la expresión lógica (doble < triple AND doble > 0). Muestre por pantalla ambos resultados. Además, calcule el inverso multiplicativo del resultado si no es cero. Muestre al final los valores de todas las variables auxiliares.

17.1. Análisis del problema

Datos de entrada:

- Número
- Factores (2 y 3)

Proceso

- doble = (2 * y)
- triple = (3 * y)
- condicion = (doble < triple && doble > 0)

Datos de salida:

- Valores de todas las variables auxiliares
- El inverso multiplicativo del resultado si no es cero

17.2. Diagrama de flujo

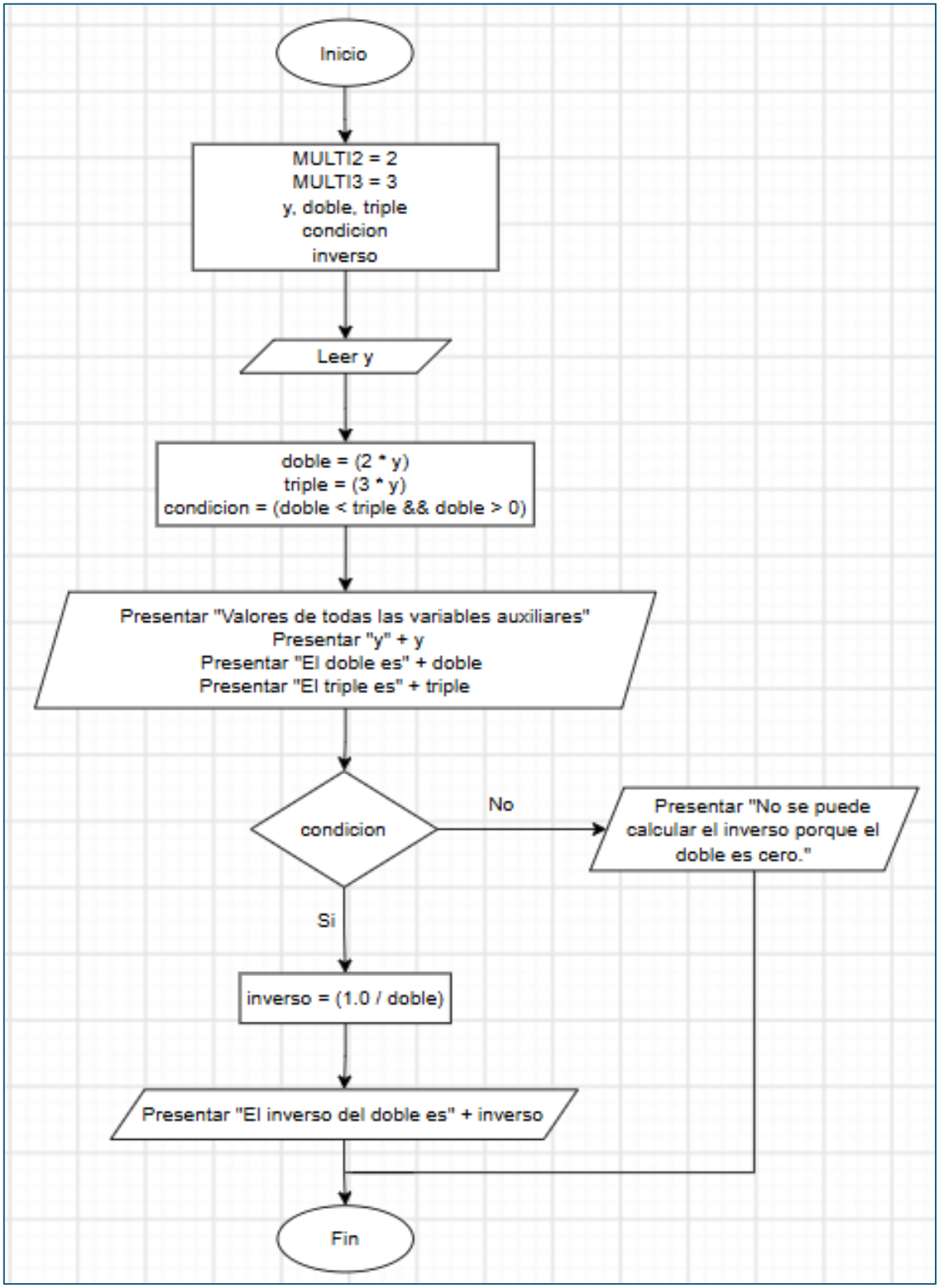


Figura 94: diagrama de flujo del ejercicio 037 en Draw.io

17.3. Pseudocódigo

```
Inicio

MULTI2 = 2;
MULTI3 = 3;
y, doble, triple;
condicion;
inverso;

Leer y;

doble = (2 * y);
triple = (3 * y);
condicion = (doble < triple && doble > 0);

Presentar "Valores de todas las variables auxiliares";
Presentar "y" + y;
Presentar "El doble es" + doble;
Presentar "El triple es" + triple;

Si (condicion) entonces
inverso = (1.0 / doble);
Presentar "El inverso del doble es" + inverso;
Sino
Presentar "No se puede calcular el inverso porque el doble es cero.";
Fin Si

Fin
```

Figura 95: pseudocódigo del ejercicio 037

17.4. Prueba de escritorio

Prueba 1: Se puede calcular el inverso

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	y		Ingrese un numero entero: 4
2	doble	(2 * y) (2 * 4) = 8	
3	triple	(3 * y) (3 * 4) = 12	
4	condicion	(doble < triple && doble > 0) (8 < 12 && 8 > 0) = TRUE	
5	Presentar		Valores de todas las variables auxiliares: y: 4 El doble es: 8 El triple es: 12
6	inverso	(1.0 / doble) (1.0 / 8) = 0.125	
7	Presentar		El inverso del doble es: 0.125000

Prueba 2: No se puede calcular el inverso

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	y		Ingrese un numero entero: 0
2	doble	(2 * y) (2 * 0) = 0	
3	triple	(3 * y) (3 * 0) = 0	
4	condicion	(doble < triple && doble > 0) (0 < 0 && 0 > 0) = FALSE	
5	Presentar		Valores de todas las variables auxiliares: y: 0 El doble es: 0 El triple es: 0 No se puede calcular el inverso porque el doble es cero.

17.5. Codificación en lenguaje C

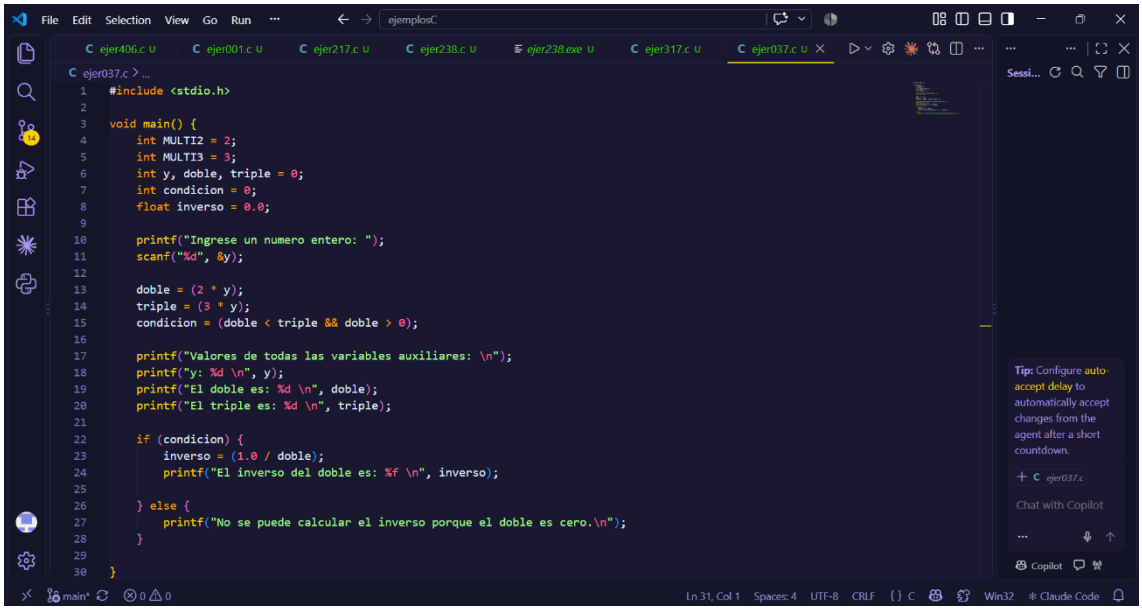


Figura 96: código del ejercicio 037, variables, proceso y salida

17.6. Salida del programa



Figura 97: comprobación de la prueba 1

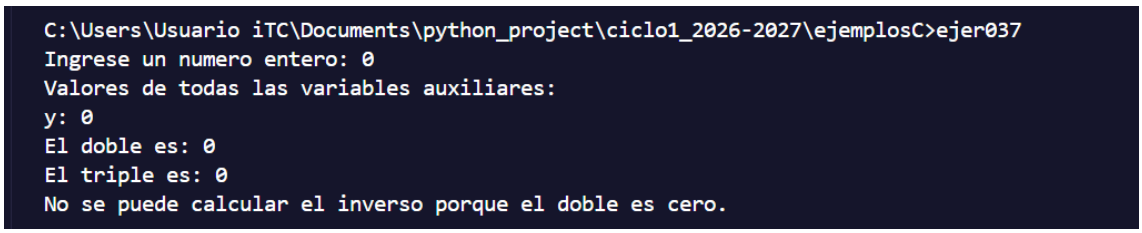


Figura 98: comprobación de la prueba 2

18. Enunciado del algoritmo

18) EJ-113

En una gasolinera, diseñe un algoritmo que lea las estaturas (reales, en metros) de cinco personas. Use la constante literal 5 como divisor. Calcule la estatura promedio con la expresión aritmética ((e1 + e2 + e3 + e4 + e5) / 5) y asígnela a promedio. Evalúe la expresión lógica (promedio > 0). Muestre por pantalla la estatura promedio. Además del cálculo principal, determine el valor máximo de las variables utilizadas. Justifique el tipo de dato simple elegido para cada variable.

18.1. Análisis del problema

Datos de entrada:

- Estaturas de las cinco personas
- Divisor (5)

Proceso

- promedio = ((e1 + e2 + e3 + e4 + e5) / 5)
- condicion = (promedio > 0)

Datos de salida:

- Estatura promedio
- El valor máximo de las variables utilizadas
- Error si no es mayor que cero

18.2. Diagrama de flujo

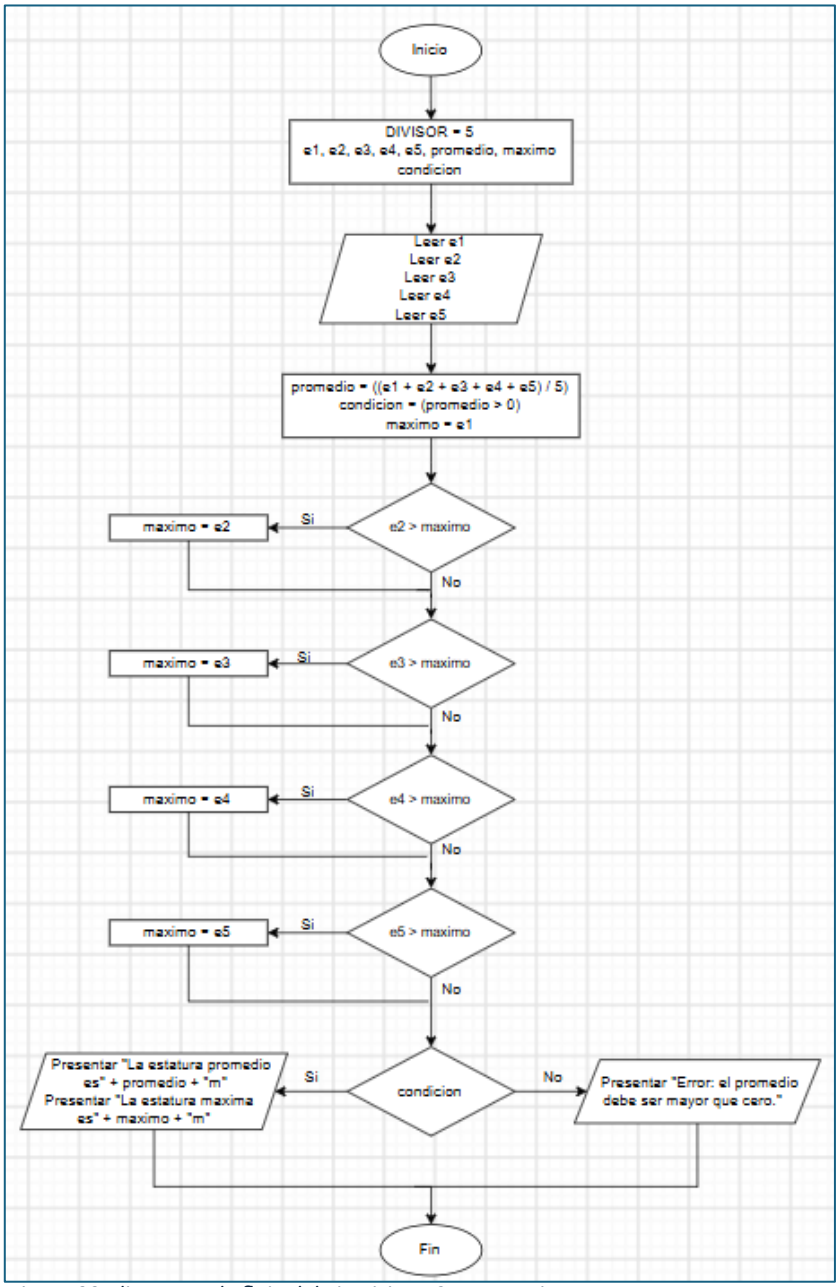


Figura 99: diagrama de flujo del ejercicio 113 en Draw.io

18.3. Pseudocódigo

```
Inicio

DIVISOR = 5;
e1, e2, e3, e4, e5, promedio, maximo;
condicion;

Leer e1;
Leer e2;
Leer e3;
Leer e4;
Leer e5;

promedio = ((e1 + e2 + e3 + e4 + e5) / 5);
condicion = (promedio > 0);
maximo = e1;

Si (e2 > maximo) entonces
maximo = e2;
Fin Si
Si (e3 > maximo) entonces
maximo = e3;
Fin Si
Si (e4 > maximo) entonces
maximo = e4;
Fin Si
Si (e5 > maximo) entonces
maximo = e5;
Fin Si

Si (condicion) entonces
Presentar "La estatura promedio es" + promedio + "m";
Presentar "La estatura maxima es" + maximo + "m";
Sino
Presentar "Error: el promedio debe ser mayor que cero.";
Fin Si

Fin
```

Figura 100: pseudocódigo del ejercicio 113

18.4. Prueba de escritorio

Prueba 1: El promedio es mayor que cero

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	e1 e2 e3 e4 e5		Ingrese la estatura de la persona 1 (m): 1.70 Ingrese la estatura de la persona 2 (m): 1.65 Ingrese la estatura de la persona 3 (m): 1.80 Ingrese la estatura de la persona 4 (m): 1.75 Ingrese la estatura de la persona 5 (m): 1.60
2	promedio	((e1 + e2 + e3 + e4 + e5) / 5) ((1.70 + 1.65 + 1.80 + 1.75 + 1.60) / 5) = 1.70	
3	condicion	(promedio > 0) (1.70 > 0) = TRUE	
4		maximo = e1 (e2 > maximo) 1.65 > 1.70 = FALSE	

5		(e3 > maximo) 1.80 > 1.70 = TRUE	
6		(e4 > maximo) 1.75 > 1.80 = FALSE	
7		(e5 > maximo) 1.60 > 1.80 = FALSE	
8	Presentar		La estatura promedio es: 1.70 m La estatura maxima es: 1.80 m

Prueba 2: El promedio no es mayor que cero

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	e1 e2 e3 e4 e5		Ingrese la estatura de la persona 1 (m): 0.00 Ingrese la estatura de la persona 2 (m): 0.00 Ingrese la estatura de la persona 3 (m): 0.00 Ingrese la estatura de la persona 4 (m): 0.00 Ingrese la estatura de la persona 5 (m): 0.00
2	promedio	((e1 + e2 + e3 + e4 + e5) / 5) ((0.00 + 0.00 + 0.00 + 0.00 + 0.00) / 5) = 0.00	
3	condicion	(promedio > 0) (0.00 > 0) = FALSE	
4	Presentar		Error: el promedio debe ser mayor que cero.

18.5. Codificación en lenguaje C

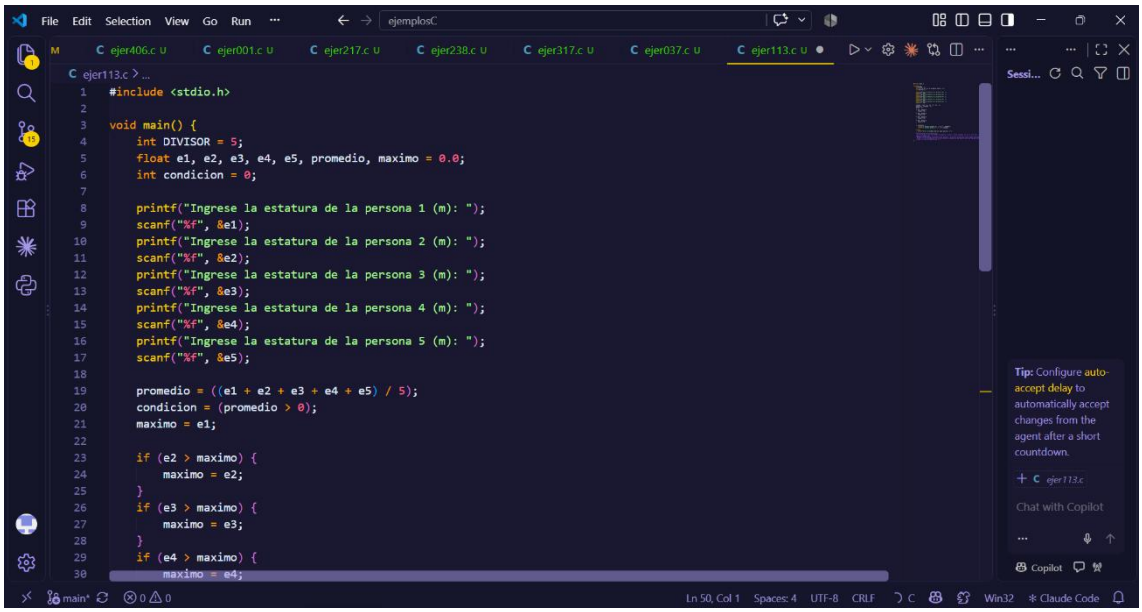


Figura 101: código del ejercicio 113, variables y proceso

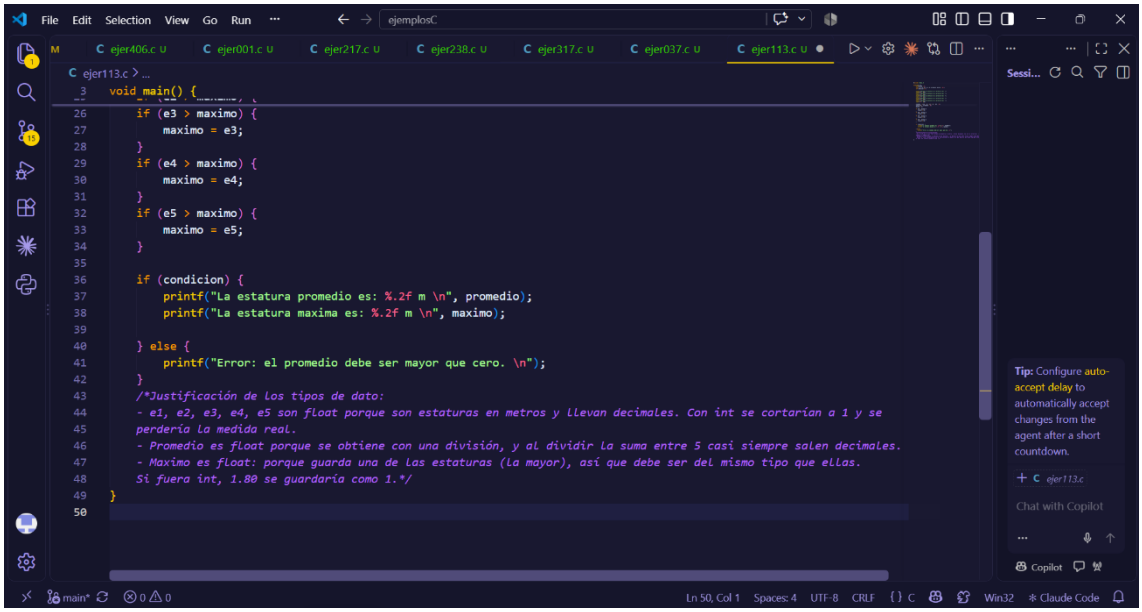


Figura 102: código del ejercicio 113, salida y justificación del tipo de dato

18.6. Salida del programa



Figura 103: comprobación de la prueba 1

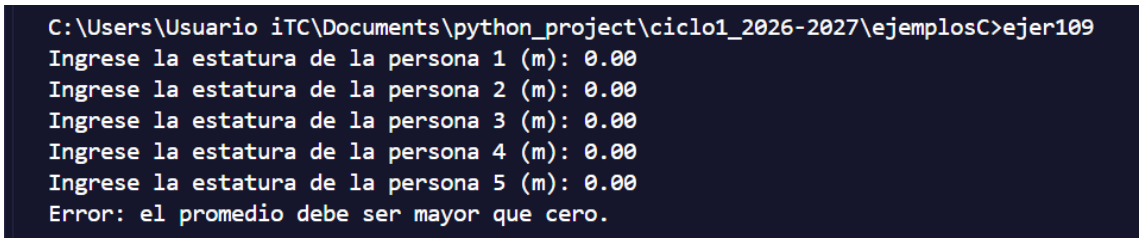


Figura 104: comprobación de la prueba 2

19. Enunciado del algoritmo

19) EJ-447

En una tienda de ropa, cree un algoritmo que lea un número real a por teclado. Use la constante literal 2 como exponente. Calcule la potencia con la expresión aritmética (a ^ 2) y asígnela a cuadrado. Evalúe la expresión relacional (cuadrado >= 0). Muestre por pantalla 'El cuadrado de X es Y'. Además del cálculo principal, calcule el resto de una división adicional. Incluya un mensaje motivacional al final del algoritmo.

19.1. Análisis del problema

Datos de entrada:

- Exponente (2)
- Número nombrado a

Proceso

- cuadrado = pow(a, 2)
- condicion = (cuadrado >= 0)
- parte_entera = (int)cuadrado
- resto = (parte_entera % 2)

Datos de salida:

- El cuadrado de X es Y
- El resto de una división adicional
- Mensaje motivacional

19.2. Diagrama de flujo

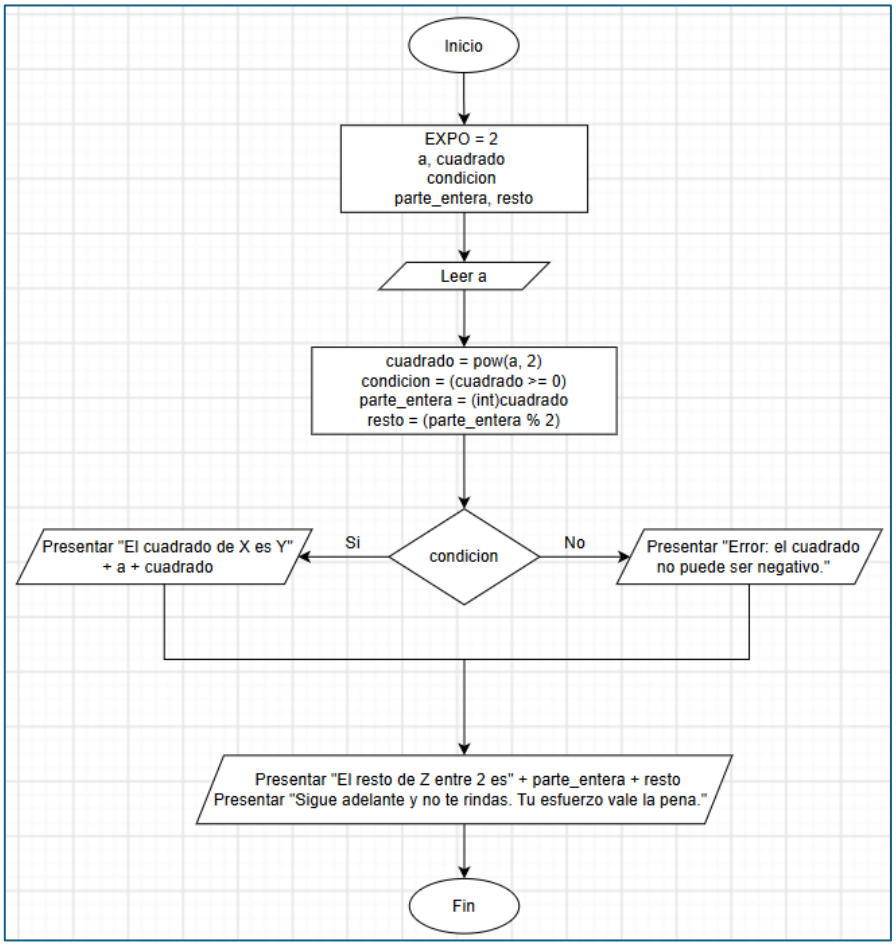


Figura 105: diagrama de flujo del ejercicio 447 en Draw.io

19.3. Pseudocódigo

```
Inicio

EXPO = 2;
a, cuadrado;
condicion;
parte_entera, resto;

Leer a;

cuadrado = pow(a, 2);
condicion = (cuadrado >= 0);
parte_entera = (int)cuadrado;
resto = (parte_entera % 2);

Si (condicion) entonces
Presentar "El cuadrado de X es Y" + a + cuadrado;
Sino
Presentar "Error: el cuadrado no puede ser negativo.";
Fin Si

Presentar "El resto de Z entre 2 es" + parte_entera + resto;
Presentar "Sigue adelante y no te rindas. Tu esfuerzo vale la pena.";

Fin
```

Figura 106: pseudocódigo del ejercicio 447

19.4. Prueba de escritorio

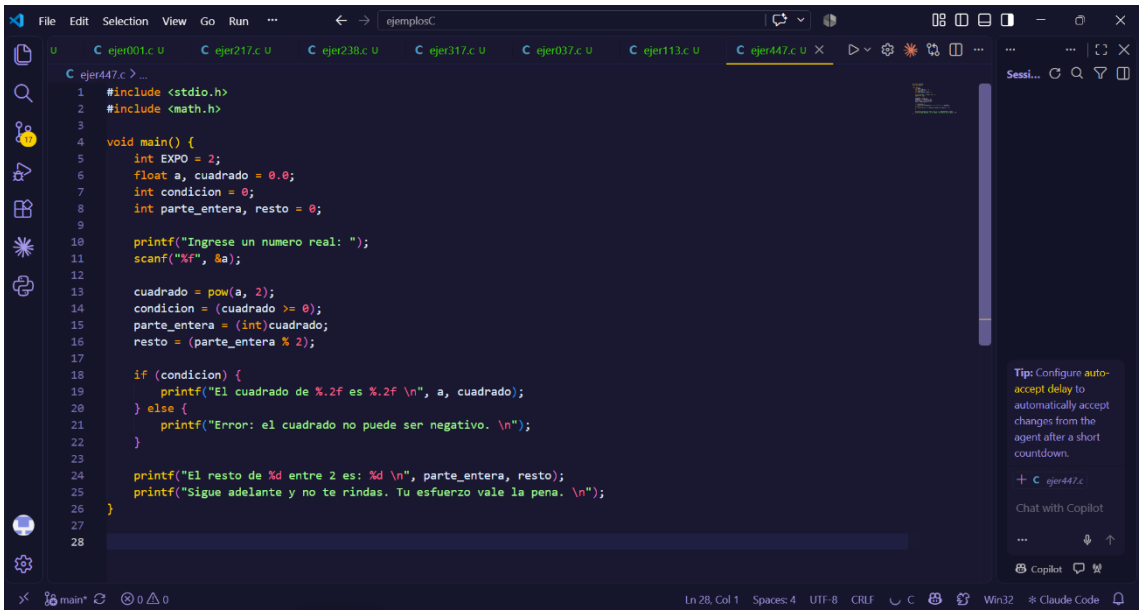
Prueba 1: Número positivo

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	a		Ingrese un numero real: 7
2	cuadrado	pow(a, 2) pw(7, 2) = 49	
3	condicion	(cuadrado >= 0) (49 >= 0) = TRUE	
4	parte_entera	(int)cuadrado (int)49 = 49	
5	resto	(parte_entera % 2) (49 % 2) = 1	
6	Presentar		El cuadrado de 7.00 es 49.00 El resto de 49 entre 2 es: 1 Sigue adelante y no te rindas. Tu esfuerzo vale la pena.

Prueba 2: Número negativo

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	a		Ingrese un numero real: -9.4
2	cuadrado	pow(a, 2) pw(-9, 2) = 88.36	
3	condicion	(cuadrado >= 0) (88.36 >= 0) = TRUE	
4	parte_entera	(int)cuadrado (int)88.36 = 88	
5	resto	(parte_entera % 2) (88 % 2) = 0	
6	Presentar		El cuadrado de -9.40 es 88.36 El resto de 88 entre 2 es: 0 Sigue adelante y no te rindas. Tu esfuerzo vale la pena.

19.5. Codificación en lenguaje C



```c
#include <stdio.h>
#include <math.h>

void main() {
    int EXPO = 2;
    float a, cuadrado = 0.0;
    int condicion = 0;
    int parte_entera, resto = 0;

    printf("Ingrese un numero real: ");
    scanf("%f", &a);

    cuadrado = pow(a, 2);
    condicion = (cuadrado >= 0);
    parte_entera = (int)cuadrado;
    resto = (parte_entera % 2);

    if (condicion) {
        printf("El cuadrado de %.2f es %.2f \n", a, cuadrado);
    } else {
        printf("Error: el cuadrado no puede ser negativo. \n");
    }

    printf("El resto de %d entre 2 es: %d \n", parte_entera, resto);
    printf("Sigue adelante y no te rindas. Tu esfuerzo vale la pena. \n");
}
```

Figura 107: código del ejercicio 447, variables, proceso y salida

19.6. Salida del programa

```
Problems  Output  Debug Console  Terminal  Ports

Microsoft Windows [Versión 10.0.26300.9711]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>gcc ejer447.c -o ejer447

C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>ejer447
Ingrese un numero real: 7
El cuadrado de 7.00 es 49.00
El resto de 49 entre 2 es: 1
Sigue adelante y no te rindas. Tu esfuerzo vale la pena.
```

Figura 108: comprobación de la prueba 1

```
C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>ejer447
Ingrese un numero real: -9.4
El cuadrado de -9.40 es 88.36
El resto de 88 entre 2 es: 0
Sigue adelante y no te rindas. Tu esfuerzo vale la pena.
```

Figura 109: comprobación de la prueba 2

20. Enunciado del algoritmo

20) EJ-501

En un banco, construya un algoritmo que lea el largo, ancho y alto (tres reales) de un prisma rectangular. Calcule el volumen con la expresión aritmética (largo * ancho * alto) y asígnelo a volumen. Evalúe la expresión lógica (largo > 0 AND ancho > 0 AND alto > 0). Muestre por pantalla el volumen del prisma. Adicionalmente, calcule la diferencia entre el resultado y el promedio. Verifique que el algoritmo funcione correctamente con valores limite.

20.1. Análisis del problema

Datos de entrada:

- Divisor para el promedio (3)
- Largo, ancho y alto de un prisma rectangular
- Número nombrado a

Proceso

- volumen = (largo * ancho * alto)
- condicion = (largo > 0 && ancho > 0 && alto > 0)
- promedio = ((largo + ancho + alto) / 3)
- diferencia = (volumen – promedio)

Datos de salida:

- El volumen del prisma
- El promedio
- La diferencia del resultado

20.2. Diagrama de flujo

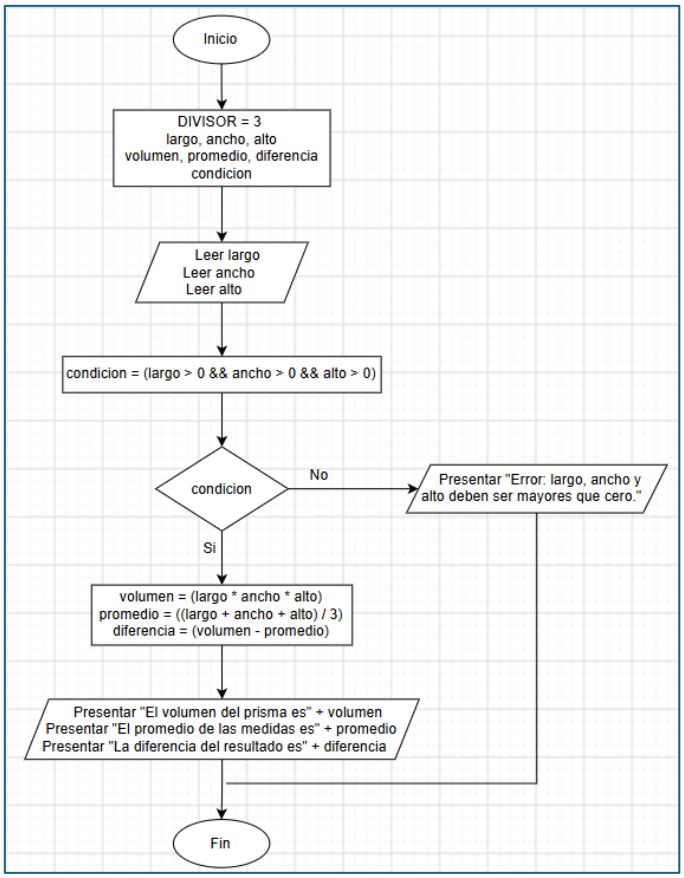


Figura 110: diagrama de flujo del ejercicio 501 en Draw.io

20.3. Pseudocódigo

```
Inicio

DIVISOR = 3;
largo, ancho, alto;
volumen, promedio, diferencia;
condicion;

Leer largo;
Leer ancho;
Leer alto;

condicion = (largo > 0 && ancho > 0 && alto > 0);

Si (condicion) entonces
volumen = (largo * ancho * alto);
promedio = ((largo + ancho + alto) / 3);
diferencia = (volumen - promedio);

Presentar "El volumen del prisma es" + volumen;
Presentar "El promedio de las medidas es" + promedio;
Presentar "La diferencia del resultado es" + diferencia;
Sino
Presentar "Error: largo, ancho y alto deben ser mayores que cero.";
Fin Si

Fin
```

Figura 111: pseudocódigo del ejercicio 501

20.4. Prueba de escritorio

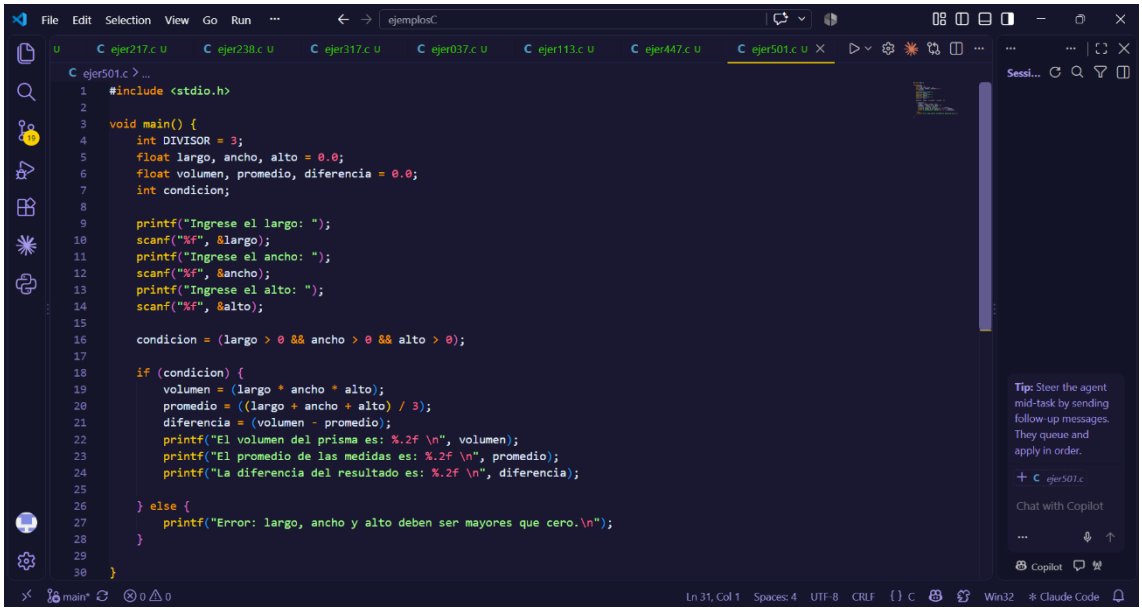
Prueba 1: Medidas válidas

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	largo ancho alto		Ingrese el largo: 2 Ingrese el ancho: 3 Ingrese el alto: 4
2	condicion	(largo > 0 && ancho > 0 && alto > 0) (2 > 0 && 3 > 0 && 4 > 0) = TRUE	
3	volumen	(largo * ancho * alto) (2 * 3 * 4) = 24	
4	promedio	((largo + ancho + alto) / 3) ((2 + 3 + 4) / 3) = 3	
5	diferencia	(volumen - promedio) (24 – 3) = 21	
6	Presentar		El volumen del prisma es: 24.00 El promedio de las medidas es: 3.00 La diferencia del resultado es: 21.00

Prueba 2: Medidas no válidas

Paso	Variables	Evaluación de condiciones	Salida en pantalla
1	largo ancho alto		Ingrese el largo: 0 Ingrese el ancho: 9 Ingrese el alto: 5.5
2	condicion	(largo > 0 && ancho > 0 && alto > 0) (0 > 0 && 9 > 0 && 5.5 > 0) = FALSE	
3	Presentar		Error: largo, ancho y alto deben ser mayores que cero.

20.5. Codificación en lenguaje C



```c
#include <stdio.h>

void main() {
    int DIVISOR = 3;
    float largo, ancho, alto = 0.0;
    float volumen, promedio, diferencia = 0.0;
    int condicion;

    printf("Ingrese el largo: ");
    scanf("%f", &largo);
    printf("Ingrese el ancho: ");
    scanf("%f", &ancho);
    printf("Ingrese el alto: ");
    scanf("%f", &alto);

    condicion = (largo > 0 && ancho > 0 && alto > 0);

    if (condicion) {
        volumen = (largo * ancho * alto);
        promedio = ((largo + ancho + alto) / 3);
        diferencia = (volumen - promedio);
        printf("El volumen del prisma es: %.2f \n", volumen);
        printf("El promedio de las medidas es: %.2f \n", promedio);
        printf("La diferencia del resultado es: %.2f \n", diferencia);

    } else {
        printf("Error: largo, ancho y alto deben ser mayores que cero.\n");
    }
}
```

Figura 112: código del ejercicio 501, variables, proceso y salida

20.6. Salida del programa

```
Problems  Output  Debug Console  Terminal  Ports

Microsoft Windows [Versión 10.0.26300.9711]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>gcc ejer501.c -o ejer501

C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>ejer501
Ingrese el largo: 2
Ingrese el ancho: 3
Ingrese el alto: 4
El volumen del prisma es: 24.00
El promedio de las medidas es: 3.00
La diferencia del resultado es: 21.00
```

Figura 113: comprobación de la prueba 1

```
C:\Users\Usuario iTC\Documents\python_project\ciclo1_2026-2027\ejemplosC>ejer501
Ingrese el largo: 0
Ingrese el ancho: 9
Ingrese el alto: 5.5
Error: largo, ancho y alto deben ser mayores que cero.
```

Figura 114: comprobación de la prueba 2

4. Conclusiones de los algoritmos

Gracias al desarrollo de estos algoritmos aprendí a diseñar diagramas de flujo que explican la estructura mediante figuras geométricas y flechas, que ayudan a detectar errores antes de trasladarlo a un algoritmo.

También logré escribir los pseudocódigos que me ayudaron a elaborar y ordenar los pasos del algoritmo antes de escribirlo en el código real. Al comienzo me resultaron un poco complicados ya que tenían una diferente estructura, pero con forme iba realizando los otros ejercicios pude comprender mejor.

Además, las pruebas de escritorio fueron muy indispensables en mi trabajo, ya que con estas logre asegurar que las soluciones propuestas eran correctas.

Por último, aprendí a trasladar de pseudocódigo a código en lenguaje C con las librerías necesarias, la declaración de variables, la estructura de entrada, proceso y salida. A cada algoritmo lo compile y ejecute con los datos de las pruebas de escritorio para verificar los resultados, los cuales eran correctos.

5. Anexos

https://github.com/brigithtoledo2/ciclo1_2026-2027/tree/main/trabajofinal_U1

Declaración de autoría

Declaro que el presente trabajo es de mi **autoría**, que **no se utilizó inteligencia artificial** (ni otra herramienta generadora de contenido) ni copia de terceros para su elaboración, y que conozco y acepto las sanciones establecidas en el reglamento institucional en caso de incumplimiento.



Firma del estudiante **Docente**